مستندات یکپارچه‌سازی شریک فان‌لندز

این راهنما برای تیم فنی اپ‌هاییه که بازی‌های فان‌لندز رو داخل محصول خودشون نشون میدن. هر چیزی که برای ورود کاربر، گرفتن رویدادها و گرفتن گزارش لازم دارید همین‌جاست، همراه با نمونه‌کد آماده کپی برای ده پشته.

۱ معرفی

کاربری که توی اپ شما وارد شده، بدون فرم ثبت‌نام و رمز تازه وارد بازی فان‌لندز میشه. شما هویتش رو با یک توکن کوتاه‌عمر که روی سرور خودتون امضا کردید به فان‌لندز معرفی می‌کنید. فان‌لندز امضا رو با کلید عمومی شما چک می‌کنه، بازیکن رو پیدا می‌کنه یا می‌سازه و نشست بازی رو باز می‌کنه.

نمایش بازی در اپ شما

بازی داخل یک `iframe` توی صفحه شما اجرا میشه. نسخه اول برای وبه و اپ موبایل و TV مسیر جدایی لازم دارن که هنوز منتشر نشده.

ورود یکپارچه با launch امضاشده

سرور شما یک JWT با `typ` مخصوص امضا می‌کنه و صفحه اون رو با یک فرم به فان‌لندز می‌فرسته. توکن هیچ‌وقت توی URL نمیره.

رویدادها و Webhook

شروع نشست، پایان بازی، حذف حساب و بقیه رویدادهای بازیکن به‌شکل SET امضاشده به نشانی ثبت‌شده شما می‌رسن. اگه چیزی جا موند، از API بازیابی می‌گیریدش.

API گزارش‌ها

آمار روزانه، آمار هر بازی و فعالیت هر بازیکن خودتون رو با یک توکن سرور به سرور می‌خونید. هر پاسخ زمان ساخت و تاخیر داده رو هم میگه.

مدل اعتماد

هر طرف با کلید خصوصی خودش امضا می‌کنه و کلیدهای عمومیش رو توی یک JWKS روی HTTPS منتشر می‌کنه. طرف مقابل کلید رو فقط از نشانی‌ای می‌خونه که موقع ثبت اعلام شده. هیچ کلید خصوصی، رمز مشترک یا کلید HMAC بین دو طرف جابه‌جا نمیشه.

توکن	امضاکننده	چک‌کننده	منبع کلید
launch JWT	شما	فان‌لندز	JWKS ثبت‌شده شما
logout token	شما	فان‌لندز	JWKS ثبت‌شده شما
client assertion (API گزارش)	شما	فان‌لندز	JWKS ثبت‌شده شما
SET (رویداد Webhook)	فان‌لندز	شما	JWKS فان‌لندز
bootstrap code و refresh token	فان‌لندز	فان‌لندز	مقدار تصادفیه و JWKS نداره

شناسه شریک (`partner_id`) همیشه از توکن یا کلید تاییدشده خونده میشه و هیچ پارامتر URL، هدر یا فیلد بدنه‌ای نمی‌تونه عوضش کنه. برای درخواست‌های بازیکن، همون access tokenی که bootstrap صادر می‌کنه شریک رو مشخص می‌کنه (فضای شریک در صفحه بازی).

محیط‌ها و نشانی‌ها

دو محیط داریم: `sandbox` برای توسعه و تست با داده غیرواقعی، و `production` برای کاربرهای واقعی. این دو کاملا از هم جدا هستن: کلید، audience، issuer، `client_id` ، نشانی Webhook و داده هر کدوم مستقله. توکنی که برای یک محیط امضا شده توی محیط دیگه رد میشه. یک

`sub` یکسان توی sandbox و production دو بازیکن جدا در فان‌لندزه و داده بازی و گزارش‌هاشون هیچ‌وقت قاطی نمیشه.

مورد	sandbox	production
نشانی پایه API (Base URL)	`https://api.sandbox.funlands.example`	`https://api.funlands.example`
Origin بازی (`FUNLANDS_GAME_ORIGIN`)	`https://play.sandbox.funlands.example`	`https://play.funlands.example`
JWKS فان‌لندز	`https://api.sandbox.funlands.example/.well-known/jwks.json`	`https://api.funlands.example/.well-known/jwks.json`
issuer فان‌لندز (روی SET و توکن گزارش)	`https://api.sandbox.funlands.example`	`https://api.funlands.example`
audience توکن launch	`https://api.sandbox.funlands.example/partner-launch`	`https://api.funlands.example/partner-launch`
audience توکن خروج	`https://api.sandbox.funlands.example/partner-logout`	`https://api.funlands.example/partner-logout`
token endpoint (audience در client assertion)	`https://api.sandbox.funlands.example/oauth/token`	`https://api.funlands.example/oauth/token`
پیشوند نوع رویداد	`https://api.funlands.example/events/`، توی هر دو محیط یکسان	
کلیدهای شما	جفت‌کلید مخصوص sandbox	جفت‌کلید جدا. کلید sandbox رو هیچ‌وقت اینجا استفاده نکنید.
داده	آزمایشی، بدون اطلاعات کاربر واقعی	واقعی

همه نشانی‌های این صفحه، مثل `api.sandbox.funlands.example`، `play.sandbox.funlands.example` و `partner.example`، نمونه‌ان و نشانی واقعی هیچ سرویسی نیستن. کلیدها، `client_id`ها و scopeهای نمونه‌کدها هم نمونه‌ان. نشانی‌ها، کلیدها، `client_id` و scopeهای واقعی هر محیط فقط از برگ شریک میان، از جمله issuer، نشانی JWKS، Origin بازی، پیشوند نوع رویداد و سه audience (launch، خروج و token endpoint). همون مقدارها رو عینا بذارید. این سه audience با هم فرق دارن تا توکن یک نوع هیچ‌وقت به‌جای نوع دیگه قبول نشه. مقایسه audience حرف‌به‌حرفه و یک `/` اضافه در انتها هم توکن رو رد می‌کنه. issuer فان‌لندز همون نشانی پایه‌ایه که این audienceها روش ساخته شدن، یعنی token endpoint بدون `/oauth/token`.

دو مقدار جدا دارید. نشانی پایه API (`FUNLANDS_BASE`) برای JWKS فان‌لندز، `/oauth/token`، API گزارش، feed رویداد و خروج سرور به سرور به کار میره. Origin بازی (`FUNLANDS_GAME_ORIGIN`) جاییه که صفحه handoff و صفحه بازی روش باز میشن: فرم launch به همین Origin میره، bootstrap فقط درخواستی رو قبول می‌کنه که `Origin`ش دقیقا همین مقدار باشه، و `frame-src`، `form-action` و چک `postMessage` صفحه شما هم باید همین مقدار رو داشته باشن. توی بعضی استقرارها این دو یکی هستن، ولی همیشه مقدار برگ شریک رو بذارید.

نسخه‌بندی

- نسخه توی مسیر میاد: `/v1/...` برای ورود و نشست، و `/partner/v1/...` برای APIهای سرور به سرور.
- تا وقتی `v1` هست فقط تغییرهای سازگار منتشر میشن: فیلد اختیاری تازه در پاسخ، نوع رویداد تازه و کد خطای تازه. کد شما باید فیلد ناشناخته رو نادیده بگیره، رویداد ناشناخته رو ذخیره کنه و باز هم `202` برگردونه، و با کد خطای ناشناخته مثل یک خطای عمومی رفتار کنه.
- تغییر ناسازگار فقط با نسخه تازه مسیر (`v2`) میاد. قبلش خبر میدیم و تا آخر دوره گذار هر دو نسخه کار می‌کنن.
- هر رویداد `schema_version` داره و هر پاسخ گزارش `metric_definition_version`. اگه روش محاسبه یک شاخص عوض بشه این عدد بالا میره، پس عددهای دو نسخه مختلف رو مستقیم با هم مقایسه نکنید.

فهرست endpointها

Method و مسیر	کی صدا می‌زنه	احراز هویت
`GET /.well-known/jwks.json`	سرور شما	عمومی
`POST /v1/partner/launch` (روی Origin بازی)	مرورگر کاربر (فرم در iframe)	launch JWT + Origin
`POST /v1/session/bootstrap` (روی Origin بازی)	صفحه handoff فان‌لندز	code + verifier + Origin بازی
`POST /v1/auth/session/refresh`	صفحه بازی	refresh token
APIهای بازی (HTTP و WebSocket)	صفحه بازی	فقط Bearer access token بازی (در WebSocket با `auth.token`)
`POST /partner/v1/backchannel-logout`	سرور شما	logout token
`POST /oauth/token`	سرور شما	client assertion (`private_key_jwt`)
`GET /partner/v1/metrics/summary`، `/metrics/daily`، `/games`	سرور شما	Bearer با `reports:read`
`GET /partner/v1/players/{external_sub}` و `/activity`	سرور شما	Bearer با `players:read`
`GET /partner/v1/events?cursor=`	سرور شما	Bearer با `events:read`
`POST` به نشانی Webhook شما	فان‌لندز	SET امضاشده با کلید فان‌لندز

۲ شروع سریع

این پنج قدم رو به ترتیب توی sandbox انجام بدید. هر قدم به بخش کامل خودش لینک داره.

۱ کلید بسازید

یک جفت‌کلید `ES256` (منحنی P-256) برای امضای launch و خروج بسازید، و بهتره یک جفت دیگه هم برای API گزارش. کلید خصوصی توی KMS یا HSM شما می‌مونه و بیرون نمیاد. الزامات کلید

۲ JWKS رو منتشر کنید

کلیدهای عمومی رو روی نشانی‌ای مثل `https://partner.example/.well-known/jwks.json` منتشر کنید، فقط با فیلدهای عمومی و بدون `d` ، `p` یا `q` . انتشار JWKS

۳ توی فان‌لندز ثبت بشید

برگ شریک رو برای هر محیط بفرستید: issuer، نشانی JWKS، Originهای وب، نشانی Webhook، قالب `sub` ، منطقه زمانی و راه تماس اضطراری. در جواب `partner_id` ، `client_id` ، scopeها، audienceها، نشانی JWKS فان‌لندز و Origin بازی رو می‌گیرید.

۴ launch رو پیاده کنید

سرور شما برای هر ورود یک verifier تازه می‌سازه، توکن launch رو امضا می‌کنه و صفحه‌ای برمی‌گردونه که فرم رو خودکار داخل iframe می‌فرسته. ورود بازیکن

۵ Webhook رو راه بندازید

یک endpoint بسازید که SET رو چک کنه، رویداد رو با کلید `event_id` ماندگار ذخیره کنه و تا ۵ ثانیه `202` بده. رویدادها

نمونه برگ شریک: چیزهایی که می‌فرستید و چیزهایی که می‌گیرید

YAML

```yaml
# What you send for each environment (sandbox shown)
partner_name: Partner Example
environment: sandbox
issuer: https://partner.example
jwks_uri: https://partner.example/.well-known/jwks.json
signing_algorithm: ES256
allowed_web_origins:
  - https://partner.example
webhook_uri: https://partner.example/funlands/events
subject_format: "opaque, pairwise, max 128 chars"
timezone: Asia/Tehran
incident_contact: security@partner.example

# What Funlands returns for the same environment
# The three audiences are per environment and compared byte for byte; use exactly these values.
partner_id: ptr_sandbox_0000example
client_id: cli_sandbox_0000example
scopes: [reports:read, players:read, events:read]
launch_audience: https://api.sandbox.funlands.example/partner-launch
logout_audience: https://api.sandbox.funlands.example/partner-logout
token_audience: https://api.sandbox.funlands.example/oauth/token
token_endpoint: https://api.sandbox.funlands.example/oauth/token
funlands_issuer: https://api.sandbox.funlands.example
funlands_jwks_uri: https://api.sandbox.funlands.example/.well-known/jwks.json
game_origin: https://play.sandbox.funlands.example   # FUNLANDS_GAME_ORIGIN: launch form, iframe, postMessage
event_type_prefix: https://api.funlands.example/events/   # fixed, identical in sandbox and production
```

```
partner_api_key: <delivered once>          # optional, client_direct only: for calls without a player token (OTP, l
ogin, guest, public reads)
set_audience: https://partner.example   # the aud of every SET: your issuer
```

مقدارهای نمونه در کدها

همه نمونه‌کدها با این مقدارها نوشته شدن. موقع کپی، اون‌ها رو با مقدارهای برگ شریک خودتون عوض کنید.

نام	مقدار نمونه
نشانی پایه API (`FUNLANDS_BASE`)	`https://api.sandbox.funlands.example`
Origin بازی (`FUNLANDS_GAME_ORIGIN`)	`https://play.sandbox.funlands.example`
issuer شما	`https://partner.example`
kid کلید launch و خروج	`partner-sandbox-2026-11`
kid کلید API گزارش	`partner-api-sandbox-2026-11`
client_id	`cli_sandbox_0000example`
نشانی Webhook شما	`https://partner.example/funlands/events`
audience رویدادها (`aud` در SET)	`https://partner.example`
شناسه بازی	`star-runner`

نصب کتابخونه‌ها

زبانی که اینجا انتخاب کنید توی همه نمونه‌کدهای صفحه اعمال میشه و دفعه بعد هم یادش می‌مونه.

NestJS

```
# Node.js 20+ ; jose 5.x or 6.x
npm install jose pg
npm install --save-dev @types/pg
```

Express

```
# Node.js 20+ ; jose 5.x or 6.x
npm install express jose pg
```

FastAPI

```
# Python 3.11+
pip install "fastapi[standard]" "PyJWT[crypto]>=2.8" cryptography httpx "psycopg[binary,pool]"
```

Django

```
# Python 3.11+ ; Django 4.2+
pip install "django>=4.2" "PyJWT[crypto]>=2.8" cryptography requests
```

Laravel

```
# PHP 8.2+ with ext-openssl ; Laravel 11+
composer require firebase/php-jwt:^6.10
```

PHP

```
# PHP 8.2+ with ext-openssl, ext-pdo_pgsql
composer require firebase/php-jwt:^6.10 guzzlehttp/guzzle:^7.8
```

Go

```
# Go 1.22+
go get github.com/lestrrat-go/jwx/v2@latest
go get github.com/jackc/pgx/v5@latest
```

Spring Boot

```
<!-- pom.xml ; Spring Boot 3.2+ , Java 17+ -->
<dependency>
  <groupId>com.nimbusds</groupId>
  <artifactId>nimbus-jose-jwt</artifactId>
  <version>10.0.2</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
```

ASP.NET Core

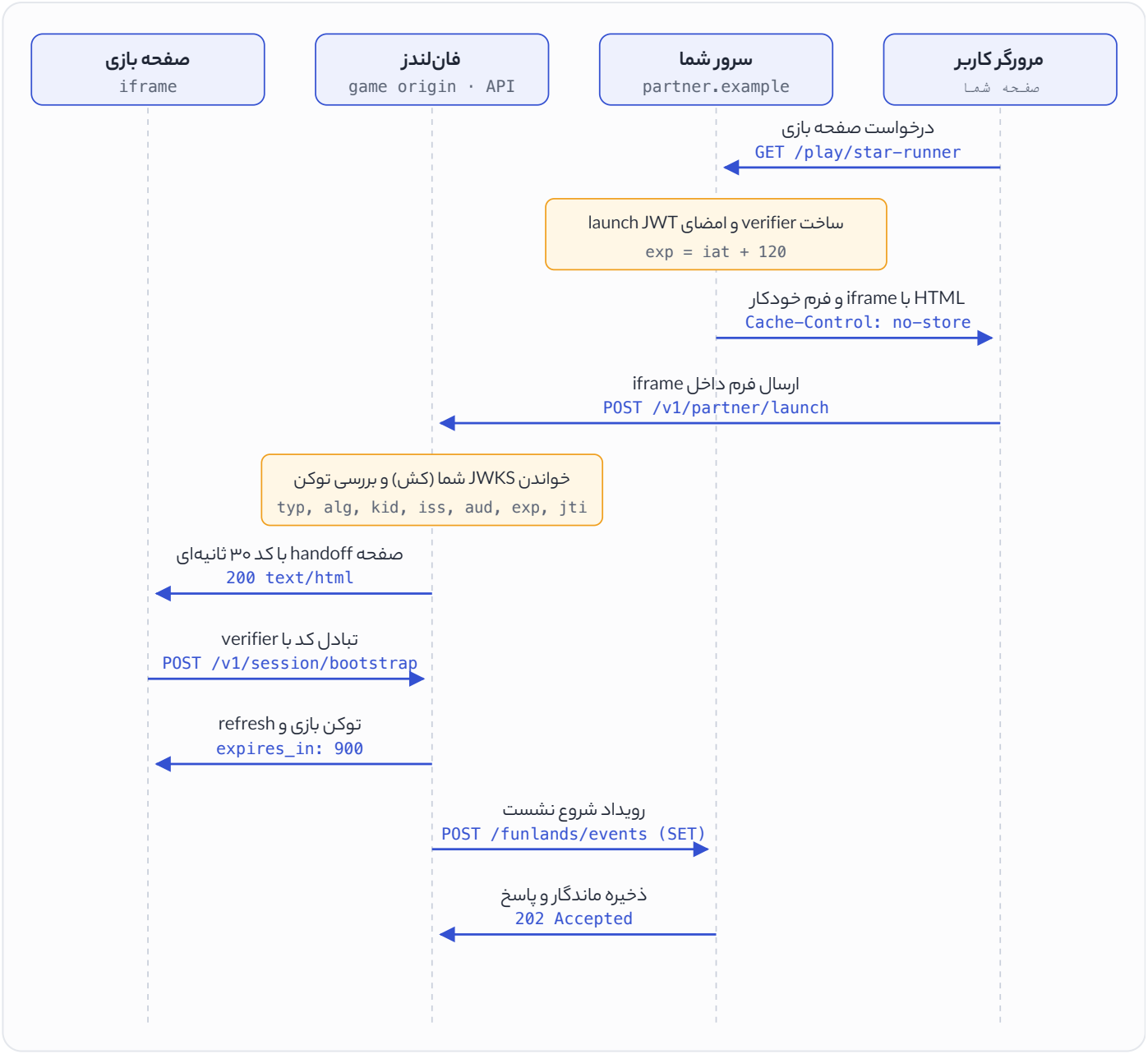
```
# .NET 8+
dotnet add package Microsoft.IdentityModel.JsonWebTokens
dotnet add package Npgsql
```

cURL

```
# macOS
brew install curl jq openssl@3
# Debian / Ubuntu
sudo apt-get install -y curl jq openssl
```

نمودار جریان ورود

از کلیک کاربر روی بازی تا اولین رویداد Webhook:


مرورگر کاربر
صفحه شما
سرور شما
partner.example
فان‌لندز
game origin · API
صفحه بازی
iframe
درخواست صفحه بازی
GET /play/star-runner
ساخت verifier و امضای launch JWT
exp = iat + 120
HTML با iframe و فرم خودکار
Cache-Control: no-store
ارسال فرم داخل iframe
POST /v1/partner/launch
خواندن JWKS شما (کش) و بررسی توکن
typ, alg, kid, iss, aud, exp, jti
صفحه handoff با کد ۳۰ ثانیه‌ای
200 text/html
تبادل کد با verifier
POST /v1/session/bootstrap
توکن بازی و refresh
expires_in: 900
رویداد شروع نشست
POST /funlands/events (SET)
ذخیره ماندگار و پاسخ
202 Accepted

۳ کلیدها و JWKS

الزامات کلید

- برای هر محیط یک الگوریتم ثبت میشه و فان‌لندز فقط همون رو قبول می‌کنه، نه الگوریتمی که توی هدر توکن اومده. پیشنهاد ما `ES256` هست. `RS256` و `PS256` هم با کلید RSA دست‌کم ۲۰۴۸ بیتی قبول میشن. `none` و کل خانواده `HS*` همیشه رد میشن.
- هر کلید یک `kid` غیرخالی داره که توی issuer و محیط شما یکتاست. `kid` کلیدی که بازنشسته شده رو هیچ‌وقت برای کلید تازه استفاده نکنید. قالب پیشنهادی: `<name>-<env>-<yyyy-mm>`، مثلا `partner-sandbox-2026-11`.
- هر کلید تازه رو دست‌کم ۲۴ ساعت قبل از اولین امضا توی JWKS منتشر کنید و هر کلید بازنشسته رو دست‌کم ۲۴ ساعت بعد از آخرین امضا توی JWKS نگه دارید.
- کلید خصوصی توی KMS یا HSM شما ساخته و نگه داشته میشه. نمونه‌کدها برای ساده بودن فایل PEM می‌خونن. توی production امضا رو به KMS بسپارید.
- کلیدهای sandbox و production جدا هستن. برای API گزارش کلید جدا با `kid` جدا داشته باشید تا لو رفتن یکی روی اون یکی اثر نذاره.
- هدرهای `jku`، `x5u`، `jwk`، `x5c` و `crit` رو توی توکن ندارید. فان‌لندز کلید رو فقط از نشانی ثبت‌شده می‌خونه و توکنی که این هدرها رو داره با `TOKEN_TYPE_INVALID` رد میشه.

انتشار JWKS شما

JWKS یک سند JSON عمومیه، به این شکل:

JSON

```
{
  "keys": [
    {
      "kty": "EC",
      "crv": "P-256",
      "x": "<base64url-x-coordinate-43-chars>",
      "y": "<base64url-y-coordinate-43-chars>",
      "kid": "partner-sandbox-2026-11",
      "alg": "ES256",
      "use": "sig"
    }
  ]
}
```

فان‌لندز JWKS شما رو این‌طوری می‌خونه:

- فقط نشانی ثبت‌شده، فقط HTTPS و بدون دنبال کردن ریدایرکت. نشانی باید به IP عمومی برسه و نشانی‌های داخلی، loopback و link-local رد میشن.
- timeout سه ثانیه‌ست، پاسخ حداکثر ۶۴ کیلوبایت و حداکثر ۲۰ کلید.
- نتیجه ۵ دقیقه کش میشه. اگه `kid` توکن توی کش نباشه، JWKS یک بار دیگه خونده میشه (برای هر محیط حداکثر یک بار در هر ۳۰ ثانیه) و اگه باز هم نبود، توکن با `KEY_NOT_FOUND` رد میشه.
- اگه JWKS شما در دسترس نباشه، فقط کلیدهایی که قبلا معتبر و توی کش بودن تا حداکثر یک ساعت استفاده میشن. کلید ناشناخته توی این حالت هیچ‌وقت قبول نمیشه.

نمونه: ساخت جفت‌کلید و انتشار JWKS

NestJS

```ts
// scripts/generate-partner-key.ts
// Run once per new key:  npx ts-node scripts/generate-partner-key.ts partner-sandbox-2026-11
import { exportPKCS8, exportSPKI, generateKeyPair } from 'jose';
import { writeFile } from 'node:fs/promises';

async function main() {
  const kid = process.argv[2] ?? 'partner-sandbox-2026-11';
  const { privateKey, publicKey } = await generateKeyPair('ES256', { extractable: true });
  // Private key: move it into your KMS / secret store right after generation.
  await writeFile(`keys/${kid}.pem`, await exportPKCS8(privateKey), { mode: 0o600 });
  await writeFile(`keys/${kid}.pub.pem`, await exportSPKI(publicKey));
}
void main();

// src/funlands/partner-jwks.service.ts
import { Injectable, OnModuleInit } from '@nestjs/common';
import { exportJWK, importSPKI } from 'jose';
import { readFile } from 'node:fs/promises';

// Every key that may still verify a token: the active key plus keys inside
// their 24 h publish-before-use or keep-after-retire window.
const PUBLISHED_KIDS = ['partner-sandbox-2026-11', 'partner-api-sandbox-2026-11'];

@Injectable()
export class PartnerJwksService implements OnModuleInit {
  private keys: Record<string, string>[] = [];

  async onModuleInit() {
    this.keys = await Promise.all(
      PUBLISHED_KIDS.map(async (kid) => {
        const pem = await readFile(`keys/${kid}.pub.pem`, 'utf8');
        const jwk = await exportJWK(await importSPKI(pem, 'ES256', { extractable: true }));
        // Copy public members only. Never publish d, p, q or any private field.
        return { kty: jwk.kty!, crv: jwk.crv!, x: jwk.x!, y: jwk.y!, kid, alg: 'ES256', use: 'sig' };
      }),
    );
  }

  document() {
    return { keys: this.keys };
  }
}

// src/funlands/jwks.controller.ts
import { Controller, Get, Header } from '@nestjs/common';
import { PartnerJwksService } from './partner-jwks.service';

@Controller('.well-known')
export class JwksController {
  constructor(private readonly jwks: PartnerJwksService) {}

  @Get('jwks.json')
  @Header('Cache-Control', 'public, max-age=300')
  get() {
    return this.jwks.document();
  }
}
```

Express

```ts
// generate-partner-key.mjs
// Run once per new key:  node generate-partner-key.mjs partner-sandbox-2026-11
import { exportPKCS8, exportSPKI, generateKeyPair } from 'jose';
import { writeFile } from 'node:fs/promises';

const kid = process.argv[2] ?? 'partner-sandbox-2026-11';
const { privateKey, publicKey } = await generateKeyPair('ES256', { extractable: true });
await writeFile(`keys/${kid}.pem`, await exportPKCS8(privateKey), { mode: 0o600 }); // then move into KMS
await writeFile(`keys/${kid}.pub.pem`, await exportSPKI(publicKey));
```

```mjs
// jwks.mjs
import express from 'express';
import { exportJWK, importSPKI } from 'jose';
import { readFile } from 'node:fs/promises';

const PUBLISHED_KIDS = ['partner-sandbox-2026-11', 'partner-api-sandbox-2026-11'];

const keys = await Promise.all(
  PUBLISHED_KIDS.map(async (kid) => {
    const pem = await readFile(`keys/${kid}.pub.pem`, 'utf8');
    const { kty, crv, x, y } = await exportJWK(await importSPKI(pem, 'ES256', { extractable: true }));
    return { kty, crv, x, y, kid, alg: 'ES256', use: 'sig' }; // public members only
  }),
);

const app = express();
app.get('/.well-known/jwks.json', (req, res) => {
  res.set('Cache-Control', 'public, max-age=300').json({ keys });
});
app.listen(3000);
```

FastAPI

```python
# generate_partner_key.py
# Run once per new key:  python generate_partner_key.py partner-sandbox-2026-11
import sys
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec

kid = sys.argv[1] if len(sys.argv) > 1 else "partner-sandbox-2026-11"
key = ec.generate_private_key(ec.SECP256R1())

private_path = Path(f"keys/{kid}.pem")
private_path.touch(mode=0o600)  # then move into your KMS / secret store
private_path.write_bytes(key.private_bytes(
    serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8, serialization.NoEncryption()))
Path(f"keys/{kid}.pub.pem").write_bytes(key.public_key().public_bytes(
    serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo))


# app/jwks.py
import json
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from fastapi import FastAPI, Response
from jwt.algorithms import ECAlgorithm

PUBLISHED_KIDS = ["partner-sandbox-2026-11", "partner-api-sandbox-2026-11"]


def public_jwk(kid: str) -> dict:
    public_key = serialization.load_pem_public_key(Path(f"keys/{kid}.pub.pem").read_bytes())
    jwk = json.loads(ECAlgorithm.to_jwk(public_key))
    # Public members only.
    return {"kty": jwk["kty"], "crv": jwk["crv"], "x": jwk["x"], "y": jwk["y"],
            "kid": kid, "alg": "ES256", "use": "sig"}


JWKS = {"keys": [public_jwk(kid) for kid in PUBLISHED_KIDS]}
app = FastAPI()


@app.get("/.well-known/jwks.json")
def jwks(response: Response) -> dict:
    response.headers["Cache-Control"] = "public, max-age=300"
    return JWKS
```

Django

```python
# partner/management/commands/generate_partner_key.py
# Run once per new key:  python manage.py generate_partner_key partner-sandbox-2026-11
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec
from django.conf import settings
from django.core.management.base import BaseCommand


class Command(BaseCommand):
    help = "Generate an ES256 (P-256) key pair for Funlands"

    def add_arguments(self, parser):
        parser.add_argument("kid")

    def handle(self, *args, kid, **options):
        key = ec.generate_private_key(ec.SECP256R1())
        private_path = Path(settings.FUNLANDS_KEY_DIR) / f"{kid}.pem"
        private_path.touch(mode=0o600)  # then move into your KMS / secret store
        private_path.write_bytes(key.private_bytes(
            serialization.Encoding.PEM, serialization.PrivateFormat.PKCS8, serialization.NoEncryption()))
        (Path(settings.FUNLANDS_KEY_DIR) / f"{kid}.pub.pem").write_bytes(key.public_key().public_bytes(
            serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo))


# partner/views.py
import json
from functools import cache
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from django.conf import settings
from django.http import JsonResponse
from django.views.decorators.http import require_GET
from jwt.algorithms import ECAlgorithm

PUBLISHED_KIDS = ["partner-sandbox-2026-11", "partner-api-sandbox-2026-11"]


@cache
def published_jwks() -> dict:
    keys = []
    for kid in PUBLISHED_KIDS:
        pem = (Path(settings.FUNLANDS_KEY_DIR) / f"{kid}.pub.pem").read_bytes()
        jwk = json.loads(ECAlgorithm.to_jwk(serialization.load_pem_public_key(pem)))
        keys.append({"kty": jwk["kty"], "crv": jwk["crv"], "x": jwk["x"], "y": jwk["y"],
                     "kid": kid, "alg": "ES256", "use": "sig"})
    return {"keys": keys}


@require_GET
def jwks(request):
    response = JsonResponse(published_jwks())
    response["Cache-Control"] = "public, max-age=300"
    return response


# partner/urls.py
from django.urls import path
from . import views

urlpatterns = [path(".well-known/jwks.json", views.jwks)]
```

Laravel

```php
<?php
// routes/console.php
// Run once per new key:  php artisan funlands:generate-key partner-sandbox-2026-11
```

```php
use Illuminate\Support\Facades\Artisan;

Artisan::command('funlands:generate-key {kid}', function (string $kid) {
    $key = openssl_pkey_new(['private_key_type' => OPENSSL_KEYTYPE_EC, 'curve_name' => 'prime256v1']);
    openssl_pkey_export($key, $privatePem);                 // PKCS#8 PEM
    $publicPem = openssl_pkey_get_details($key)['key'];      // SPKI PEM

    $dir = storage_path('keys');
    file_put_contents("{$dir}/{$kid}.pem", $privatePem);     // then move into your KMS / secret store
    chmod("{$dir}/{$kid}.pem", 0600);
    file_put_contents("{$dir}/{$kid}.pub.pem", $publicPem);
    $this->info("created {$kid}");
});

// app/Http/Controllers/JwksController.php
namespace App\Http\Controllers;

use Illuminate\Http\JsonResponse;

final class JwksController
{
    private const PUBLISHED_KIDS = ['partner-sandbox-2026-11', 'partner-api-sandbox-2026-11'];

    public function __invoke(): JsonResponse
    {
        $keys = array_map(fn (string $kid) => self::publicJwk($kid), self::PUBLISHED_KIDS);

        return response()->json(['keys' => $keys])->header('Cache-Control', 'public, max-age=300');
    }

    private static function publicJwk(string $kid): array
    {
        $public = openssl_pkey_get_public(file_get_contents(storage_path("keys/{$kid}.pub.pem")));
        $ec = openssl_pkey_get_details($public)['ec'];

        return [
            'kty' => 'EC',
            'crv' => 'P-256',
            'x' => self::b64url(str_pad($ec['x'], 32, "\0", STR_PAD_LEFT)), // coordinates are fixed 32 bytes
            'y' => self::b64url(str_pad($ec['y'], 32, "\0", STR_PAD_LEFT)),
            'kid' => $kid,
            'alg' => 'ES256',
            'use' => 'sig',
        ];
    }

    private static function b64url(string $bytes): string
    {
        return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
    }
}

// routes/web.php
Route::get('/.well-known/jwks.json', \App\Http\Controllers\JwksController::class);
```

PHP

```php
<?php
// bin/generate-key.php
// Run once per new key:  php bin/generate-key.php partner-sandbox-2026-11
$kid = $argv[1] ?? 'partner-sandbox-2026-11';
$key = openssl_pkey_new(['private_key_type' => OPENSSL_KEYTYPE_EC, 'curve_name' => 'prime256v1']);
openssl_pkey_export($key, $privatePem);
file_put_contents(__DIR__ . "/../keys/{$kid}.pem", $privatePem);   // then move into your KMS
chmod(__DIR__ . "/../keys/{$kid}.pem", 0600);
file_put_contents(__DIR__ . "/../keys/{$kid}.pub.pem", openssl_pkey_get_details($key)['key']);

// public/jwks.php  (rewrite /.well-known/jwks.json to this file in nginx/Apache)
<?php
declare(strict_types=1);
```

```php
const PUBLISHED_KIDS = ['partner-sandbox-2026-11', 'partner-api-sandbox-2026-11'];

function b64url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

$keys = [];
foreach (PUBLISHED_KIDS as $kid) {
    $public = openssl_pkey_get_public(file_get_contents(__DIR__ . "/../keys/{$kid}.pub.pem"));
    $ec = openssl_pkey_get_details($public)['ec'];
    $keys[] = [
        'kty' => 'EC',
        'crv' => 'P-256',
        'x' => b64url(str_pad($ec['x'], 32, "\0", STR_PAD_LEFT)),
        'y' => b64url(str_pad($ec['y'], 32, "\0", STR_PAD_LEFT)),
        'kid' => $kid,
        'alg' => 'ES256',
        'use' => 'sig',
    ];
}

header('Content-Type: application/json');
header('Cache-Control: public, max-age=300');
echo json_encode(['keys' => $keys], JSON_UNESCAPED_SLASHES);
```

Go

```go
// cmd/genkey/main.go
// Run once per new key:  go run ./cmd/genkey partner-sandbox-2026-11
package main

import (
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/x509"
    "encoding/pem"
    "os"
)

func main() {
    kid := "partner-sandbox-2026-11"
    if len(os.Args) > 1 {
        kid = os.Args[1]
    }
    priv, err := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    must(err)
    der, err := x509.MarshalPKCS8PrivateKey(priv)
    must(err)
    // Private key: move it into your KMS / secret store right after generation.
    must(os.WriteFile("keys/"+kid+".pem", pem.EncodeToMemory(&pem.Block{Type: "PRIVATE KEY", Bytes: der}), 0o600))
    pub, err := x509.MarshalPKIXPublicKey(&priv.PublicKey)
    must(err)
    must(os.WriteFile("keys/"+kid+".pub.pem", pem.EncodeToMemory(&pem.Block{Type: "PUBLIC KEY", Bytes: pub}), 0o64
4))
}

func must(err error) {
    if err != nil {
        panic(err)
    }
}

// jwks.go
package partner

import (
    "encoding/json"
    "net/http"
    "os"
```

```
    "github.com/lestrrat-go/jwx/v2/jwa"
    "github.com/lestrrat-go/jwx/v2/jwk"
)

var publishedKIDs = []string{"partner-sandbox-2026-11", "partner-api-sandbox-2026-11"}

func LoadPublishedJWKS() (jwk.Set, error) {
    set := jwk.NewSet()
    for _, kid := range publishedKIDs {
        raw, err := os.ReadFile("keys/" + kid + ".pub.pem")
        if err != nil {
            return nil, err
        }
        key, err := jwk.ParseKey(raw, jwk.WithPEM(true)) // public PEM: no private members
        if err != nil {
            return nil, err
        }
        _ = key.Set(jwk.KeyIDKey, kid)
        _ = key.Set(jwk.AlgorithmKey, jwa.ES256)
        _ = key.Set(jwk.KeyUsageKey, jwk.ForSignature)
        if err := set.AddKey(key); err != nil {
            return nil, err
        }
    }
    return set, nil
}

func JWKSHandler(set jwk.Set) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        w.Header().Set("Cache-Control", "public, max-age=300")
        _ = json.NewEncoder(w).Encode(set)
    }
}

// main.go:  mux.Handle("GET /.well-known/jwks.json", partner.JWKSHandler(set))
```

Spring Boot

```
// GenerateKey.java  (run once per new key:  java GenerateKey.java partner-sandbox-2026-11)
import java.nio.file.*;
import java.security.*;
import java.security.spec.ECGenParameterSpec;
import java.util.Base64;

public class GenerateKey {
    public static void main(String[] args) throws Exception {
        String kid = args.length > 0 ? args[0] : "partner-sandbox-2026-11";
        KeyPairGenerator gen = KeyPairGenerator.getInstance("EC");
        gen.initialize(new ECGenParameterSpec("secp256r1"));
        KeyPair pair = gen.generateKeyPair();
        Path priv = Path.of("keys", kid + ".pem");
        Files.writeString(priv, pem("PRIVATE KEY", pair.getPrivate().getEncoded()));   // PKCS#8; move into KMS
        Files.setPosixFilePermissions(priv, java.nio.file.attribute.PosixFilePermissions.fromString("rw-------"));
        Files.writeString(Path.of("keys", kid + ".pub.pem"), pem("PUBLIC KEY", pair.getPublic().getEncoded()));
    }

    static String pem(String type, byte[] der) {
        return "-----BEGIN " + type + "-----\n"
            + Base64.getMimeEncoder(64, "\n".getBytes()).encodeToString(der)
            + "\n-----END " + type + "-----\n";
    }
}

// src/main/java/com/partner/funlands/PemKeys.java
package com.partner.funlands;

import java.nio.file.*;
import java.security.KeyFactory;
import java.security.interfaces.*;
```

```
import java.security.spec.*;
import java.util.Base64;

public final class PemKeys {
    private PemKeys() {}

    public static ECPrivateKey privateKey(String kid) throws Exception {
        return (ECPrivateKey) KeyFactory.getInstance("EC").generatePrivate(new PKCS8EncodedKeySpec(der("keys/" + ki
d + ".pem")));
    }

    public static ECPublicKey publicKey(String kid) throws Exception {
        return (ECPublicKey) KeyFactory.getInstance("EC").generatePublic(new X509EncodedKeySpec(der("keys/" + kid +
".pub.pem")));
    }

    private static byte[] der(String path) throws Exception {
        String body = Files.readString(Path.of(path))
            .replaceAll("-----(BEGIN|END) [A-Z ]+-----", "").replaceAll("\\s", "");
        return Base64.getDecoder().decode(body);
    }
}

// src/main/java/com/partner/funlands/JwksController.java
package com.partner.funlands;

import com.nimbusds.jose.JWSAlgorithm;
import com.nimbusds.jose.jwk.*;
import java.time.Duration;
import java.util.*;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@RestController
public class JwksController {
    private static final List<String> PUBLISHED_KIDS = List.of("partner-sandbox-2026-11", "partner-api-sandbox-2026
-11");
    private final Map<String, Object> document;

    public JwksController() throws Exception {
        List<JWK> keys = new ArrayList<>();
        for (String kid : PUBLISHED_KIDS) {
            keys.add(new ECKey.Builder(Curve.P_256, PemKeys.publicKey(kid))
                .keyID(kid).keyUse(KeyUse.SIGNATURE).algorithm(JWSAlgorithm.ES256).build());
        }
        this.document = new JWKSet(keys).toJSONObject(true); // true = public members only
    }

    @GetMapping(value = "/.well-known/jwks.json", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Map<String, Object>> jwks() {
        return ResponseEntity.ok()
            .cacheControl(CacheControl.maxAge(Duration.ofMinutes(5)).cachePublic())
            .body(document);
    }
}
```

ASP.NET Core

```
// Tools/GenerateKey.cs  (run once per new key)
using System.Security.Cryptography;

var kid = args.Length > 0 ? args[0] : "partner-sandbox-2026-11";
using var ec = ECDsa.Create(ECCurve.NamedCurves.nistP256);
File.WriteAllText($"keys/{kid}.pem", ec.ExportPkcs8PrivateKeyPem());      // then move into your KMS
if (!OperatingSystem.IsWindows())
    File.SetUnixFileMode($"keys/{kid}.pem", UnixFileMode.UserRead | UnixFileMode.UserWrite);
File.WriteAllText($"keys/{kid}.pub.pem", ec.ExportSubjectPublicKeyInfoPem());

// Funlands/PartnerJwks.cs
using System.Security.Cryptography;
using Microsoft.IdentityModel.Tokens;
```

```
public static class PartnerJwks
{
    private static readonly string[] PublishedKids = ["partner-sandbox-2026-11", "partner-api-sandbox-2026-11"];

    public static object Document()
    {
        var keys = PublishedKids.Select(kid =>
        {
            using var ec = ECDsa.Create();
            ec.ImportFromPem(File.ReadAllText($"keys/{kid}.pub.pem"));
            var p = ec.ExportParameters(includePrivateParameters: false);
            return new Dictionary<string, string>
            {
                ["kty"] = "EC", ["crv"] = "P-256",
                ["x"] = Base64UrlEncoder.Encode(p.Q.X!), ["y"] = Base64UrlEncoder.Encode(p.Q.Y!),
                ["kid"] = kid, ["alg"] = "ES256", ["use"] = "sig",
            };
        }).ToArray();
        return new { keys };
    }
}

// Program.cs
var jwks = PartnerJwks.Document();
app.MapGet("/.well-known/jwks.json", (HttpContext http) =>
{
    http.Response.Headers.CacheControl = "public, max-age=300";
    return Results.Json(jwks);
});
```

cURL

```
# 1) Generate an ES256 (P-256) key pair, PKCS#8 private key
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-256 -out partner-sandbox-2026-11.pem
openssl pkey -in partner-sandbox-2026-11.pem -pubout -out partner-sandbox-2026-11.pub.pem
chmod 600 partner-sandbox-2026-11.pem

# 2) Check what you publish: HTTPS, no redirect, public members only
curl -sS --max-redirs 0 -D - https://partner.example/.well-known/jwks.json \
  | tee /dev/stderr | sed '1,/^\r$/d' \
  | jq '.keys[] | {kid, kty, crv, alg, use, private_member: (has("d") or has("p") or has("q"))}'
```

JWKS فان‌لندز

فان‌لندز رویدادهای Webhook رو با کلید خودش امضا می‌کنه و کلیدهای عمومیش رو اینجا منتشر می‌کنه. این endpoint عمومیه و احراز هویت نمی‌خواد.

HTTP

```
GET /.well-known/jwks.json HTTP/1.1
Host: api.sandbox.funlands.example

HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=300

{
  "keys": [
    {
      "kty": "EC", "crv": "P-256", "alg": "ES256", "use": "sig",
      "kid": "fl-sandbox-2026-11",
      "x": "<base64url-x>", "y": "<base64url-y>"
    }
  ]
}
```

- ممکنه چند کلید هم‌زمان توی این سند باشه: کلید فعال و کلیدهایی که توی دوره چرخش هنوز معتبرن. کلید رو با `kid` انتخاب کنید.
- حداکثر ۵ دقیقه کش کنید (طبق `Cache-Control`). اگه `kid` ناشناخته دیدید یک بار دوباره بخونید، با دست‌کم ۳۰ ثانیه فاصله بین دو بار.
- نشانی JWKS رو از برگ شریک بگیرید و توی تنظیمات خودتون ثابت نگه دارید. نشانی کلید رو هیچ‌وقت از هدر توکن نخونید.
- فان‌لندز هم کلید تازه رو دست‌کم ۲۴ ساعت قبل از استفاده منتشر می‌کنه و کلید قدیمی رو دست‌کم ۲۴ ساعت بعد از آخرین امضا نگه می‌داره.

چرخش برنامه‌ریزی‌شده کلید

۱ **آماده‌سازی**

کلید تازه K2 رو با `kid` تازه توی KMS بسازید و هنوز با K1 امضا کنید.

۲ **انتشار قبل از استفاده**

K1 و K2 رو با هم توی JWKS منتشر کنید و دست‌کم ۲۴ ساعت صبر کنید.

۳ **عوض کردن کلید امضا**

از این به بعد با K2 امضا کنید. فان‌لندز کلید درست رو با `kid` پیدا می‌کنه.

۴ **پایان هم‌پوشانی**

K1 رو دست‌کم ۲۴ ساعت بعد از آخرین امضا توی JWKS نگه دارید، بعد حذفش کنید و توی KMS غیرفعالش کنید.

عوض کردن الگوریتم (مثلا از `ES256` به `RS256`) چرخش کلید حساب نمیشه. این تغییر تنظیمات ثبته و باید جدا هماهنگ و روی sandbox تست بشه.

ابطال اضطراری کلید

اگه فکر می‌کنید کلید خصوصیتون لو رفته، منتظر چرخش عادی نمونید:

۱ **توقف امضا**

امضا با کلید لو رفته رو همین الان متوقف کنید و کلید رو توی KMS غیرفعال کنید.

۲ **کلید جایگزین**

کلید تازه با `kid` تازه بسازید و منتشر کنید. اینجا منتظر مهلت ۲۴ ساعته نمونید، چون فان‌لندز با دیدن `kid` ناشناخته JWKS رو دوباره می‌خونه.

۳ **خبر دادن به فان‌لندز**

از کانال امنیتی ثبت‌شده (تماس اضطراری برگ شریک) خبر بدید: issuer، محیط، `kid` لو رفته و زمان تقریبی. این پیام نباید با کلید لو رفته امضا شده باشه.

۴ **ابطال در فان‌لندز**

فان‌لندز جفت issuer و `kid` رو برای اون محیط توی denylist می‌ذاره. denylist قبل از کش چک میشه، پس از همون لحظه هر توکن با اون `kid`، حتی با `exp` معتبر، با `KEY_NOT_FOUND` رد میشه. نشست‌هایی که با اون کلید باز شدن هم طبق روال حادثه بسته میشن.

> فقط حذف کلید از JWKS کافی نیست، چون نسخه کش‌شده تا چند دقیقه معتبر می‌مونه. مرحله خبر دادن رو هم همیشه انجام بدید.

اگه کلید فان‌لندز لو بره، همین روال برعکس اجرا میشه: از کانال ثبت‌شده بهتون خبر میدیم، شما `kid` لو رفته رو توی سیستم خودتون رد کنید، و بعد از رفع مشکل رویدادها با امضای تازه و همون `event_id` دوباره فرستاده میشن.

۴ ورود بازیکن (Launch)

ورود وب چهار مرحله داره. فقط دو مرحله اول کار شماست و دو مرحله بعد رو صفحه‌های فان‌لندز انجام میدن.

۱. سرور شما بعد از احراز هویت کاربر یک `bootstrap_verifier` تصادفی می‌سازه و توکن launch رو با `launch_challenge` همون verifier امضا می‌کنه.

۲. صفحه شما یک فرم با `launch_token` ، `game` و `bootstrap_verifier` رو داخل iframe به `POST /v1/partner/launch` روی Origin بازی (`FUNLANDS_GAME_ORIGIN`) می‌فرسته.

۳. فان‌لندز توکن رو چک می‌کنه، بازیکن و نشست رو می‌سازه و توی iframe یک صفحه HTML با یک کد یک‌بارمصرف ۳۰ ثانیه‌ای برمی‌گردونه.

۴. همون صفحه کد و verifier رو روی همون Origin به `POST /v1/session/bootstrap` می‌فرسته، توکن‌ها رو به صفحه بازی همین Origin میده و اون رو باز می‌کنه. صفحه بازی توکن‌ها رو فقط توی حافظه نگه می‌داره و از این به بعد فقط با access token کار می‌کنه.

توکن launch

توکن launch فقط روی سرور شما ساخته میشه، اونم فقط بعد از اینکه کاربر واقعا وارد شده و بازی براش مجازه. یک `id_token` معمولی OIDC یا access token خود شما توکن launch نیست و رد میشه.

هدر

فیلد	مقدار	توضیح
`alg`	`ES256`	باید همون الگوریتم ثبت‌شده برای این محیط باشه.
`typ`	`funlands-launch+jwt`	دقیقا همین مقدار. مقدار دیگه یا نبودنش `TOKEN_TYPE_INVALID` می‌گیره.
`kid`	`partner-sandbox-2026-11`	رشته غیرخالی که توی JWKS شما هست.
`jku` ، `x5u` ، `jwk` ، `x5c` ، `crit`	ممنوع	توکن با `TOKEN_TYPE_INVALID` رد میشه.

JSON

```
{
  "alg": "ES256",
  "typ": "funlands-launch+jwt",
  "kid": "partner-sandbox-2026-11"
}
```

Claimها

Claim	الزامی	قاعده
`iss`	بله	دقیقا issuer ثبت‌شده شما توی این محیط.
`aud`	بله	دقیقا audience توکن launch همین محیط. توکن sandbox روی production خطای `AUDIENCE_MISMATCH` می‌گیره.
`sub`	بله	شناسه مستعار، پایدار و مخصوص فان‌لندز برای این کاربر، حداکثر ۱۲۸ کاراکتر. ایمیل، شماره تلفن، کد ملی یا شناسه سراسری قابل ردیابی نباشه.
`sid`	بله	شناسه نشست کاربر توی سیستم شما که با همین می‌تونید دقیقا همین نشست رو خارج کنید. مقدار کوکی نشست خودتون رو نفرستید و یک شناسه تصادفی جدا توی نشست نگه دارید.
`iat` ، `exp`	بله	`exp - iat` حداکثر ۱۲۰ ثانیه. اگه `iat` بیشتر از ۳۰ ثانیه در آینده باشه یا `exp` گذشته باشه، خطای `TOKEN_EXPIRED` میاد.
`jti`	بله	شناسه یکتای همین توکن (مثلا UUID). هر توکن فقط یک بار مصرف میشه.
`game`	بله	شناسه بازی. باید با فیلد `game` فرم یکی باشه و بازی برای شما فعال باشه.
`launch_challenge`	بله	`base64url(SHA-256(bootstrap_verifier))` . جزئیات
`nbf`	اختیاری	اگه باشه با ۳۰ ثانیه اختلاف مجاز چک میشه.
`locale`	اختیاری	برچسب BCP 47، پیش‌فرض `fa-IR` .
`age_band`	اختیاری	رده سنی کاربر از مقدارهایی که توی قرارداد داده توافق شده، مثل `under-13` ، `13-17` و `18+` . اگه نفرستید، سن «نامعلوم» حساب میشه و محتوای محدودشده نشون داده نمیشه.
`nickname`	اختیاری	نام نمایشی، حداکثر ۲۴ کاراکتر، که از فیلتر کلمات نامناسب رد میشه.

JSON

```
{
  "iss": "https://partner.example",
  "aud": "https://api.sandbox.funlands.example/partner-launch",
  "sub": "u_7f3a9c2e5b1d4e08",
  "sid": "5d0c6a1e-2f9b-4c37-9a51-0e6b8f2d7c44",
  "iat": 1793871000,
  "exp": 1793871120,
  "jti": "0b9f6d4e-8a2c-4f1b-9e37-5c2d1a6b8e90",
  "game": "star-runner",
  "launch_challenge": "E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM",
  "locale": "fa-IR",
  "age_band": "13-17",
  "nickname": "Sara"
}
```

اختلاف ساعت مجاز بین سرور شما و فان‌لندز حداکثر ۳۰ ثانیه‌ست. ساعت سرورهای امضاکننده رو با NTP هماهنگ نگه دارید و `iat` و `exp` رو از یک بار خوندن ساعت حساب کنید.

verifier و challenge

این دو مقدار کد bootstrap رو به همون پنجره‌ای گره می‌زنن که فرم رو فرستاده. اگه کد از جایی مثل لاگ یا هدر Referer بیرون بره، بدون verifier به درد نمی‌خوره.

```Pseudo
bootstrap_verifier = base64url(random_bytes(32))             # 43 chars, no "=" padding
launch_challenge   = base64url(SHA-256(ascii(bootstrap_verifier)))

# example
bootstrap_verifier = dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
launch_challenge   = E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
```

- برای هر launch یک verifier تازه بسازید: ۳۲ بایت تصادفی امن، با base64url و بدون `=` .
- challenge رو از بایت‌های ASCII خود رشته verifier حساب کنید، نه از ۳۲ بایت خام.
- verifier رو لاگ نکنید و توی هیچ URLی نذارید. جاش فقط فیلد مخفی فرمه.
- اگه `SHA-256(bootstrap_verifier)` با `launch_challenge` نخونه، خطای `CLAIMS_INVALID` می‌گیرید.

ارسال فرم به `POST /v1/partner/launch`

```HTTP
POST /v1/partner/launch HTTP/1.1
Host: play.sandbox.funlands.example
Content-Type: application/x-www-form-urlencoded
Origin: https://partner.example

launch_token=eyJhbGciOiJFUzI1NiIsInR5cCI6ImZ1bmxhbmRzLWxhdW5jaCtqd3QiLCJraWQiOiJwYXJ0bmVyLXNhbmRib3gtMjAyNi0xMSJ9.e
yJ...
&game=star-runner
&bootstrap_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

- بدنه `application/x-www-form-urlencoded` هست با سه فیلد `launch_token` ، `game` و `bootstrap_verifier` . توکن هیچ‌وقت توی query string یا fragment نمیاد.
- فرم رو به Origin بازی بفرستید، نه به نشانی پایه API. صفحه handoff کد رو روی همون Origin با bootstrap عوض می‌کنه و bootstrap فقط `Origin` ی رو قبول می‌کنه که دقیقا برابر Origin بازی باشه. فرمی که به Origin دیگه‌ای بره به `funlands:launch_failed` با کد `ORIGIN_NOT_ALLOWED` می‌رسه.
- هدر `Origin` ی که مرورگر می‌فرسته باید دقیقا یکی از Originهای ثبت‌شده شما باشه. درخواست بدون `Origin` یا با `Origin: null` رد میشه.
- پاسخ موفق یک صفحه HTML هست که داخل iframe باز میشه، با `Cache-Control: no-store` ، `Referrer-Policy: same-origin` و `frame-ancestors` محدود به Originهای ثبت‌شده شما. کار bootstrap رو همین صفحه انجام میده.
- CSP صفحه خودتون رو طوری بذارید که `frame-src` و `form-action` فقط Origin بازی فان‌لندز رو مجاز کنن. صفحه‌ای که توکن رو داره با `Cache-Control: no-store` و `Referrer-Policy: strict-origin-when-cross-origin` برگردونید.
- بار اولی که یک `sub` وارد میشه، بازیکن تازه ساخته میشه. ورودهای بعدی با همون `sub` توی همون محیط به همون بازیکن می‌رسن، حتی اگه دو ورود هم‌زمان باشن. همون `sub` توی محیط دیگه بازیکن جداییه. فان‌لندز بازیکن رو خودکار به حساب دیگه‌ای در فان‌لندز یا به بازیکن شریک دیگه وصل نمی‌کنه.

> روی صفحه‌ای که فرم launch رو می‌فرسته `Referrer-Policy: no-referrer` نذارید. با این مقدار مرورگر توی POST بین دو دامنه هدر `Origin: null` می‌فرسته و ورود با `ORIGIN_NOT_ALLOWED` رد میشه.
>
> `strict-origin-when-cross-origin` رو بذارید (پیش‌فرض مرورگرها)، همون مقداری که نمونه‌های زیر می‌ذارن.

نمونه: امضای توکن launch و ساخت فرم خودکار

NestJS

```
// src/funlands/funlands.config.ts
export const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';           // production: https://api.funlands.example
export const FUNLANDS_GAME_ORIGIN = 'https://play.sandbox.funlands.example';   // from your registration sheet; launch form, iframe, postMessage
export const LAUNCH_AUDIENCE = `${FUNLANDS_BASE}/partner-launch`;          // exact value from your registration sheet
export const PARTNER_ISSUER = 'https://partner.example';
export const LAUNCH_KID = 'partner-sandbox-2026-11';

// src/funlands/launch-token.service.ts
import { Injectable, OnModuleInit } from '@nestjs/common';
import { importPKCS8, SignJWT } from 'jose';
import { createHash, randomBytes, randomUUID } from 'node:crypto';
import { readFile } from 'node:fs/promises';
import { LAUNCH_AUDIENCE, LAUNCH_KID, PARTNER_ISSUER } from './funlands.config';

type PrivateKey = Awaited<ReturnType<typeof importPKCS8>>;

@Injectable()
export class LaunchTokenService implements OnModuleInit {
  private privateKey!: PrivateKey;

  async onModuleInit() {
    // Production: sign through your KMS/HSM instead of a PEM file.
    this.privateKey = await importPKCS8(await readFile(`keys/${LAUNCH_KID}.pem`, 'utf8'), 'ES256');
  }

  signingKey() {
    return this.privateKey;     // also signs logout tokens
  }

  async mint(input: { subject: string; sid: string; game: string }) {
    const verifier = randomBytes(32).toString('base64url');
    const challenge = createHash('sha256').update(verifier, 'ascii').digest('base64url');
    const now = Math.floor(Date.now() / 1000);

    const token = await new SignJWT({ sid: input.sid, game: input.game, launch_challenge: challenge, locale: 'fa-IR' })
      .setProtectedHeader({ alg: 'ES256', typ: 'funlands-launch+jwt', kid: LAUNCH_KID })
      .setIssuer(PARTNER_ISSUER)
      .setAudience(LAUNCH_AUDIENCE)
      .setSubject(input.subject)
      .setIssuedAt(now)
      .setExpirationTime(now + 120)
      .setJti(randomUUID())
      .sign(this.privateKey);

    return { token, verifier };
  }
}

// src/funlands/launch-page.ts
const esc = (v: string) => v.replace(/[&<>"']/g, (c) => `&#${c.charCodeAt(0)};`);

export function launchPage(p: { action: string; token: string; verifier: string; game: string; nonce: string }) {
  return `<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></iframe>
<form id="funlands-launch" method="post" action="${esc(p.action)}" target="funlands-game">
  <input type="hidden" name="launch_token" value="${esc(p.token)}">
  <input type="hidden" name="game" value="${esc(p.game)}">
  <input type="hidden" name="bootstrap_verifier" value="${esc(p.verifier)}">
</form>
<script nonce="${p.nonce}">document.getElementById('funlands-launch').submit();</script>
```

```
</body></html>`;
}

// src/funlands/play.controller.ts
import { BadRequestException, Controller, Get, Param, Req, Res, UseGuards } from '@nestjs/common';
import type { Request, Response } from 'express';
import { randomBytes, randomUUID } from 'node:crypto';
import { AuthenticatedGuard } from '../auth/authenticated.guard'; // your own session guard
import { FUNLANDS_GAME_ORIGIN } from './funlands.config';
import { launchPage } from './launch-page';
import { LaunchTokenService } from './launch-token.service';

@Controller('play')
export class PlayController {
  constructor(private readonly launch: LaunchTokenService) {}

  @Get(':game')
  @UseGuards(AuthenticatedGuard)
  async play(@Param('game') game: string, @Req() req: Request, @Res() res: Response) {
    if (!/^[a-z0-9-]{1,64}$/.test(game)) throw new BadRequestException();
    const user = req.user as { funlandsSubject: string };     // stable pairwise id, never email/phone
    const session = req.session as unknown as { funlandsSid?: string };
    session.funlandsSid ??= randomUUID();                      // not your session cookie value

    const { token, verifier } = await this.launch.mint({ subject: user.funlandsSubject, sid: session.funlandsSid, g
ame });
    const nonce = randomBytes(16).toString('base64');

    res.set({
      'Cache-Control': 'no-store',
      'Referrer-Policy': 'strict-origin-when-cross-origin',
      'Content-Security-Policy':
        `frame-src ${FUNLANDS_GAME_ORIGIN}; form-action ${FUNLANDS_GAME_ORIGIN}; script-src 'nonce-${nonce}'; objec
t-src 'none'; base-uri 'none'`,
    });
    res.type('html').send(launchPage({ action: `${FUNLANDS_GAME_ORIGIN}/v1/partner/launch`, token, verifier, game,
nonce }));
  }
}
```

Express

```
// launch.mjs
import express from 'express';
import { importPKCS8, SignJWT } from 'jose';
import { createHash, randomBytes, randomUUID } from 'node:crypto';
import { readFile } from 'node:fs/promises';

const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';         // production: https://api.funlands.example
const FUNLANDS_GAME_ORIGIN = 'https://play.sandbox.funlands.example'; // from your registration sheet
const LAUNCH_AUDIENCE = `${FUNLANDS_BASE}/partner-launch`;
const PARTNER_ISSUER = 'https://partner.example';
const LAUNCH_KID = 'partner-sandbox-2026-11';
const privateKey = await importPKCS8(await readFile(`keys/${LAUNCH_KID}.pem`, 'utf8'), 'ES256');

const esc = (v) => String(v).replace(/[&<>"']/g, (c) => `&#${c.charCodeAt(0)};`);

async function mintLaunch({ subject, sid, game }) {
  const verifier = randomBytes(32).toString('base64url');
  const challenge = createHash('sha256').update(verifier, 'ascii').digest('base64url');
  const now = Math.floor(Date.now() / 1000);
  const token = await new SignJWT({ sid, game, launch_challenge: challenge, locale: 'fa-IR' })
    .setProtectedHeader({ alg: 'ES256', typ: 'funlands-launch+jwt', kid: LAUNCH_KID })
    .setIssuer(PARTNER_ISSUER)
    .setAudience(LAUNCH_AUDIENCE)
    .setSubject(subject)
    .setIssuedAt(now)
    .setExpirationTime(now + 120)
    .setJti(randomUUID())
    .sign(privateKey);
  return { token, verifier };
```

```
}

export const router = express.Router();

// requireLogin and req.session come from your existing auth (e.g. express-session + passport).
router.get('/play/:game', requireLogin, async (req, res, next) => {
  try {
    const { game } = req.params;
    if (!/^[a-z0-9-]{1,64}$/.test(game)) return res.sendStatus(400);
    req.session.funlandsSid ??= randomUUID();
    const { token, verifier } = await mintLaunch({ subject: req.user.funlandsSubject, sid: req.session.funlandsSid,
game });
    const nonce = randomBytes(16).toString('base64');

    res.set({
      'Cache-Control': 'no-store',
      'Referrer-Policy': 'strict-origin-when-cross-origin',
      'Content-Security-Policy':
        `frame-src ${FUNLANDS_GAME_ORIGIN}; form-action ${FUNLANDS_GAME_ORIGIN}; script-src 'nonce-${nonce}'; objec
t-src 'none'; base-uri 'none'`,
    });
    res.type('html').send(`<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="${FUNLANDS_GAME_ORIGIN}/v1/partner/launch" target="funlands-game">
  <input type="hidden" name="launch_token" value="${esc(token)}">
  <input type="hidden" name="game" value="${esc(game)}">
  <input type="hidden" name="bootstrap_verifier" value="${esc(verifier)}">
</form>
<script nonce="${nonce}">document.getElementById('funlands-launch').submit();</script>
</body></html>`);
  } catch (err) {
    next(err);
  }
});
```

FastAPI

```
# app/launch.py
import base64
import hashlib
import re
import secrets
import time
import uuid
from html import escape
from pathlib import Path

import jwt
from cryptography.hazmat.primitives import serialization
from fastapi import APIRouter, Depends, HTTPException, Request
from fastapi.responses import HTMLResponse

from app.auth import current_user  # your existing session auth; returns a user with .funlands_subject

FUNLANDS_BASE = "https://api.sandbox.funlands.example"      # production: https://api.funlands.example
FUNLANDS_GAME_ORIGIN = "https://play.sandbox.funlands.example"  # from your registration sheet
LAUNCH_AUDIENCE = f"{FUNLANDS_BASE}/partner-launch"
PARTNER_ISSUER = "https://partner.example"
LAUNCH_KID = "partner-sandbox-2026-11"
PRIVATE_KEY = serialization.load_pem_private_key(Path(f"keys/{LAUNCH_KID}.pem").read_bytes(), password=None)

router = APIRouter()


def b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")
```

```python
def mint_launch(subject: str, sid: str, game: str) -> tuple[str, str]:
    verifier = secrets.token_urlsafe(32)                      # 32 random bytes, base64url, no padding
    challenge = b64url(hashlib.sha256(verifier.encode("ascii")).digest())
    now = int(time.time())
    claims = {
        "iss": PARTNER_ISSUER, "aud": LAUNCH_AUDIENCE, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": str(uuid.uuid4()),
        "game": game, "launch_challenge": challenge, "locale": "fa-IR",
    }
    token = jwt.encode(claims, PRIVATE_KEY, algorithm="ES256",
                       headers={"typ": "funlands-launch+jwt", "kid": LAUNCH_KID})
    return token, verifier


@router.get("/play/{game}", response_class=HTMLResponse)
def play(game: str, request: Request, user=Depends(current_user)):
    if not re.fullmatch(r"[a-z0-9-]{1,64}", game):
        raise HTTPException(400)
    sid = request.session.setdefault("funlands_sid", str(uuid.uuid4()))   # needs SessionMiddleware
    token, verifier = mint_launch(user.funlands_subject, sid, game)
    nonce = secrets.token_urlsafe(16)
    html = f"""<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="{FUNLANDS_GAME_ORIGIN}/v1/partner/launch" target="funlands-game">
  <input type="hidden" name="launch_token" value="{escape(token)}">
  <input type="hidden" name="game" value="{escape(game)}">
  <input type="hidden" name="bootstrap_verifier" value="{escape(verifier)}">
</form>
<script nonce="{nonce}">document.getElementById('funlands-launch').submit();</script>
</body></html>"""
    return HTMLResponse(html, headers={
        "Cache-Control": "no-store",
        "Referrer-Policy": "strict-origin-when-cross-origin",
        "Content-Security-Policy": f"frame-src {FUNLANDS_GAME_ORIGIN}; form-action {FUNLANDS_GAME_ORIGIN}; "
                                   f"script-src 'nonce-{nonce}'; object-src 'none'; base-uri 'none'",
    })
```

Django

```python
# partner/funlands.py
import base64
import hashlib
import secrets
import time
import uuid
from functools import cache
from pathlib import Path

import jwt
from cryptography.hazmat.primitives import serialization
from django.conf import settings

FUNLANDS_BASE = "https://api.sandbox.funlands.example"      # production: https://api.funlands.example
FUNLANDS_GAME_ORIGIN = "https://play.sandbox.funlands.example"  # from your registration sheet
LAUNCH_AUDIENCE = f"{FUNLANDS_BASE}/partner-launch"
PARTNER_ISSUER = "https://partner.example"
LAUNCH_KID = "partner-sandbox-2026-11"


@cache
def private_key(kid: str):
    pem = (Path(settings.FUNLANDS_KEY_DIR) / f"{kid}.pem").read_bytes()
    return serialization.load_pem_private_key(pem, password=None)


def b64url(data: bytes) -> str:
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode("ascii")
```

```
def mint_launch(subject: str, sid: str, game: str) -> tuple[str, str]:
    verifier = secrets.token_urlsafe(32)
    challenge = b64url(hashlib.sha256(verifier.encode("ascii")).digest())
    now = int(time.time())
    claims = {
        "iss": PARTNER_ISSUER, "aud": LAUNCH_AUDIENCE, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": str(uuid.uuid4()),
        "game": game, "launch_challenge": challenge, "locale": "fa-IR",
    }
    token = jwt.encode(claims, private_key(LAUNCH_KID), algorithm="ES256",
                       headers={"typ": "funlands-launch+jwt", "kid": LAUNCH_KID})
    return token, verifier


# partner/views.py
import re
import secrets
import uuid

from django.contrib.auth.decorators import login_required
from django.http import HttpResponseBadRequest
from django.shortcuts import render
from django.views.decorators.http import require_GET

from .funlands import FUNLANDS_GAME_ORIGIN, mint_launch


@require_GET
@login_required
def play(request, game: str):
    if not re.fullmatch(r"[a-z0-9-]{1,64}", game):
        return HttpResponseBadRequest()
    sid = request.session.setdefault("funlands_sid", str(uuid.uuid4()))
    token, verifier = mint_launch(request.user.funlands_subject, sid, game)
    nonce = secrets.token_urlsafe(16)
    response = render(request, "partner/launch.html", {
        "action": f"{FUNLANDS_GAME_ORIGIN}/v1/partner/launch",
        "token": token, "verifier": verifier, "game": game, "nonce": nonce,
    })
    response["Cache-Control"] = "no-store"
    response["Referrer-Policy"] = "strict-origin-when-cross-origin"
    response["Content-Security-Policy"] = (
        f"frame-src {FUNLANDS_GAME_ORIGIN}; form-action {FUNLANDS_GAME_ORIGIN}; "
        f"script-src 'nonce-{nonce}'; object-src 'none'; base-uri 'none'")
    return response


# partner/templates/partner/launch.html   (Django escapes every {{ }} value)
<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="{{ action }}" target="funlands-game">
  <input type="hidden" name="launch_token" value="{{ token }}">
  <input type="hidden" name="game" value="{{ game }}">
  <input type="hidden" name="bootstrap_verifier" value="{{ verifier }}">
</form>
<script nonce="{{ nonce }}">document.getElementById('funlands-launch').submit();</script>
</body></html>

# partner/urls.py:  path("play/<slug:game>", views.play)
```

Laravel

```
<?php
// config/funlands.php
return [
    'base' => env('FUNLANDS_BASE', 'https://api.sandbox.funlands.example'),          // production: https://api.fun
lands.example
```

```php
    'game_origin' => env('FUNLANDS_GAME_ORIGIN', 'https://play.sandbox.funlands.example'), // from your registratio
n sheet
    'launch_audience' => env('FUNLANDS_LAUNCH_AUD', 'https://api.sandbox.funlands.example/partner-launch'),
    'issuer' => 'https://partner.example',
    'launch_kid' => 'partner-sandbox-2026-11',
];

// app/Funlands/LaunchToken.php
namespace App\Funlands;

use Firebase\JWT\JWT;
use Illuminate\Support\Str;

final class LaunchToken
{
    /** @return array{0: string, 1: string} [token, verifier] */
    public static function mint(string $subject, string $sid, string $game): array
    {
        $verifier = self::b64url(random_bytes(32));
        $challenge = self::b64url(hash('sha256', $verifier, true));
        $now = time();
        $kid = config('funlands.launch_kid');

        $claims = [
            'iss' => config('funlands.issuer'),
            'aud' => config('funlands.launch_audience'),
            'sub' => $subject,
            'sid' => $sid,
            'iat' => $now,
            'exp' => $now + 120,
            'jti' => (string) Str::uuid(),
            'game' => $game,
            'launch_challenge' => $challenge,
            'locale' => 'fa-IR',
        ];
        $privatePem = file_get_contents(storage_path("keys/{$kid}.pem")); // production: KMS signer
        $token = JWT::encode($claims, $privatePem, 'ES256', $kid, ['typ' => 'funlands-launch+jwt']);

        return [$token, $verifier];
    }

    public static function b64url(string $bytes): string
    {
        return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
    }
}

// app/Http/Controllers/PlayController.php
namespace App\Http\Controllers;

use App\Funlands\LaunchToken;
use Illuminate\Http\Request;
use Illuminate\Support\Str;

final class PlayController
{
    public function __invoke(Request $request, string $game)
    {
        abort_unless(preg_match('/^[a-z0-9-]{1,64}$/', $game) === 1, 400);
        if (! $request->session()->has('funlands_sid')) {
            $request->session()->put('funlands_sid', (string) Str::uuid());
        }
        [$token, $verifier] = LaunchToken::mint(
            $request->user()->funlands_subject,
            $request->session()->get('funlands_sid'),
            $game,
        );
        $nonce = base64_encode(random_bytes(16));
        $gameOrigin = config('funlands.game_origin');

        return response()
```

```
            ->view('funlands.launch', [
                'action' => "{$gameOrigin}/v1/partner/launch",
                'token' => $token, 'verifier' => $verifier, 'game' => $game, 'nonce' => $nonce,
            ])
            ->header('Cache-Control', 'no-store')
            ->header('Referrer-Policy', 'strict-origin-when-cross-origin')
            ->header('Content-Security-Policy',
                "frame-src {$gameOrigin}; form-action {$gameOrigin}; script-src 'nonce-{$nonce}'; object-src 'non
e'; base-uri 'none'");
    }
}

// routes/web.php
Route::get('/play/{game}', \App\Http\Controllers\PlayController::class)->middleware('auth');

{{-- resources/views/funlands/launch.blade.php  (Blade escapes every {{ }} value) --}}
<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="{{ $action }}" target="funlands-game">
  <input type="hidden" name="launch_token" value="{{ $token }}">
  <input type="hidden" name="game" value="{{ $game }}">
  <input type="hidden" name="bootstrap_verifier" value="{{ $verifier }}">
</form>
<script nonce="{{ $nonce }}">document.getElementById('funlands-launch').submit();</script>
</body></html>
```

PHP

```
<?php
// public/play.php   (GET /play.php?game=star-runner, user already signed in)
declare(strict_types=1);
require __DIR__ . '/../vendor/autoload.php';

use Firebase\JWT\JWT;

const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';             // production: https://api.funlands.examp
le
const FUNLANDS_GAME_ORIGIN = 'https://play.sandbox.funlands.example';     // from your registration sheet
const LAUNCH_AUDIENCE = FUNLANDS_BASE . '/partner-launch';
const PARTNER_ISSUER = 'https://partner.example';
const LAUNCH_KID = 'partner-sandbox-2026-11';

function b64url(string $bytes): string
{
    return rtrim(strtr(base64_encode($bytes), '+/', '-_'), '=');
}

function h(string $v): string
{
    return htmlspecialchars($v, ENT_QUOTES | ENT_HTML5, 'UTF-8');
}

session_start();
if (!isset($_SESSION['user'])) {
    http_response_code(401);
    exit;
}
$game = (string) ($_GET['game'] ?? '');
if (preg_match('/^[a-z0-9-]{1,64}$/', $game) !== 1) {
    http_response_code(400);
    exit;
}
$_SESSION['funlands_sid'] ??= bin2hex(random_bytes(16));   // not the PHPSESSID value

$verifier = b64url(random_bytes(32));
$challenge = b64url(hash('sha256', $verifier, true));
$now = time();
$token = JWT::encode([
```

```
    'iss' => PARTNER_ISSUER,
    'aud' => LAUNCH_AUDIENCE,
    'sub' => $_SESSION['user']['funlands_subject'],
    'sid' => $_SESSION['funlands_sid'],
    'iat' => $now,
    'exp' => $now + 120,
    'jti' => bin2hex(random_bytes(16)),
    'game' => $game,
    'launch_challenge' => $challenge,
    'locale' => 'fa-IR',
], file_get_contents(__DIR__ . '/../keys/' . LAUNCH_KID . '.pem'), 'ES256', LAUNCH_KID, ['typ' => 'funlands-launch+
jwt']);

$nonce = base64_encode(random_bytes(16));
header('Cache-Control: no-store');
header('Referrer-Policy: strict-origin-when-cross-origin');
header("Content-Security-Policy: frame-src " . FUNLANDS_GAME_ORIGIN . "; form-action " . FUNLANDS_GAME_ORIGIN
    . "; script-src 'nonce-{$nonce}'; object-src 'none'; base-uri 'none'");
?>
<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="<?= h(FUNLANDS_GAME_ORIGIN . '/v1/partner/launch') ?>" target="fun
lands-game">
  <input type="hidden" name="launch_token" value="<?= h($token) ?>">
  <input type="hidden" name="game" value="<?= h($game) ?>">
  <input type="hidden" name="bootstrap_verifier" value="<?= h($verifier) ?>">
</form>
<script nonce="<?= h($nonce) ?>">document.getElementById('funlands-launch').submit();</script>
</body></html>
```

Go

```
// launch.go
package partner

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/base64"
    "encoding/json"
    "html/template"
    "net/http"
    "os"
    "regexp"
    "time"

    "github.com/lestrrat-go/jwx/v2/jwa"
    "github.com/lestrrat-go/jwx/v2/jwk"
    "github.com/lestrrat-go/jwx/v2/jws"
)

const (
    FunlandsBase   = "https://api.sandbox.funlands.example" // production: https://api.funlands.example
    GameOrigin     = "https://play.sandbox.funlands.example" // FUNLANDS_GAME_ORIGIN from your registration sheet
    LaunchAudience = FunlandsBase + "/partner-launch"
    PartnerIssuer  = "https://partner.example"
    LaunchKID      = "partner-sandbox-2026-11"
)

var gamePattern = regexp.MustCompile(`^[a-z0-9-]{1,64}$`)

// LoadPrivateKey reads a PKCS#8 PEM key. Production: sign through your KMS/HSM.
func LoadPrivateKey(kid string) (jwk.Key, error) {
    raw, err := os.ReadFile("keys/" + kid + ".pem")
    if err != nil {
        return nil, err
    }
    key, err := jwk.ParseKey(raw, jwk.WithPEM(true))
```

```
    if err != nil {
        return nil, err
    }
    return key, key.Set(jwk.KeyIDKey, kid)
}

func randomB64(n int) string {
    b := make([]byte, n)
    if _, err := rand.Read(b); err != nil {
        panic(err)
    }
    return base64.RawURLEncoding.EncodeToString(b)
}

// SignJWT signs claims with ES256 and the exact typ header given.
func SignJWT(key jwk.Key, typ string, claims map[string]any) (string, error) {
    payload, err := json.Marshal(claims)
    if err != nil {
        return "", err
    }
    hdrs := jws.NewHeaders()
    _ = hdrs.Set(jws.TypeKey, typ)
    _ = hdrs.Set(jws.KeyIDKey, key.KeyID())
    signed, err := jws.Sign(payload, jws.WithKey(jwa.ES256, key, jws.WithProtectedHeaders(hdrs)))
    return string(signed), err
}

func MintLaunch(key jwk.Key, subject, sid, game string) (token, verifier string, err error) {
    verifier = randomB64(32)
    sum := sha256.Sum256([]byte(verifier))
    now := time.Now().Unix()
    token, err = SignJWT(key, "funlands-launch+jwt", map[string]any{
        "iss": PartnerIssuer, "aud": LaunchAudience, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": randomB64(16),
        "game": game, "launch_challenge": base64.RawURLEncoding.EncodeToString(sum[:]), "locale": "fa-IR",
    })
    return token, verifier, err
}

var launchTmpl = template.Must(template.New("launch").Parse(`<!doctype html>
<html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
<body style="margin:0">
<iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;height:100vh"></
iframe>
<form id="funlands-launch" method="post" action="{{.Action}}" target="funlands-game">
  <input type="hidden" name="launch_token" value="{{.Token}}">
  <input type="hidden" name="game" value="{{.Game}}">
  <input type="hidden" name="bootstrap_verifier" value="{{.Verifier}}">
</form>
<script nonce="{{.Nonce}}">document.getElementById('funlands-launch').submit();</script>
</body></html>`))

// PlayHandler serves GET /play/{game}. SessionFrom is your auth middleware's accessor; it returns the
// user's stable pairwise subject and a random per-login FunlandsSID (never your session cookie value).
func PlayHandler(key jwk.Key) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        sess, ok := SessionFrom(r)
        if !ok {
            http.Error(w, "", http.StatusUnauthorized)
            return
        }
        game := r.PathValue("game")
        if !gamePattern.MatchString(game) {
            http.Error(w, "", http.StatusBadRequest)
            return
        }
        token, verifier, err := MintLaunch(key, sess.FunlandsSubject, sess.FunlandsSID, game)
        if err != nil {
            http.Error(w, "", http.StatusInternalServerError)
            return
        }
```

```
        nonce := randomB64(16)
        h := w.Header()
        h.Set("Content-Type", "text/html; charset=utf-8")
        h.Set("Cache-Control", "no-store")
        h.Set("Referrer-Policy", "strict-origin-when-cross-origin")
        h.Set("Content-Security-Policy", "frame-src "+GameOrigin+"; form-action "+GameOrigin+
            "; script-src 'nonce-"+nonce+"'; object-src 'none'; base-uri 'none'")
        _ = launchTmpl.Execute(w, map[string]string{
            "Action": GameOrigin + "/v1/partner/launch", "Token": token, "Game": game, "Verifier": verifier, "Nonc
e": nonce,
        })
    }
}

// main.go:  mux.Handle("GET /play/{game}", partner.PlayHandler(key))
```

Spring Boot

```
// src/main/java/com/partner/funlands/FunlandsConfig.java
package com.partner.funlands;

public final class FunlandsConfig {
    public static final String BASE = "https://api.sandbox.funlands.example";      // production: https://api.funla
nds.example
    public static final String GAME_ORIGIN = "https://play.sandbox.funlands.example"; // FUNLANDS_GAME_ORIGIN from
your registration sheet
    public static final String LAUNCH_AUDIENCE = BASE + "/partner-launch";
    public static final String LOGOUT_AUDIENCE = BASE + "/partner-logout";
    public static final String TOKEN_ENDPOINT = BASE + "/oauth/token";
    public static final String PARTNER_ISSUER = "https://partner.example";
    public static final String LAUNCH_KID = "partner-sandbox-2026-11";
    private FunlandsConfig() {}
}

// src/main/java/com/partner/funlands/LaunchTokenService.java
package com.partner.funlands;

import static com.partner.funlands.FunlandsConfig.*;

import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.ECDSASigner;
import com.nimbusds.jwt.*;
import java.nio.charset.StandardCharsets;
import java.security.*;
import java.time.Instant;
import java.util.*;
import org.springframework.stereotype.Service;

@Service
public class LaunchTokenService {
    public record Launch(String token, String verifier) {}

    private final SecureRandom random = new SecureRandom();
    private final JWSSigner signer;

    public LaunchTokenService() throws Exception {
        this.signer = new ECDSASigner(PemKeys.privateKey(LAUNCH_KID));   // production: KMS-backed signer
    }

    public Launch mint(String subject, String sid, String game) throws Exception {
        byte[] raw = new byte[32];
        random.nextBytes(raw);
        Base64.Encoder b64url = Base64.getUrlEncoder().withoutPadding();
        String verifier = b64url.encodeToString(raw);
        String challenge = b64url.encodeToString(
            MessageDigest.getInstance("SHA-256").digest(verifier.getBytes(StandardCharsets.US_ASCII)));
        Instant now = Instant.now();

        JWSHeader header = new JWSHeader.Builder(JWSAlgorithm.ES256)
            .type(new JOSEObjectType("funlands-launch+jwt")).keyID(LAUNCH_KID).build();
        JWTClaimsSet claims = new JWTClaimsSet.Builder()
```

```
            .issuer(PARTNER_ISSUER).audience(LAUNCH_AUDIENCE).subject(subject)
            .issueTime(Date.from(now)).expirationTime(Date.from(now.plusSeconds(120)))
            .jwtID(UUID.randomUUID().toString())
            .claim("sid", sid).claim("game", game).claim("launch_challenge", challenge).claim("locale", "fa-IR")
            .build();
        SignedJWT jwt = new SignedJWT(header, claims);
        jwt.sign(signer);
        return new Launch(jwt.serialize(), verifier);
    }
}

// src/main/java/com/partner/funlands/PlayController.java
package com.partner.funlands;

import static com.partner.funlands.FunlandsConfig.GAME_ORIGIN;

import jakarta.servlet.http.HttpSession;
import java.security.SecureRandom;
import java.util.*;
import org.springframework.http.*;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.util.HtmlUtils;

@RestController
public class PlayController {
    private final LaunchTokenService launches;
    private final SecureRandom random = new SecureRandom();

    public PlayController(LaunchTokenService launches) { this.launches = launches; }

    @GetMapping(value = "/play/{game}", produces = MediaType.TEXT_HTML_VALUE)
    public ResponseEntity<String> play(@PathVariable String game, HttpSession session,
                                       // PartnerUser: your UserDetails, exposing the stable pairwise subject
                                       @AuthenticationPrincipal PartnerUser user) throws Exception {
        if (!game.matches("[a-z0-9-]{1,64}")) return ResponseEntity.badRequest().build();
        String sid = (String) session.getAttribute("funlands_sid");
        if (sid == null) {
            sid = UUID.randomUUID().toString();          // not the JSESSIONID value
            session.setAttribute("funlands_sid", sid);
        }
        var launch = launches.mint(user.funlandsSubject(), sid, game);
        byte[] n = new byte[16];
        random.nextBytes(n);
        String nonce = Base64.getEncoder().encodeToString(n);

        String html = """
            <!doctype html>
            <html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
            <body style="margin:0">
            <iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;heig
ht:100vh"></iframe>
            <form id="funlands-launch" method="post" action="%s/v1/partner/launch" target="funlands-game">
              <input type="hidden" name="launch_token" value="%s">
              <input type="hidden" name="game" value="%s">
              <input type="hidden" name="bootstrap_verifier" value="%s">
            </form>
            <script nonce="%s">document.getElementById('funlands-launch').submit();</script>
            </body></html>
            """.formatted(GAME_ORIGIN, HtmlUtils.htmlEscape(launch.token()), HtmlUtils.htmlEscape(game),
                          HtmlUtils.htmlEscape(launch.verifier()), nonce);

        return ResponseEntity.ok()
            .cacheControl(CacheControl.noStore())
            .header("Referrer-Policy", "strict-origin-when-cross-origin")
            .header("Content-Security-Policy", "frame-src " + GAME_ORIGIN + "; form-action " + GAME_ORIGIN
                + "; script-src 'nonce-" + nonce + "'; object-src 'none'; base-uri 'none'")
            .body(html);
    }
}
```

ASP.NET Core

```
// Funlands/FunlandsConfig.cs
public static class FunlandsConfig
{
    public const string Base = "https://api.sandbox.funlands.example";        // production: https://api.funlands.e
xample
    public const string GameOrigin = "https://play.sandbox.funlands.example"; // FUNLANDS_GAME_ORIGIN from your reg
istration sheet
    public const string LaunchAudience = Base + "/partner-launch";
    public const string LogoutAudience = Base + "/partner-logout";
    public const string TokenEndpoint = Base + "/oauth/token";
    public const string PartnerIssuer = "https://partner.example";
    public const string LaunchKid = "partner-sandbox-2026-11";
}

// Funlands/PartnerSigner.cs  (register as singleton; production: KMS-backed key)
using System.Security.Cryptography;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;

public sealed class PartnerSigner
{
    private readonly JsonWebTokenHandler _handler = new() { SetDefaultTimesOnTokenCreation = false };

    public SigningCredentials Credentials(string kid)
    {
        var ec = ECDsa.Create();
        ec.ImportFromPem(File.ReadAllText($"keys/{kid}.pem"));
        return new SigningCredentials(new ECDsaSecurityKey(ec) { KeyId = kid }, SecurityAlgorithms.EcdsaSha256);
    }

    public string Sign(string typ, SigningCredentials credentials, string issuer, string audience,
                       int lifetimeSeconds, Dictionary<string, object> claims)
    {
        var now = DateTime.UtcNow;
        return _handler.CreateToken(new SecurityTokenDescriptor
        {
            TokenType = typ,                              // header "typ"
            Issuer = issuer,
            Audience = audience,
            IssuedAt = now,
            Expires = now.AddSeconds(lifetimeSeconds),
            Claims = claims,
            SigningCredentials = credentials,
        });
    }
}

// Funlands/LaunchEndpoints.cs
using System.Net;
using System.Security.Cryptography;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public static class LaunchEndpoints
{
    public static void MapFunlandsLaunch(this WebApplication app)
    {
        var signer = app.Services.GetRequiredService<PartnerSigner>();
        var credentials = signer.Credentials(FunlandsConfig.LaunchKid);

        app.MapGet("/play/{game:regex(^[a-z0-9-]{{1,64}}$)}", (string game, HttpContext http) =>
        {
            // Both claims are added to the auth cookie at sign-in: a stable pairwise subject
            // and a random per-login sid (never the cookie value itself).
            var subject = http.User.FindFirst("funlands_sub")!.Value;
            var sid = http.User.FindFirst("funlands_sid")!.Value;

            var verifier = Base64UrlEncoder.Encode(RandomNumberGenerator.GetBytes(32));
            var challenge = Base64UrlEncoder.Encode(SHA256.HashData(Encoding.ASCII.GetBytes(verifier)));
```

```
            var token = signer.Sign("funlands-launch+jwt", credentials,
                FunlandsConfig.PartnerIssuer, FunlandsConfig.LaunchAudience, 120, new()
            {
                ["sub"] = subject, ["sid"] = sid, ["jti"] = Guid.NewGuid().ToString(),
                ["game"] = game, ["launch_challenge"] = challenge, ["locale"] = "fa-IR",
            });

            var nonce = Convert.ToBase64String(RandomNumberGenerator.GetBytes(16));
            var h = http.Response.Headers;
            h.CacheControl = "no-store";
            h["Referrer-Policy"] = "strict-origin-when-cross-origin";
            h.ContentSecurityPolicy = $"frame-src {FunlandsConfig.GameOrigin}; form-action {FunlandsConfig.GameOrig
in}; " +
                                      $"script-src 'nonce-{nonce}'; object-src 'none'; base-uri 'none'";
            var e = WebUtility.HtmlEncode;
            return Results.Content($"""
                <!doctype html>
                <html lang="fa" dir="rtl"><head><meta charset="utf-8"><title>Game</title></head>
                <body style="margin:0">
                <iframe name="funlands-game" title="Game" allow="autoplay; fullscreen" style="border:0;width:100vw;
height:100vh"></iframe>
                <form id="funlands-launch" method="post" action="{FunlandsConfig.GameOrigin}/v1/partner/launch" tar
get="funlands-game">
                  <input type="hidden" name="launch_token" value="{e(token)}">
                  <input type="hidden" name="game" value="{e(game)}">
                  <input type="hidden" name="bootstrap_verifier" value="{e(verifier)}">
                </form>
                <script nonce="{nonce}">document.getElementById('funlands-launch').submit();</script>
                </body></html>
                """, "text/html; charset=utf-8");
        }).RequireAuthorization();
    }
}
```

cURL

```
# Sandbox only: replay the browser form POST by hand to test your token.
# LAUNCH_TOKEN and VERIFIER come from your server (any tab above).
curl -i https://play.sandbox.funlands.example/v1/partner/launch \
  -H 'Origin: https://partner.example' \
  --data-urlencode "launch_token=${LAUNCH_TOKEN}" \
  --data-urlencode 'game=star-runner' \
  --data-urlencode "bootstrap_verifier=${VERIFIER}"

# Expected: 200 with Content-Type: text/html, Cache-Control: no-store.
# A failure returns the error envelope (correlation ID in the X-Correlation-Id header), for example:
# {"state":false,"code":"AUDIENCE_MISMATCH","message":"..."}

# Inspect the claims you signed (header and payload are only base64url, not encrypted):
cut -d. -f1 <<<"$LAUNCH_TOKEN" | tr '_-' '/+' | base64 -d 2>/dev/null; echo
cut -d. -f2 <<<"$LAUNCH_TOKEN" | tr '_-' '/+' | base64 -d 2>/dev/null; echo
```

bootstrap: `POST /v1/session/bootstrap`

این درخواست رو صفحه handoff فان‌لندز داخل iframe روی Origin بازی می‌فرسته و شما معمولا مستقیم صداش نمی‌زنید. شکلش رو اینجا آوردیم تا موقع عیب‌یابی بدونید چه اتفاقی می‌افته. پاسخ موفق bootstrap، برخلاف refresh، پاکت `{ "state", "data" }` نداره و فیلدهاش snake_case هستن.

HTTP

```
POST /v1/session/bootstrap HTTP/1.1
Host: play.sandbox.funlands.example
Origin: https://play.sandbox.funlands.example
Content-Type: application/json

{ "code": "boot_01JD3X9F4K7M2N8P5Q", "bootstrap_verifier": "dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk" }
```

```
HTTP/1.1 200 OK
Cache-Control: no-store
Content-Type: application/json

{
  "access_token": "eyJhbGciOiJFUzI1NiIs...",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "rt_opaque_value_shown_once"
}
```

- کد ۳۰ ثانیه عمر داره و فقط یک بار مصرف میشه.
- کد اشتباه، verifier اشتباه، کد منقضی و کد مصرف‌شده همه یک خطای یکسان می‌گیرن: `400 BOOTSTRAP_INVALID` . این حالت‌ها از بیرون قابل تشخیص نیستن و نباید باشن.
- `Origin` باید دقیقا Origin بازی باشه، وگرنه جواب `403 ORIGIN_NOT_ALLOWED` هست.
- اگه خروج همین `sid` زودتر از bootstrap رسیده باشه، جواب `401 SESSION_REVOKED` هست. اگه شریک یا محیط وسط همین bootstrap غیرفعال بشه، جواب `403 PARTNER_SUSPENDED` هست.
- پاسخ‌های توکن `no-store` هستن و CORS عمومی ندارن.

عمر توکن‌ها

توکن	عمر	نکته
launch JWT	حداکثر ۱۲۰ ثانیه	یک‌بارمصرف با `jti`
bootstrap code	۳۰ ثانیه	یک‌بارمصرف و گره‌خورده به verifier
access token بازی	۹۰۰ ثانیه	توکن نشست بازی فان‌لندز که شریک بازیکن رو هم با خودش داره. برای API گزارش کار نمی‌کنه.
refresh token	چرخشی، با سقف مطلق ۱۲ ساعت از launch	بعد از ۱۲ ساعت ورود دوباره لازمه
logout token	حداکثر ۱۲۰ ثانیه	یک‌بارمصرف با `jti`
client assertion	حداکثر ۶۰ ثانیه	یک‌بارمصرف با `jti`
توکن API گزارش	۹۰۰ ثانیه	refresh نداره و باید توکن تازه بگیرید

تمدید نشست: `POST /v1/auth/session/refresh`

این endpoint رو صفحه بازی صدا می‌زنه و همون endpoint تمدید نشست عادی فان‌لندزه. بدنه و پاسخش قالب camelCase فان‌لندز رو دارن، پاسخ موفق داخل پاکت `{ "state": true, "data": ... }` میاد و هیچ هدر شریکی لازم نیست.

HTTP

```
POST /v1/auth/session/refresh HTTP/1.1
Host: api.sandbox.funlands.example
Content-Type: application/json

{ "refreshToken": "rt_opaque_value_shown_once" }

HTTP/1.1 200 OK
Content-Type: application/json

{
```

```
  "state": true,
  "data": {
    "accessToken": "eyJhbGciOi...",
    "refreshToken": "rt_rotated_opaque_value",
    "accessTokenExpiresAt": "2026-11-05T09:45:00.000Z",
    "refreshTokenExpiresAt": "2026-11-05T21:30:00.000Z"
  }
}

HTTP/1.1 401 Unauthorized
Content-Type: application/json

// after a back-channel logout: the session is already deleted
{ "state": false, "code": "TOKEN_EXPIRED", "message": "توکن منقضی شده است" }
```

- بدنه فقط یک فیلد داره: `data` . پاسخ موفق چهار فیلد داره: `accessToken` ، `refreshToken` ، `accessTokenExpiresAt` و `refreshTokenExpiresAt` (هر دو زمان به ISO ۸۶۰۱ و UTC).
- هر refresh موفق یک refresh token تازه میده و قبلی رو مصرف‌شده علامت می‌زنه. access token نشست شریک حداکثر ۹۰۰ ثانیه عمر داره.
- refresh token ناشناخته یا منقضی، یا refresh بعد از سقف ۱۲ ساعته، جواب `401 TOKEN_EXPIRED` می‌گیره.
- اگه یک refresh token که قبلا چرخیده دوباره استفاده بشه، همه نشست‌های اون بازیکن باطل میشن و جواب `401 TOKEN_REUSE` هست. کاربر باید دوباره launch بشه.
- خروج سرور به سرور نشست رو همون لحظه حذف می‌کنه، پس refresh بعد از اون معمولا `401 TOKEN_EXPIRED` می‌گیره و access token همون نشست هم از اون به بعد `401 TOKEN_EXPIRED` می‌گیره. `401 SESSION_REVOKED` فقط وقتی میاد که refresh قبل از حذف نشست برسه، مثلا وقتی خروج یا غیرفعال شدن محیط ثبت شده ولی بستن نشست هنوز کامل نشده (خروجی که `503` گرفته و هنوز تکرار نشده).
- اگه شریک غیرفعال بشه، access token همه بازیکن‌هاش از همون لحظه روی هر درخواستی که کاربر لازم داره و روی handshake جواب `401 TOKEN_EXPIRED` می‌گیره. روی مسیرهای عمومی، درخواست بدون کاربر ادامه پیدا می‌کنه. refresh نشستی که هنوز حذف نشده `401 SESSION_REVOKED` می‌گیره و همون موقع حذف میشه. نشستی که با launch باز شده با غیرفعال شدن شریک باطل میشه و کمی بعد حذف میشه، و از اون به بعد refreshش `TOKEN_EXPIRED` می‌گیره. نشستی که بدون launch باز شده (OTP، رمز یا مهمان با کلید همون شریک) تا اولین refresh می‌مونه و همون refresh با `SESSION_REVOKED` رد میشه. اتصال‌های WebSocket باز هم توی ضربان بعدی، حدود ۲۵ ثانیه، بسته میشن.
- هر دو کد `TOKEN_EXPIRED` و `SESSION_REVOKED` یعنی «نشست تموم شده»: صفحه بازی دیگه refresh نمی‌کنه و `funlands:relaunch_requested` می‌فرسته.
- توکن‌ها توی حافظه صفحه نگه داشته میشن، نه توی `localStorage` . با reload صفحه از بین میرن و ورود دوباره لازم میشه.

فضای شریک در صفحه بازی

بعد از bootstrap، صفحه بازی روی هر درخواست HTTP به APIهای بازی فقط `Authorization: Bearer <access_token>` رو می‌فرسته و روی هر اتصال WebSocket همین توکن رو با `auth.token` در handshake میده. هیچ کلید شریکی لازم نیست: access token بازیکن شریک، شریک خودش رو با خودش داره و فان‌لندز فضای شریک رو از توکن تاییدشده می‌خونه. refresh هم هدر شریک نمی‌خواد.

JavaScript (browser)

```
// Runs in the game page. The access token from bootstrap carries the partner scope; no partner key is sent.
const res = await fetch('/v1/profile', {
  headers: { authorization: `Bearer ${session.accessToken}` },
  credentials: 'omit',
});
```

```
// socket.io: the same access token in the handshake
const socket = io('/game', { auth: { token: session.accessToken } });
```

- کلیدهای شریک اختیاری‌ان و فقط برای درخواست‌های بدون توکن به کار میان (OTP، ورود، مهمان و خوندن‌های عمومی)، اونم اگه از این مسیرها استفاده کنید. شریک `client_direct` برای این کار `x-partner-api-key` رو می‌فرسته. فان‌لندز فقط hash این کلید رو نگه می‌داره: مقدار خام فقط یک بار، موقع صدور، ساخته میشه و از طریق برگ شریک به شما می‌رسه. با صدور کلید تازه، کلید قبلی همون لحظه باطل میشه. کلید رو لاگ نکنید و توی URL نذارید.
- اگه درخواست یا handshakeای که توکن داره کلید شریک هم داشته باشه، اون کلید باید مال همون شریکی باشه که توکن براش صادر شده، وگرنه جواب `403 PARTNER_TOKEN_MISMATCH` هست. توکن پلتفرم، یعنی توکنی که مال هیچ شریکی نیست، همراه کلید شریک همین جواب رو می‌گیره. کلید ناشناخته یا چرخیده روی هر درخواستی `401 PARTNER_KEY_INVALID` می‌گیره.
- شریک‌های `server_to_server` با `X-Partner-Public-Key` و `X-Partner-Secret` کار می‌کنن و این سکرت هیچ‌وقت نباید به مرورگر، اپ یا صفحه بازی برسه. بازیکن‌های این شریک‌ها هم فقط با access token از مرورگر بازی می‌کنن، پس سکرت فقط سمت سرور شما می‌مونه.

ورود دوباره با postMessage

توی iframe دو صفحه مختلف ممکنه به صفحه شما پیام بدن و هر دو روی Origin بازی هستن. صفحه handoff فان‌لندز (پاسخ launch) فقط `funlands:launch_failed` رو می‌فرسته. `funlands:relaunch_requested` رو خود صفحه بازی می‌فرسته، وقتی نشستی نداره: بعد از reload، یا وقتی refresh با `TOKEN_EXPIRED` یا `SESSION_REVOKED` رد شده. توی پیام‌ها هیچ توکن یا `sub` ی نمیاد.

جهت	پیام	معنی
صفحه بازی به صفحه شما	`{"type":"funlands:relaunch_requested","request_id":"r_123"}`	launch تازه لازمه.
صفحه شما به صفحه بازی	`{"type":"partner:relaunch_ready","request_id":"r_123"}`	فرم تازه آماده‌ست و همین الان فرستاده میشه.
صفحه handoff به صفحه شما	`{"type":"funlands:launch_failed","code":"TOKEN_EXPIRED","request_id":"req_01JD3X8Q2M5N7P9R"}`	launch یا bootstrap رد شده. `code` یکی از کدهای بخش خطاهاست و `request_id` شناسه درخواست launch در فان‌لندزه (`X-Correlation-Id`)، نه شناسه relaunch.

در هر دو طرف `event.origin` رو با Origin بازی (`FUNLANDS_GAME_ORIGIN`) و `event.source` رو با پنجره iframe بسنجید و پیام رو با همین `targetOrigin` دقیق بفرستید، نه `*`. اگه launch از Origin ثبت‌نشده بیاد، صفحه handoff ساخته نمیشه، پیامی هم نمیاد و iframe فقط بدنه خطای JSON رو نشون میده. اگه سرور شما در دسترس نیست، یک پیام قابل فهم با دکمه تلاش دوباره نشون بدید.

JavaScript (browser)

```
// Runs in your game page (the page that holds the iframe and the launch form).
const FUNLANDS_GAME_ORIGIN = 'https://play.sandbox.funlands.example';   // from your registration sheet; productio
n: https://play.funlands.example
const frame = document.querySelector('iframe[name="funlands-game"]');

window.addEventListener('message', async (event) => {
  if (event.origin !== FUNLANDS_GAME_ORIGIN || event.source !== frame.contentWindow) return;
  const msg = event.data;
  if (typeof msg !== 'object' || msg === null || typeof msg.request_id !== 'string') return;

  if (msg.type === 'funlands:relaunch_requested') {
    // Sent by the game page itself when it has no session (reload, or TOKEN_EXPIRED / SESSION_REVOKED on refresh).
    // Ask YOUR server for a fresh launch (new jti, new verifier). Never reuse the old form values.
    const res = await fetch(`/play/${encodeURIComponent(gameId)}/launch-form`, { credentials: 'same-origin', cache:
```

```
'no-store' });
    if (!res.ok) return showRetry();
    const { token, verifier } = await res.json();
    const form = document.getElementById('funlands-launch');
    form.elements.launch_token.value = token;
    form.elements.bootstrap_verifier.value = verifier;
    frame.contentWindow.postMessage({ type: 'partner:relaunch_ready', request_id: msg.request_id }, FUNLANDS_GAME_O
RIGIN);
    form.submit();
  }

  if (msg.type === 'funlands:launch_failed') {
    // From the handoff page. msg.code is one of the error codes in the errors section;
    // msg.request_id is the launch request's X-Correlation-Id. Log both.
    reportLaunchFailure(msg.code, msg.request_id);
  }
});
```

launch تکراری

ممکنه یک فرم دو بار فرستاده بشه (دو بار کلیک، تکرار مرورگر یا قطعی شبکه). فان‌لندز برای یک `jti` هیچ‌وقت دو بازیکن یا دو نشست نمی‌سازه:

درخواست دوم با همون `jti`	نتیجه
همون محتوای توکن، verifier درست، قبل از `exp`	`200` با یک کد bootstrap تازه برای همون نشست، حداکثر ۳ بار.
همون محتوا، بدون verifier یا با verifier نادرست	`401 TOKEN_REPLAYED`
محتوای متفاوت با همون `jti`	`409 LAUNCH_PAYLOAD_CHANGED`
بعد از `exp` یا بعد از ۳ بار	`401 TOKEN_EXPIRED` یا `401 TOKEN_REPLAYED`. launch تازه بسازید.

راه درست «دوباره امتحان کردن» همیشه یک launch تازه با `jti` و verifier تازه‌ست.

۵ خروج و حذف حساب

وقتی کاربر از اپ شما خارج میشه یا نشستش رو باطل می‌کنید، نشست بازیش هم باید بسته بشه. این کار سرور به سرور انجام میشه و به مرورگر کاربر بستگی نداره.

توکن خروج و `POST /partner/v1/backchannel-logout`

فیلد	قاعده
هدر `typ`	`logout+jwt`
`iss`	issuer ثبت‌شده شما
`aud`	audience خروج همین محیط (جدا از audience launch)
`sub` ، `sid`	همون مقدارهایی که توی launch فرستادید. `sid` اجباریه و خروج همه نشست‌های یک `sub` بدون `sid` فقط با توافق جدا ممکنه.
`exp` ، `iat`	`exp` حداکثر ۱۲۰ ثانیه بعد از `iat`
`jti`	یکتا و یک‌بارمصرف
`events`	اجباری: `{"http://schemas.openid.net/event/backchannel-logout": {}}`
`nonce`	ممنوع. توکنی که `nonce` داره رد میشه.

JSON

```
// header
{ "alg": "ES256", "typ": "logout+jwt", "kid": "partner-sandbox-2026-11" }

// claims
{
  "iss": "https://partner.example",
  "aud": "https://api.sandbox.funlands.example/partner-logout",
  "sub": "u_7f3a9c2e5b1d4e08",
  "sid": "5d0c6a1e-2f9b-4c37-9a51-0e6b8f2d7c44",
  "iat": 1793871000,
  "exp": 1793871120,
  "jti": "7c1e4b2a-9d3f-4a68-b5e0-2f8c6d1a9b37",
  "events": { "http://schemas.openid.net/event/backchannel-logout": {} }
}
```

HTTP

```
POST /partner/v1/backchannel-logout HTTP/1.1
Host: api.sandbox.funlands.example
Content-Type: application/x-www-form-urlencoded

logout_token=eyJhbGciOiJFUzI1NiIsInR5cCI6ImxvZ291dCtqd3QiLCJraWQiOiJwYXJ0bmVyLXNhbmRib3gtMjAyNi0xMSJ9.eyJ...

HTTP/1.1 200 OK
```

- پاسخ `200` یعنی خروج انجام شده یا قبلا انجام شده بوده. فرستادن دوباره همون درخواست بی‌خطره.
- به محض قبول شدن، نشست و خانواده refreshش باطل میشن و اتصال‌های زنده بازی (WebSocket) بسته میشن. access tokenی که قبلا صادر شده دیگه به APIهای حساس دسترسی نداره، چون وضعیت نشست توی هر درخواست چک میشه.
- خطاهای این endpoint همون کدهای توکن launch هستن (`TOKEN_TYPE_INVALID` ، `AUDIENCE_MISMATCH` ، `TOKEN_EXPIRED` و بقیه).

- اگه `5xx` یا خطای شبکه گرفتید، درخواست رو با یک توکن تازه (`jti` ، `iat` و `exp` تازه) تکرار کنید. خروج روی `sid` idempotent هست و تکرارش مشکلی نمی‌سازه.
- خروج رو توی یک صف پس‌زمینه بفرستید تا کندی شبکه خروج کاربر رو معطل نکنه.

نمونه: ارسال خروج سرور به سرور

NestJS

```
// src/funlands/funlands-logout.service.ts
import { Injectable, Logger } from '@nestjs/common';
import { SignJWT } from 'jose';
import { randomUUID } from 'node:crypto';
import { FUNLANDS_BASE, LAUNCH_KID, PARTNER_ISSUER } from './funlands.config';
import { LaunchTokenService } from './launch-token.service';

const LOGOUT_AUDIENCE = `${FUNLANDS_BASE}/partner-logout`;
const BACKCHANNEL_LOGOUT_EVENT = 'http://schemas.openid.net/event/backchannel-logout';

@Injectable()
export class FunlandsLogoutService {
  private readonly logger = new Logger(FunlandsLogoutService.name);

  constructor(private readonly launch: LaunchTokenService) {}   // reuses the loaded signing key

  async send(subject: string, sid: string): Promise<void> {
    const now = Math.floor(Date.now() / 1000);
    const logoutToken = await new SignJWT({ sid, events: { [BACKCHANNEL_LOGOUT_EVENT]: {} } })
      .setProtectedHeader({ alg: 'ES256', typ: 'logout+jwt', kid: LAUNCH_KID })
      .setIssuer(PARTNER_ISSUER)
      .setAudience(LOGOUT_AUDIENCE)
      .setSubject(subject)
      .setIssuedAt(now)
      .setExpirationTime(now + 120)
      .setJti(randomUUID())
      .sign(this.launch.signingKey());

    const res = await fetch(`${FUNLANDS_BASE}/partner/v1/backchannel-logout`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
      body: new URLSearchParams({ logout_token: logoutToken }),
      signal: AbortSignal.timeout(5000),
    });
    if (res.status !== 200) {
      // 200 = executed or already executed. Otherwise queue a retry with a NEW token (new jti, iat, exp).
      this.logger.warn(`back-channel logout failed: ${res.status}`);
      throw new Error(`backchannel-logout ${res.status}`);
    }
  }
}

// src/auth/auth.controller.ts (your logout route)
@Post('logout')
async logout(@Req() req: Request) {
  const session = req.session as unknown as { funlandsSid?: string };
  const user = req.user as { funlandsSubject: string } | undefined;
  if (user && session.funlandsSid) {
    await this.funlandsLogout.send(user.funlandsSubject, session.funlandsSid)
      .catch(() => this.retryQueue.add('funlands-logout', { sub: user.funlandsSubject, sid: session.funlandsSid
}));
  }
  await new Promise<void>((resolve) => req.session.destroy(() => resolve()));
}
```

Express

```
// funlands-logout.mjs
import { importPKCS8, SignJWT } from 'jose';
import { randomUUID } from 'node:crypto';
import { readFile } from 'node:fs/promises';
```

```
const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';                // production: https://api.funlands.example
const LOGOUT_AUDIENCE = `${FUNLANDS_BASE}/partner-logout`;
const PARTNER_ISSUER = 'https://partner.example';
const LAUNCH_KID = 'partner-sandbox-2026-11';
const privateKey = await importPKCS8(await readFile(`keys/${LAUNCH_KID}.pem`, 'utf8'), 'ES256');

export async function sendBackchannelLogout(subject, sid) {
  const now = Math.floor(Date.now() / 1000);
  const logoutToken = await new SignJWT({
    sid,
    events: { 'http://schemas.openid.net/event/backchannel-logout': {} },
  })
    .setProtectedHeader({ alg: 'ES256', typ: 'logout+jwt', kid: LAUNCH_KID })
    .setIssuer(PARTNER_ISSUER)
    .setAudience(LOGOUT_AUDIENCE)
    .setSubject(subject)
    .setIssuedAt(now)
    .setExpirationTime(now + 120)
    .setJti(randomUUID())
    .sign(privateKey);

  const res = await fetch(`${FUNLANDS_BASE}/partner/v1/backchannel-logout`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({ logout_token: logoutToken }),
    signal: AbortSignal.timeout(5000),
  });
  if (res.status !== 200) throw new Error(`backchannel-logout ${res.status}`); // retry later with a new token
}

// your logout route
router.post('/logout', requireLogin, async (req, res) => {
  const { funlandsSid } = req.session;
  if (funlandsSid) {
    await sendBackchannelLogout(req.user.funlandsSubject, funlandsSid)
      .catch((err) => retryQueue.add({ sub: req.user.funlandsSubject, sid: funlandsSid, err: err.message }));
  }
  req.session.destroy(() => res.redirect('/'));
});
```

FastAPI

```
# app/funlands_logout.py
import time
import uuid

import httpx
import jwt

from app.launch import FUNLANDS_BASE, LAUNCH_KID, PARTNER_ISSUER, PRIVATE_KEY

LOGOUT_AUDIENCE = f"{FUNLANDS_BASE}/partner-logout"
BACKCHANNEL_LOGOUT_EVENT = "http://schemas.openid.net/event/backchannel-logout"


async def send_backchannel_logout(subject: str, sid: str) -> None:
    now = int(time.time())
    claims = {
        "iss": PARTNER_ISSUER, "aud": LOGOUT_AUDIENCE, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": str(uuid.uuid4()),
        "events": {BACKCHANNEL_LOGOUT_EVENT: {}},
    }
    logout_token = jwt.encode(claims, PRIVATE_KEY, algorithm="ES256",
                              headers={"typ": "logout+jwt", "kid": LAUNCH_KID})
    async with httpx.AsyncClient(timeout=5, follow_redirects=False) as client:
        res = await client.post(f"{FUNLANDS_BASE}/partner/v1/backchannel-logout",
                                data={"logout_token": logout_token})
    if res.status_code != 200:   # 200 = executed or already executed
        raise RuntimeError(f"backchannel-logout {res.status_code}")   # retry with a NEW token
```

```python
# app/auth_routes.py (your logout route)
from fastapi import APIRouter, BackgroundTasks, Depends, Request
from fastapi.responses import RedirectResponse

from app.auth import current_user
from app.funlands_logout import send_backchannel_logout

router = APIRouter()


@router.post("/logout")
async def logout(request: Request, background: BackgroundTasks, user=Depends(current_user)):
    sid = request.session.get("funlands_sid")
    if sid:
        background.add_task(send_backchannel_logout, user.funlands_subject, sid)  # use a durable queue in producti
on
    request.session.clear()
    return RedirectResponse("/", status_code=303)
```

Django

```python
# partner/funlands_logout.py
import time
import uuid

import jwt
import requests
from django.contrib.auth.signals import user_logged_out
from django.dispatch import receiver

from .funlands import FUNLANDS_BASE, LAUNCH_KID, PARTNER_ISSUER, private_key

LOGOUT_AUDIENCE = f"{FUNLANDS_BASE}/partner-logout"
BACKCHANNEL_LOGOUT_EVENT = "http://schemas.openid.net/event/backchannel-logout"


def send_backchannel_logout(subject: str, sid: str) -> None:
    now = int(time.time())
    claims = {
        "iss": PARTNER_ISSUER, "aud": LOGOUT_AUDIENCE, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": str(uuid.uuid4()),
        "events": {BACKCHANNEL_LOGOUT_EVENT: {}},
    }
    logout_token = jwt.encode(claims, private_key(LAUNCH_KID), algorithm="ES256",
                              headers={"typ": "logout+jwt", "kid": LAUNCH_KID})
    res = requests.post(f"{FUNLANDS_BASE}/partner/v1/backchannel-logout",
                        data={"logout_token": logout_token}, timeout=5, allow_redirects=False)
    if res.status_code != 200:
        raise RuntimeError(f"backchannel-logout {res.status_code}")   # retry with a NEW token


# Fired by django.contrib.auth.logout() before the session is flushed.
@receiver(user_logged_out)
def notify_funlands(sender, request, user, **kwargs):
    sid = request.session.get("funlands_sid")
    if user is not None and sid:
        # Prefer a task queue (Celery, RQ) so a slow call never blocks logout.
        send_backchannel_logout(user.funlands_subject, sid)

# Import this module in PartnerConfig.ready() so the receiver is connected.
```

Laravel

```php
<?php
// app/Jobs/SendFunlandsLogout.php
namespace App\Jobs;

use Firebase\JWT\JWT;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
```

```php
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Str;

final class SendFunlandsLogout implements ShouldQueue
{
    use Dispatchable, Queueable;

    public int $tries = 6;
    public array $backoff = [10, 30, 60, 300, 900];

    public function __construct(public string $subject, public string $sid) {}

    public function handle(): void
    {
        $now = time();
        $kid = config('funlands.launch_kid');
        // Built on every attempt: a retry always carries a new jti, iat and exp.
        $logoutToken = JWT::encode([
            'iss' => config('funlands.issuer'),
            'aud' => config('funlands.base') . '/partner-logout',
            'sub' => $this->subject,
            'sid' => $this->sid,
            'iat' => $now,
            'exp' => $now + 120,
            'jti' => (string) Str::uuid(),
            'events' => ['http://schemas.openid.net/event/backchannel-logout' => new \stdClass()], // {} not []
        ], file_get_contents(storage_path("keys/{$kid}.pem")), 'ES256', $kid, ['typ' => 'logout+jwt']);

        Http::asForm()->timeout(5)->withoutRedirecting()
            ->post(config('funlands.base') . '/partner/v1/backchannel-logout', ['logout_token' => $logoutToken])
            ->throw();                      // anything but 2xx -> job retried
    }
}

// app/Listeners/NotifyFunlandsOnLogout.php  (listens to Illuminate\Auth\Events\Logout)
namespace App\Listeners;

use App\Jobs\SendFunlandsLogout;
use Illuminate\Auth\Events\Logout;

final class NotifyFunlandsOnLogout
{
    public function handle(Logout $event): void
    {
        $sid = session('funlands_sid');      // the session is still intact when Logout fires
        if ($event->user && $sid) {
            SendFunlandsLogout::dispatch($event->user->funlands_subject, $sid);
        }
    }
}
```

PHP

```php
<?php
// src/funlands_logout.php
declare(strict_types=1);

use Firebase\JWT\JWT;
use GuzzleHttp\Client;

function send_backchannel_logout(string $subject, string $sid): void
{
    $base = 'https://api.sandbox.funlands.example';                    // production: https://api.funlands.example
    $kid = 'partner-sandbox-2026-11';
    $now = time();
    $logoutToken = JWT::encode([
        'iss' => 'https://partner.example',
        'aud' => $base . '/partner-logout',
        'sub' => $subject,
        'sid' => $sid,
```

```php
        'iat' => $now,
        'exp' => $now + 120,
        'jti' => bin2hex(random_bytes(16)),
        'events' => ['http://schemas.openid.net/event/backchannel-logout' => new stdClass()], // {} not []
    ], file_get_contents(__DIR__ . "/../keys/{$kid}.pem"), 'ES256', $kid, ['typ' => 'logout+jwt']);

    $res = (new Client(['timeout' => 5, 'allow_redirects' => false, 'http_errors' => false]))
        ->post($base . '/partner/v1/backchannel-logout', ['form_params' => ['logout_token' => $logoutToken]]);
    if ($res->getStatusCode() !== 200) {
        throw new RuntimeException('backchannel-logout ' . $res->getStatusCode()); // retry with a NEW token
    }
}

// public/logout.php
session_start();
if (isset($_SESSION['user'], $_SESSION['funlands_sid'])) {
    try {
        send_backchannel_logout($_SESSION['user']['funlands_subject'], $_SESSION['funlands_sid']);
    } catch (Throwable $e) {
        enqueue_retry('funlands-logout', $_SESSION['user']['funlands_subject'], $_SESSION['funlands_sid']); // your
queue
    }
}
session_destroy();
header('Location: /', true, 303);
```

Go

```go
// logout.go
package partner

import (
    "fmt"
    "net/http"
    "net/url"
    "strings"
    "time"

    "github.com/lestrrat-go/jwx/v2/jwk"
)

const LogoutAudience = FunlandsBase + "/partner-logout"

var logoutHTTP = &http.Client{Timeout: 5 * time.Second,
    CheckRedirect: func(*http.Request, []*http.Request) error { return http.ErrUseLastResponse }}

// SendBackchannelLogout signs a fresh logout token on every call, so retries are safe.
func SendBackchannelLogout(key jwk.Key, subject, sid string) error {
    now := time.Now().Unix()
    token, err := SignJWT(key, "logout+jwt", map[string]any{
        "iss": PartnerIssuer, "aud": LogoutAudience, "sub": subject, "sid": sid,
        "iat": now, "exp": now + 120, "jti": randomB64(16),
        "events": map[string]any{"http://schemas.openid.net/event/backchannel-logout": map[string]any{}},
    })
    if err != nil {
        return err
    }
    form := url.Values{"logout_token": {token}}
    res, err := logoutHTTP.Post(FunlandsBase+"/partner/v1/backchannel-logout",
        "application/x-www-form-urlencoded", strings.NewReader(form.Encode()))
    if err != nil {
        return err
    }
    defer res.Body.Close()
    if res.StatusCode != http.StatusOK { // 200 = executed or already executed
        return fmt.Errorf("backchannel-logout %d", res.StatusCode)
    }
    return nil
}

// In your logout handler, before deleting the session:
```

```
//   if err := partner.SendBackchannelLogout(key, sess.FunlandsSubject, sess.FunlandsSID); err != nil {
//       retryQueue.Enqueue(sess.FunlandsSubject, sess.FunlandsSID)
//   }
```

Spring Boot

```
// src/main/java/com/partner/funlands/FunlandsLogoutClient.java
package com.partner.funlands;

import static com.partner.funlands.FunlandsConfig.*;

import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.ECDSASigner;
import com.nimbusds.jwt.*;
import java.time.Instant;
import java.util.*;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.web.client.RestClient;

@Service
public class FunlandsLogoutClient {
    private final RestClient http = RestClient.create(BASE);
    private final JWSSigner signer;

    public FunlandsLogoutClient() throws Exception {
        this.signer = new ECDSASigner(PemKeys.privateKey(LAUNCH_KID));
    }

    /** Signs a fresh logout token per call; throws on anything but 200. */
    public void send(String subject, String sid) throws JOSEException {
        Instant now = Instant.now();
        var jwt = new SignedJWT(
            new JWSHeader.Builder(JWSAlgorithm.ES256).type(new JOSEObjectType("logout+jwt")).keyID(LAUNCH_KID).buil
d(),
            new JWTClaimsSet.Builder()
                .issuer(PARTNER_ISSUER).audience(LOGOUT_AUDIENCE).subject(subject)
                .issueTime(Date.from(now)).expirationTime(Date.from(now.plusSeconds(120)))
                .jwtID(UUID.randomUUID().toString())
                .claim("sid", sid)
                .claim("events", Map.of("http://schemas.openid.net/event/backchannel-logout", Map.of()))
                .build());
        jwt.sign(signer);

        var form = new LinkedMultiValueMap<String, String>();
        form.add("logout_token", jwt.serialize());
        http.post().uri("/partner/v1/backchannel-logout")
            .contentType(MediaType.APPLICATION_FORM_URLENCODED).body(form)
            .retrieve().toBodilessEntity();          // 4xx/5xx throw: queue a retry
    }
}

// SecurityConfig.java (inside your SecurityFilterChain)
http.logout(logout -> logout.addLogoutHandler((request, response, auth) -> {
    var session = request.getSession(false);
    String sid = session == null ? null : (String) session.getAttribute("funlands_sid");
    if (auth != null && sid != null && auth.getPrincipal() instanceof PartnerUser user) {
        try {
            funlandsLogout.send(user.funlandsSubject(), sid);
        } catch (Exception e) {
            retryQueue.enqueue(user.funlandsSubject(), sid);   // your durable retry
        }
    }
}));
```

ASP.NET Core

```
// Funlands/FunlandsLogoutClient.cs  (register as singleton)
public sealed class FunlandsLogoutClient(PartnerSigner signer)
{
    private readonly HttpClient _http = new(new HttpClientHandler { AllowAutoRedirect = false })
```

```
    {
        BaseAddress = new Uri(FunlandsConfig.Base),
        Timeout = TimeSpan.FromSeconds(5),
    };
    private readonly Microsoft.IdentityModel.Tokens.SigningCredentials _credentials =
        signer.Credentials(FunlandsConfig.LaunchKid);

    /// Signs a fresh logout token per call, so retries are safe.
    public async Task SendAsync(string subject, string sid)
    {
        var logoutToken = signer.Sign("logout+jwt", _credentials,
            FunlandsConfig.PartnerIssuer, FunlandsConfig.LogoutAudience, 120, new()
            {
                ["sub"] = subject,
                ["sid"] = sid,
                ["jti"] = Guid.NewGuid().ToString(),
                ["events"] = new Dictionary<string, object>
                {
                    ["http://schemas.openid.net/event/backchannel-logout"] = new Dictionary<string, object>(),
                },
            });
        using var res = await _http.PostAsync("/partner/v1/backchannel-logout",
            new FormUrlEncodedContent(new Dictionary<string, string> { ["logout_token"] = logoutToken }));
        if ((int)res.StatusCode != 200)
            throw new HttpRequestException($"backchannel-logout {(int)res.StatusCode}");
    }
}

// Program.cs (your logout endpoint)
app.MapPost("/logout", async (HttpContext http, FunlandsLogoutClient funlands, IRetryQueue retries) =>
{
    var sub = http.User.FindFirst("funlands_sub")?.Value;
    var sid = http.User.FindFirst("funlands_sid")?.Value;
    if (sub is not null && sid is not null)
    {
        try { await funlands.SendAsync(sub, sid); }
        catch (Exception) { await retries.EnqueueAsync("funlands-logout", sub, sid); }
    }
    await http.SignOutAsync();
    return Results.Redirect("/");
}).RequireAuthorization();
```

cURL

```
# LOGOUT_TOKEN: header {"alg":"ES256","typ":"logout+jwt","kid":"partner-sandbox-2026-11"}
# claims: iss, aud=<logout audience>, sub, sid, iat, exp (<= iat + 120), jti,
#         events={"http://schemas.openid.net/event/backchannel-logout":{}}   (no nonce)
curl -i https://api.sandbox.funlands.example/partner/v1/backchannel-logout \
  --data-urlencode "logout_token=${LOGOUT_TOKEN}"

# HTTP/1.1 200 OK      -> session and refresh family revoked (or already revoked)
```

حذف حساب

وقتی کاربری حسابش رو توی اپ شما حذف می‌کنه، داده‌هاش در فان‌لندز هم باید حذف بشه. قاعده‌های این کار:

- درخواست حذف با یک `request_id` پایدار، `sub` و زمان درخواست ثبت میشه. ثبت دوباره همون `request_id` درخواست دوم نمی‌سازه.
- بلافاصله بعد از ثبت، همه نشست‌های اون بازیکن بسته میشن.
- بعد پیوند هویت و داده شخصی طبق قرارداد داده حذف یا ناشناس میشه. فقط یک رکورد حداقلی برای جلوگیری از پردازش تکراری و تایید پایان کار می‌مونه. آمارهای تجمیعی که واقعا شخصی نیستن ممکنه طبق توافق بمونن.
- «دریافت شد» و «حذف کامل شد» دو وضعیت جدا هستن. رویداد `player.deleted` فقط بعد از کامل شدن حذف براتون فرستاده میشه، نه موقع دریافت درخواست.

- endpoint ثبت درخواست حذف و مهلت پایان کار توی قرارداد داده تعیین میشه و بعد از امضای اون توی همین صفحه منتشر میشه. تا اون موقع درخواست حذف از کانالی که توی قرارداد اومده فرستاده میشه و خروج (بالا) رو هم هم‌زمان بفرستید.
- `player.deleted` همین الان هم فرستاده میشه، هر وقت حذف حساب یکی از بازیکن‌های شما در فان‌لندز کامل بشه. چون این حذف از درخواست شما نیومده، `request_id` این رویداد فعلا همیشه `null` هست.

۶ رویدادها و Webhook

فان‌لندز رویدادهای بازیکن‌های شما رو به نشانی Webhook ثبت‌شده می‌فرسته. هر رویداد دست‌کم یک بار تحویل میشه، یعنی ممکنه تکراری برسه و ترتیب رسیدن هم تضمینی نیست. کلید یکتایی همیشه `event_id` هست.

قالب SET

بدنه درخواست یک Security Event Token (RFC 8417) هست: یک JWT که با کلید فان‌لندز امضا شده و نوع محتواش `application/secevent+jwt` هست.

HTTP

```
POST /funlands/events HTTP/1.1
Host: partner.example
Content-Type: application/secevent+jwt

eyJhbGciOiJFUzI1NiIsInR5cCI6InNlY2V2ZW50K2p3dCIsImtpZCI6ImZsLXNhbmRib3gtMjAyNi0xMSJ9.eyJ...
```

هدر و claimهای همون SET بعد از decode

JSON

```
// header
{ "alg": "ES256", "typ": "secevent+jwt", "kid": "fl-sandbox-2026-11" }

// claims
{
  "iss": "https://api.sandbox.funlands.example",
  "aud": "https://partner.example",
  "iat": 1793871100,
  "jti": "dlv_0193a0e2c4d57e6f8a90b1c2d3e4f5a6",
  "events": {
    "https://api.funlands.example/events/game.completed": {
      "event_id": "evt_0193a0e1b2c37d4e8f50a1b2c3d4e5f6",
      "schema_version": 1,
      "subject": { "format": "iss_sub", "iss": "https://partner.example", "sub": "u_7f3a9c2e5b1d4e08" },
      "player_seq": 42,
      "occurred_at": "2026-11-05T09:31:12Z",
      "game_id": "star-runner",
      "game_version": null,
      "score": 1250,
      "duration_ms": null,
      "result": "win"
    }
  }
}
```

فیلد	توضیح
`iss`	issuer فان‌لندز توی همین محیط، یعنی همون نشانی پایه‌ای که audienceهای این محیط روش ساخته شدن (token endpoint بدون `/oauth/token`).
`aud`	issuer خود شما توی همین محیط. توی برگ شریک با نام `set_audience` میاد.
`iat`، `jti`	مخصوص همین تلاش ارسال. هر تلاش دوباره یک SET تازه با `iat` و `jti` تازه‌ست. SET فیلد `exp` نداره.
`events`	یک شیء که کلیدش URI نوع رویداده، مثل `https://api.funlands.example/events/game.completed`. این URI از یک پیشوند ثابت و نام کوتاه رویداد ساخته میشه. پیشوند توی هر دو محیط یکسانه، از نشانی پایه API یا issuer محیط ساخته نمیشه و مقدار دقیقش توی برگ شریک میاد.
`event_id`	شناسه ثابت رویداد که توی تکرار و replay عوض نمیشه. کلید dedupe شما همینه، نه `jti`.
`schema_version`	نسخه قرارداد payload این نوع رویداد.
`subject`	`{"format":"iss_sub","iss":...,"sub":...}`، با همون `sub`ی که توی launch فرستادید.
`player_seq`	شمارنده‌ای که برای هر بازیکن یکی‌یکی زیاد میشه. جاافتادگی رو با همین پیدا می‌کنید.
`occurred_at`	زمان رخ دادن به UTC، با قالب ISO ۸۶۰۱ و دقت ثانیه.

انواع رویداد

نام کوتاه هر نوع، بخش آخر URI اونه. فهرست رویدادهایی که برای شما فعاله موقع ثبت مشخص میشه.

نوع	کِی	فیلدهای اختصاصی
`player.created`	اولین ورود موفق یک `sub`	`locale`، `age_band`
`session.started`	هر ورود موفق	`game_id`، `platform`
`session.ended`	پایان نشست؛ فعلا وقتی نشست با خروج back-channel شما بسته بشه	`duration_ms`، `games_played`
`game.started`	شروع یک دور یا مرحله	`game_id`، `game_version`، `level` (اختیاری)
`game.completed`	پایان معتبر یک دور که سرور تاییدش کرده (امتیازی که دور رو می‌بنده)، نه صرفا بسته شدن بازی	`game_id`، `game_version`، `score`، `result`، `duration_ms`
`achievement.unlocked`	باز شدن یک دستاورد	`achievement_id`
`player.suspended`	تعلیق بازیکن از سوی فان‌لندز	`reason_code`
`player.deleted`	بعد از کامل شدن حذف حساب بازیکن در فان‌لندز	`request_id`

اگه محاسبه یک عدد بعدا اصلاح بشه، یک رویداد اصلاحی جدا با `event_id` تازه فرستاده میشه که به رویداد اصلی اشاره می‌کنه. رویداد قبلی هیچ‌وقت بی‌صدا عوض نمیشه.

فعلا این رویدادها فرستاده میشن: `player.created`، `session.started`، `session.ended`، `game.completed` و `player.deleted`. `game.started`، `achievement.unlocked`، `player.suspended` و رویداد اصلاحی هنوز فرستاده نمیشن و وقتی فعال بشن با همین قالب و همین URIها میان. این فیلدها فعلا همیشه `null` هستن: `games_played` در `session.ended`، `game_version` و `duration_ms` در `game.completed`، و `request_id` در `player.deleted`. همون نتیجه‌ایه که بازی همراه امتیاز ثبت می‌کنه، `game.completed` در `result` یعنی `win`، `loss` یا `forfeit`، و اگه بازی نتیجه‌ای نفرسته `null` هست. فیلدها توی payload می‌مونن و کد شما باید `null` رو قبول کنه.

چک کردن و ذخیره رویداد

۱. نوع محتوا باید `application/secevent+jwt` باشه و بدنه حداکثر ۶۴ کیلوبایت. بدنه رو به‌شکل متن خام بخونید، نه JSON.

۲. توی هدر: `typ` برابر `secevent+jwt`، `alg` برابر `ES256` و `kid` موجود باشه، و هیچ‌کدوم از `jku`، `x5u`، `jwk`، `x5c` و `crit` نباشه.

۳. کلید رو با `kid` از JWKS فان‌لندز بردارید (کش ۵ دقیقه‌ای و یک بار خوندن دوباره برای `kid` ناشناخته).

۴. امضا رو چک کنید، و اینکه `iss` برابر issuer فان‌لندز توی همین محیط و `aud` برابر issuer خودتون (`set_audience`) باشه. این دو مقدار رو از تنظیمات خودتون بخونید، نه از درخواست.

۵. `iat` قدیمی‌تر از ۵ دقیقه یا بیشتر از ۳۰ ثانیه در آینده رو رد کنید.

۶. هر ورودی `events` رو با کلید یکتای `event_id` توی یک تراکنش ذخیره کنید. `event_id` تکراری خطا نیست: نادیده‌اش بگیرید و جواب موفق بدید.

۷. بعد از commit جواب `202` بدید و پردازش رو بعدا و جدا از روی جدول انجام بدید. جواب موفقی که فقط توی حافظه یا یک صف غیرماندگار ثبت شده قابل قبول نیست.

اگه مشکل از امضا یا claimهاست `401` بدید. اگه خودتون نتونستید JWKS فان‌لندز رو بخونید یا دیتابیس در دسترس نبود، `503` بدید تا ارسال تکرار بشه. `401` توی این حالت ارسال رو متوقف می‌کنه.

جدول پیشنهادی صندوق ورودی

SQL (PostgreSQL)

```sql
CREATE TABLE funlands_inbox (
  event_id     text PRIMARY KEY,          -- dedupe key, stable across retries
  type         text NOT NULL,             -- event type URI
  player_seq   bigint,
  payload      jsonb NOT NULL,
  received_at  timestamptz NOT NULL DEFAULT now(),
  processed_at timestamptz                -- set by your async worker
);
CREATE INDEX funlands_inbox_unprocessed ON funlands_inbox (received_at) WHERE processed_at IS NULL;
```

نمونه: چک کردن SET با JWKS فان‌لندز و ذخیره بدون تکرار

NestJS

```
// src/main.ts  (Nest's default parser ignores application/secevent+jwt)
const app = await NestFactory.create<NestExpressApplication>(AppModule);
```

```ts
app.useBodyParser('text', { type: 'application/secevent+jwt', limit: '64kb' });

// src/funlands/funlands-set.verifier.ts
import { Injectable } from '@nestjs/common';
import { createRemoteJWKSet, decodeProtectedHeader, jwtVerify } from 'jose';

const FUNLANDS_ISSUER = 'https://api.sandbox.funlands.example';                    // production: https://api.funl
ands.example
const FUNLANDS_JWKS = new URL('https://api.sandbox.funlands.example/.well-known/jwks.json');
const SET_AUDIENCE = 'https://partner.example';                                    // set_audience: your own issuer
const FORBIDDEN_HEADERS = ['jku', 'x5u', 'jwk', 'x5c', 'crit'];

export interface FunlandsEvent {
  type: string;                    // full event type URI
  event_id: string;
  schema_version: number;
  player_seq: number;
  occurred_at: string;
  subject: { format: 'iss_sub'; iss: string; sub: string };
  [field: string]: unknown;
}

export class SetRejected extends Error {}

@Injectable()
export class FunlandsSetVerifier {
  // Caches keys; an unknown kid triggers one refetch, at most once per 30 s.
  private readonly jwks = createRemoteJWKSet(FUNLANDS_JWKS, {
    timeoutDuration: 3000, cooldownDuration: 30_000, cacheMaxAge: 300_000,
  });

  async verify(token: string): Promise<FunlandsEvent[]> {
    let header: ReturnType<typeof decodeProtectedHeader>;
    try {
      header = decodeProtectedHeader(token);
    } catch {
      throw new SetRejected('malformed token');
    }
    if (header.typ !== 'secevent+jwt' || header.alg !== 'ES256' || !header.kid ||
        FORBIDDEN_HEADERS.some((name) => name in header)) {
      throw new SetRejected('unexpected header');
    }
    const { payload } = await jwtVerify(token, this.jwks, {
      algorithms: ['ES256'],
      typ: 'secevent+jwt',
      issuer: FUNLANDS_ISSUER,
      audience: SET_AUDIENCE,
      clockTolerance: 30,
      maxTokenAge: '5m',            // rejects iat older than 5 min or in the future
      requiredClaims: ['iat', 'jti', 'events'],
    });
    const events = payload.events as Record<string, Omit<FunlandsEvent, 'type'>>;
    return Object.entries(events).map(([type, body]) => ({ ...body, type }));
  }
}

// src/funlands/funlands-inbox.ts
import { Inject, Injectable } from '@nestjs/common';
import type { Pool } from 'pg';
import type { FunlandsEvent } from './funlands-set.verifier';

export const PG_POOL = Symbol('PG_POOL');

@Injectable()
export class FunlandsInbox {
  constructor(@Inject(PG_POOL) private readonly pool: Pool) {}

  // Durable and idempotent: a repeated event_id is a no-op, not an error.
  async storeOnce(events: FunlandsEvent[]) {
    const client = await this.pool.connect();
    try {
```

```
      await client.query('BEGIN');
      for (const e of events) {
        await client.query(
          `INSERT INTO funlands_inbox (event_id, type, player_seq, payload)
           VALUES ($1, $2, $3, $4) ON CONFLICT (event_id) DO NOTHING`,
          [e.event_id, e.type, e.player_seq ?? null, e],
        );
      }
      await client.query('COMMIT');
    } catch (err) {
      await client.query('ROLLBACK');
      throw err;
    } finally {
      client.release();
    }
  }
}

// src/funlands/funlands-webhook.controller.ts
import { Body, Controller, Headers, HttpCode, Post, ServiceUnavailableException,
  UnauthorizedException, UnsupportedMediaTypeException } from '@nestjs/common';
import { errors } from 'jose';
import { FunlandsInbox } from './funlands-inbox';
import { FunlandsSetVerifier, SetRejected } from './funlands-set.verifier';

@Controller('funlands')
export class FunlandsWebhookController {
  constructor(private readonly verifier: FunlandsSetVerifier, private readonly inbox: FunlandsInbox) {}

  @Post('events')
  @HttpCode(202)
  async receive(@Headers('content-type') contentType: string | undefined, @Body() body: unknown) {
    if (!contentType?.startsWith('application/secevent+jwt') || typeof body !== 'string') {
      throw new UnsupportedMediaTypeException();
    }
    let events;
    try {
      events = await this.verifier.verify(body.trim());
    } catch (err) {
      const rejected = err instanceof SetRejected ||
        (err instanceof errors.JOSEError && !(err instanceof errors.JWKSTimeout));
      // 401 tells Funlands to pause and alert; 503 makes it retry later.
      throw rejected ? new UnauthorizedException() : new ServiceUnavailableException();
    }
    await this.inbox.storeOnce(events);   // answer 202 only after the commit; process asynchronously
  }
}
```

Express

```
// webhook.mjs
import express from 'express';
import { createRemoteJWKSet, decodeProtectedHeader, errors, jwtVerify } from 'jose';
import pg from 'pg';

const FUNLANDS_ISSUER = 'https://api.sandbox.funlands.example';                    // production: https://api.funl
ands.example
const FUNLANDS_JWKS = new URL('https://api.sandbox.funlands.example/.well-known/jwks.json');
const SET_AUDIENCE = 'https://partner.example';
const FORBIDDEN_HEADERS = ['jku', 'x5u', 'jwk', 'x5c', 'crit'];

const jwks = createRemoteJWKSet(FUNLANDS_JWKS, { timeoutDuration: 3000, cooldownDuration: 30_000, cacheMaxAge: 300_
000 });
const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });

class SetRejected extends Error {}

async function verifySet(token) {
  let header;
  try {
    header = decodeProtectedHeader(token);
```

```
  } catch {
    throw new SetRejected('malformed token');
  }
  if (header.typ !== 'secevent+jwt' || header.alg !== 'ES256' || !header.kid ||
      FORBIDDEN_HEADERS.some((name) => name in header)) {
    throw new SetRejected('unexpected header');
  }
  const { payload } = await jwtVerify(token, jwks, {
    algorithms: ['ES256'], typ: 'secevent+jwt', issuer: FUNLANDS_ISSUER, audience: SET_AUDIENCE,
    clockTolerance: 30, maxTokenAge: '5m', requiredClaims: ['iat', 'jti', 'events'],
  });
  return Object.entries(payload.events).map(([type, body]) => ({ ...body, type }));
}

async function storeOnce(events) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    for (const e of events) {
      await client.query(
        `INSERT INTO funlands_inbox (event_id, type, player_seq, payload)
         VALUES ($1, $2, $3, $4) ON CONFLICT (event_id) DO NOTHING`,
        [e.event_id, e.type, e.player_seq ?? null, e],
      );
    }
    await client.query('COMMIT');
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release();
  }
}

export const router = express.Router();

router.post('/funlands/events', express.text({ type: 'application/secevent+jwt', limit: '64kb' }), async (req, res)
=> {
  if (typeof req.body !== 'string' || !req.is('application/secevent+jwt')) return res.sendStatus(415);
  let events;
  try {
    events = await verifySet(req.body.trim());
  } catch (err) {
    const rejected = err instanceof SetRejected ||
      (err instanceof errors.JOSEError && !(err instanceof errors.JWKSTimeout));
    return res.sendStatus(rejected ? 401 : 503);
  }
  try {
    await storeOnce(events);
  } catch {
    return res.sendStatus(503);      // Funlands retries with the same event_id
  }
  res.sendStatus(202);               // durable now; process from funlands_inbox asynchronously
});
```

FastAPI

```
# app/funlands_set.py
import time

import jwt
from jwt import PyJWKClient
from jwt.exceptions import PyJWKClientConnectionError

FUNLANDS_ISSUER = "https://api.sandbox.funlands.example"                 # production: https://api.funlands.exampl
e
FUNLANDS_JWKS = "https://api.sandbox.funlands.example/.well-known/jwks.json"
SET_AUDIENCE = "https://partner.example"
FORBIDDEN_HEADERS = {"jku", "x5u", "jwk", "x5c", "crit"}

# Caches the key set for 5 min and refetches once on an unknown kid.
```

```python
# PyJWKClient has no refetch cooldown: rate-limit this endpoint at your proxy.
jwks_client = PyJWKClient(FUNLANDS_JWKS, cache_jwk_set=True, lifespan=300, timeout=3)


class SetRejected(Exception):
    pass


def verify_set(token: str) -> list[dict]:
    try:
        header = jwt.get_unverified_header(token)
    except jwt.PyJWTError as exc:
        raise SetRejected("malformed token") from exc
    if (header.get("typ") != "secevent+jwt" or header.get("alg") != "ES256"
            or not header.get("kid") or FORBIDDEN_HEADERS & header.keys()):
        raise SetRejected("unexpected header")
    try:
        signing_key = jwks_client.get_signing_key_from_jwt(token)
        claims = jwt.decode(token, signing_key.key, algorithms=["ES256"], audience=SET_AUDIENCE,
                            issuer=FUNLANDS_ISSUER, leeway=30,
                            options={"require": ["iat", "jti", "events"]})
    except PyJWKClientConnectionError:
        raise                                       # infrastructure problem -> 503
    except jwt.PyJWTError as exc:
        raise SetRejected(str(exc)) from exc
    now = time.time()
    if not (now - 300 - 30 <= claims["iat"] <= now + 30):   # no older than 5 min, no future iat
        raise SetRejected("stale or future iat")
    return [{**body, "type": event_type} for event_type, body in claims["events"].items()]


# app/webhook.py
from fastapi import APIRouter, HTTPException, Request, Response
from fastapi.concurrency import run_in_threadpool
from jwt.exceptions import PyJWKClientConnectionError
from psycopg.types.json import Jsonb
from psycopg_pool import ConnectionPool

from app.funlands_set import SetRejected, verify_set

pool = ConnectionPool("postgresql://partner@localhost/partner", open=True)
router = APIRouter()


def store_once(events: list[dict]) -> None:
    with pool.connection() as conn, conn.transaction():
        for e in events:
            conn.execute(
                "INSERT INTO funlands_inbox (event_id, type, player_seq, payload) "
                "VALUES (%s, %s, %s, %s) ON CONFLICT (event_id) DO NOTHING",
                (e["event_id"], e["type"], e.get("player_seq"), Jsonb(e)),
            )


@router.post("/funlands/events", status_code=202)
async def funlands_events(request: Request) -> Response:
    if not request.headers.get("content-type", "").startswith("application/secevent+jwt"):
        raise HTTPException(415)
    body = await request.body()
    if len(body) > 64 * 1024:
        raise HTTPException(413)
    try:
        events = await run_in_threadpool(verify_set, body.decode("ascii").strip())
    except (SetRejected, UnicodeDecodeError):
        raise HTTPException(401)                    # Funlands pauses delivery and alerts
    except PyJWKClientConnectionError:
        raise HTTPException(503)                    # Funlands retries
    await run_in_threadpool(store_once, events)     # an exception here becomes 500 -> retry
    return Response(status_code=202)
```

Django

```python
# partner/funlands_set.py
import time

import jwt
from jwt import PyJWKClient
from jwt.exceptions import PyJWKClientConnectionError

FUNLANDS_ISSUER = "https://api.sandbox.funlands.example"                # production: https://api.funlands.exampl
e
FUNLANDS_JWKS = "https://api.sandbox.funlands.example/.well-known/jwks.json"
SET_AUDIENCE = "https://partner.example"
FORBIDDEN_HEADERS = {"jku", "x5u", "jwk", "x5c", "crit"}

jwks_client = PyJWKClient(FUNLANDS_JWKS, cache_jwk_set=True, lifespan=300, timeout=3)


class SetRejected(Exception):
    pass


def verify_set(token: str) -> list[dict]:
    try:
        header = jwt.get_unverified_header(token)
    except jwt.PyJWTError as exc:
        raise SetRejected("malformed token") from exc
    if (header.get("typ") != "secevent+jwt" or header.get("alg") != "ES256"
            or not header.get("kid") or FORBIDDEN_HEADERS & header.keys()):
        raise SetRejected("unexpected header")
    try:
        signing_key = jwks_client.get_signing_key_from_jwt(token)
        claims = jwt.decode(token, signing_key.key, algorithms=["ES256"], audience=SET_AUDIENCE,
                            issuer=FUNLANDS_ISSUER, leeway=30,
                            options={"require": ["iat", "jti", "events"]})
    except PyJWKClientConnectionError:
        raise
    except jwt.PyJWTError as exc:
        raise SetRejected(str(exc)) from exc
    now = time.time()
    if not (now - 300 - 30 <= claims["iat"] <= now + 30):
        raise SetRejected("stale or future iat")
    return [{**body, "type": event_type} for event_type, body in claims["events"].items()]


# partner/models.py
from django.db import models


class FunlandsInboxEvent(models.Model):
    event_id = models.CharField(max_length=64, primary_key=True)
    type = models.CharField(max_length=200)
    player_seq = models.BigIntegerField(null=True)
    payload = models.JSONField()
    received_at = models.DateTimeField(auto_now_add=True)
    processed_at = models.DateTimeField(null=True)

    class Meta:
        db_table = "funlands_inbox"


# partner/views.py
from django.db import transaction
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST
from jwt.exceptions import PyJWKClientConnectionError

from .funlands_set import SetRejected, verify_set
from .models import FunlandsInboxEvent
```

```python
@csrf_exempt
@require_POST
def funlands_events(request):
    if not request.content_type.startswith("application/secevent+jwt"):
        return HttpResponse(status=415)
    if len(request.body) > 64 * 1024:
        return HttpResponse(status=413)
    try:
        events = verify_set(request.body.decode("ascii").strip())
    except (SetRejected, UnicodeDecodeError):
        return HttpResponse(status=401)
    except PyJWKClientConnectionError:
        return HttpResponse(status=503)
    with transaction.atomic():
        FunlandsInboxEvent.objects.bulk_create(
            [FunlandsInboxEvent(event_id=e["event_id"], type=e["type"],
                                player_seq=e.get("player_seq"), payload=e) for e in events],
            ignore_conflicts=True,                    # repeated event_id is a no-op
        )
    return HttpResponse(status=202)

# partner/urls.py:  path("funlands/events", views.funlands_events)
```

Laravel

```php
<?php
// app/Funlands/FunlandsSetVerifier.php
namespace App\Funlands;

use Firebase\JWT\JWK;
use Firebase\JWT\JWT;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Http;

// app/Funlands/SetRejected.php:  final class SetRejected extends \RuntimeException {}

final class FunlandsSetVerifier
{
    private const ISSUER = 'https://api.sandbox.funlands.example';                  // production: https://api.funlands.example
    private const JWKS_URL = 'https://api.sandbox.funlands.example/.well-known/jwks.json';
    private const AUDIENCE = 'https://partner.example';
    private const FORBIDDEN_HEADERS = ['jku', 'x5u', 'jwk', 'x5c', 'crit'];

    /** @return list<array<string, mixed>> */
    public function verify(string $token): array
    {
        $parts = explode('.', $token);
        $header = count($parts) === 3 ? json_decode(JWT::urlsafeB64Decode($parts[0]), true) : null;
        if (! is_array($header) || ($header['typ'] ?? null) !== 'secevent+jwt' || ($header['alg'] ?? null) !== 'ES256'
            || empty($header['kid']) || array_intersect_key($header, array_flip(self::FORBIDDEN_HEADERS))) {
            throw new SetRejected('unexpected header');
        }

        $keys = $this->keySet(false);
        if (! isset($keys[$header['kid']])) {
            $keys = $this->keySet(true);           // unknown kid: one refetch
        }

        JWT::$leeway = 30;
        try {
            $claims = JWT::decode($token, $keys);  // signature, alg from the JWK, exp/nbf/iat bounds
        } catch (\UnexpectedValueException|\DomainException|\InvalidArgumentException $e) {
            throw new SetRejected($e->getMessage(), 0, $e);
        }
        if (($claims->iss ?? null) !== self::ISSUER || ($claims->aud ?? null) !== self::AUDIENCE
            || ! isset($claims->iat, $claims->jti, $claims->events) || $claims->iat < time() - 330) {
            throw new SetRejected('claims');
        }
```

```php
        $events = [];
        foreach (json_decode(json_encode($claims->events), true) as $type => $body) {
            $events[] = $body + ['type' => $type];
        }
        return $events;
    }

    private function keySet(bool $refresh): array
    {
        // At most one forced refetch per 30 s.
        if ($refresh && Cache::add('funlands:jwks:cooldown', true, 30)) {
            Cache::forget('funlands:jwks');
        }
        $jwks = Cache::remember('funlands:jwks', 300, fn () => Http::timeout(3)->withoutRedirecting()
            ->acceptJson()->get(self::JWKS_URL)->throw()->json());

        return JWK::parseKeySet($jwks);
    }
}

// app/Http/Controllers/FunlandsWebhookController.php
namespace App\Http\Controllers;

use App\Funlands\FunlandsSetVerifier;
use App\Funlands\SetRejected;
use Illuminate\Http\Client\HttpClientException;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

final class FunlandsWebhookController
{
    public function __invoke(Request $request, FunlandsSetVerifier $verifier)
    {
        abort_unless(str_starts_with((string) $request->header('Content-Type'), 'application/secevent+jwt'), 415);
        $token = trim($request->getContent());
        abort_if(strlen($token) > 65536, 413);

        try {
            $events = $verifier->verify($token);
        } catch (SetRejected) {
            abort(401);                     // Funlands pauses delivery and alerts
        } catch (HttpClientException) {
            abort(503);                     // JWKS unreachable: Funlands retries
        }

        DB::transaction(function () use ($events) {
            foreach ($events as $e) {
                DB::table('funlands_inbox')->insertOrIgnore([
                    'event_id' => $e['event_id'],
                    'type' => $e['type'],
                    'player_seq' => $e['player_seq'] ?? null,
                    'payload' => json_encode($e),
                ]);
            }
        });

        return response()->noContent(202);
    }
}

// routes/web.php
Route::post('/funlands/events', \App\Http\Controllers\FunlandsWebhookController::class);

// bootstrap/app.php: exclude the webhook from CSRF
->withMiddleware(function (Middleware $middleware) {
    $middleware->validateCsrfTokens(except: ['funlands/events']);
})
```

PHP

```php
<?php
// public/funlands-events.php   (route POST /funlands/events here)
declare(strict_types=1);
require __DIR__ . '/../vendor/autoload.php';

use Firebase\JWT\JWK;
use Firebase\JWT\JWT;
use GuzzleHttp\Client;
use GuzzleHttp\Exception\GuzzleException;

const FUNLANDS_ISSUER = 'https://api.sandbox.funlands.example';                    // production: https://api.funla
nds.example
const FUNLANDS_JWKS = 'https://api.sandbox.funlands.example/.well-known/jwks.json';
const SET_AUDIENCE = 'https://partner.example';
const FORBIDDEN_HEADERS = ['jku', 'x5u', 'jwk', 'x5c', 'crit'];

final class SetRejected extends RuntimeException {}

function funlands_jwks(bool $refresh): array
{
    $cache = sys_get_temp_dir() . '/funlands-jwks.json';
    $age = is_file($cache) ? time() - filemtime($cache) : PHP_INT_MAX;
    // Refetch when older than 5 min, or on an unknown kid if the last fetch is older than 30 s.
    if ($age > 300 || ($refresh && $age > 30)) {
        $body = (string) (new Client(['timeout' => 3, 'allow_redirects' => false]))
            ->get(FUNLANDS_JWKS, ['headers' => ['Accept' => 'application/json']])->getBody();
        file_put_contents($cache, $body, LOCK_EX);
    }
    return JWK::parseKeySet(json_decode(file_get_contents($cache), true, 16, JSON_THROW_ON_ERROR));
}

function verify_set(string $token): array
{
    $parts = explode('.', $token);
    $header = count($parts) === 3 ? json_decode(JWT::urlsafeB64Decode($parts[0]), true) : null;
    if (!is_array($header) || ($header['typ'] ?? null) !== 'secevent+jwt' || ($header['alg'] ?? null) !== 'ES256'
        || empty($header['kid']) || array_intersect_key($header, array_flip(FORBIDDEN_HEADERS))) {
        throw new SetRejected('unexpected header');
    }
    $keys = funlands_jwks(false);
    if (!isset($keys[$header['kid']])) {
        $keys = funlands_jwks(true);
    }
    JWT::$leeway = 30;
    try {
        $claims = JWT::decode($token, $keys);
    } catch (UnexpectedValueException|DomainException|InvalidArgumentException $e) {
        throw new SetRejected($e->getMessage(), 0, $e);
    }
    if (($claims->iss ?? null) !== FUNLANDS_ISSUER || ($claims->aud ?? null) !== SET_AUDIENCE
        || !isset($claims->iat, $claims->jti, $claims->events) || $claims->iat < time() - 330) {
        throw new SetRejected('claims');
    }
    $events = [];
    foreach (json_decode(json_encode($claims->events), true) as $type => $body) {
        $events[] = $body + ['type' => $type];
    }
    return $events;
}

if ($_SERVER['REQUEST_METHOD'] !== 'POST' || !str_starts_with($_SERVER['CONTENT_TYPE'] ?? '', 'application/secevent
+jwt')) {
    http_response_code(415);
    exit;
}
$token = trim((string) file_get_contents('php://input', false, null, 0, 65537));
if (strlen($token) > 65536) {
    http_response_code(413);
    exit;
}
```

```php
try {
    $events = verify_set($token);
} catch (SetRejected) {
    http_response_code(401);
    exit;
} catch (GuzzleException) {
    http_response_code(503);
    exit;
}

$pdo = new PDO(getenv('DATABASE_DSN'), null, null, [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]);
$pdo->beginTransaction();
$insert = $pdo->prepare('INSERT INTO funlands_inbox (event_id, type, player_seq, payload)
                         VALUES (?, ?, ?, ?::jsonb) ON CONFLICT (event_id) DO NOTHING');
foreach ($events as $e) {
    $insert->execute([$e['event_id'], $e['type'], $e['player_seq'] ?? null, json_encode($e)]);
}
$pdo->commit();
http_response_code(202);
```

Go

```go
// webhook.go
package partner

import (
    "context"
    "database/sql"
    "encoding/json"
    "errors"
    "io"
    "mime"
    "net/http"
    "strings"
    "sync"
    "time"

    "github.com/lestrrat-go/jwx/v2/jwa"
    "github.com/lestrrat-go/jwx/v2/jwk"
    "github.com/lestrrat-go/jwx/v2/jws"
    "github.com/lestrrat-go/jwx/v2/jwt"
)

const (
    FunlandsIssuer = "https://api.sandbox.funlands.example" // production: https://api.funlands.example
    FunlandsJWKS   = "https://api.sandbox.funlands.example/.well-known/jwks.json"
    SetAudience    = "https://partner.example"
)

var errRejected = errors.New("set rejected")

type SetVerifier struct {
    cache       *jwk.Cache
    mu          sync.Mutex
    lastRefresh time.Time
}

func NewSetVerifier(ctx context.Context) (*SetVerifier, error) {
    c := jwk.NewCache(ctx)
    noRedirect := &http.Client{Timeout: 3 * time.Second,
        CheckRedirect: func(*http.Request, []*http.Request) error { return http.ErrUseLastResponse }}
    if err := c.Register(FunlandsJWKS, jwk.WithMinRefreshInterval(5*time.Minute), jwk.WithHTTPClient(noRedirect));
err != nil {
        return nil, err
    }
    return &SetVerifier{cache: c}, nil
}

// keyFor returns the Funlands key for kid; an unknown kid causes one refetch, at most every 30 s.
func (v *SetVerifier) keyFor(ctx context.Context, kid string) (jwk.Key, error) {
```

```
    set, err := v.cache.Get(ctx, FunlandsJWKS)
    if err != nil {
        return nil, err // infrastructure: 503
    }
    if key, ok := set.LookupKeyID(kid); ok {
        return key, nil
    }
    v.mu.Lock()
    defer v.mu.Unlock()
    if time.Since(v.lastRefresh) > 30*time.Second {
        v.lastRefresh = time.Now()
        if set, err = v.cache.Refresh(ctx, FunlandsJWKS); err != nil {
            return nil, err
        }
    }
    if key, ok := set.LookupKeyID(kid); ok {
        return key, nil
    }
    return nil, errRejected
}

func (v *SetVerifier) Verify(ctx context.Context, raw []byte) ([]map[string]any, error) {
    msg, err := jws.Parse(raw)
    if err != nil || len(msg.Signatures()) != 1 {
        return nil, errRejected
    }
    h := msg.Signatures()[0].ProtectedHeaders()
    if h.Type() != "secevent+jwt" || h.Algorithm() != jwa.ES256 || h.KeyID() == "" ||
        h.JWKSetURL() != "" || h.X509URL() != "" || h.JWK() != nil || h.X509CertChain() != nil || len(h.Critical())
> 0 {
        return nil, errRejected
    }
    key, err := v.keyFor(ctx, h.KeyID())
    if err != nil {
        return nil, err
    }
    tok, err := jwt.Parse(raw, jwt.WithKey(jwa.ES256, key), jwt.WithIssuer(FunlandsIssuer),
        jwt.WithAudience(SetAudience), jwt.WithAcceptableSkew(30*time.Second),
        jwt.WithRequiredClaim("iat"), jwt.WithRequiredClaim("jti"), jwt.WithRequiredClaim("events"))
    if err != nil {
        return nil, errRejected
    }
    if age := time.Since(tok.IssuedAt()); age > 5*time.Minute+30*time.Second || age < -30*time.Second {
        return nil, errRejected
    }
    rawEvents, _ := tok.Get("events")
    eventMap, ok := rawEvents.(map[string]any)
    if !ok {
        return nil, errRejected
    }
    var out []map[string]any
    for typ, body := range eventMap {
        e, ok := body.(map[string]any)
        if !ok {
            return nil, errRejected
        }
        e["type"] = typ
        out = append(out, e)
    }
    return out, nil
}

// WebhookHandler serves POST /funlands/events. db uses the pgx stdlib driver.
func WebhookHandler(v *SetVerifier, db *sql.DB) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if mt, _, _ := mime.ParseMediaType(r.Header.Get("Content-Type")); mt != "application/secevent+jwt" {
            w.WriteHeader(http.StatusUnsupportedMediaType)
            return
        }
        body, err := io.ReadAll(http.MaxBytesReader(w, r.Body, 64<<10))
        if err != nil {
```

```
            w.WriteHeader(http.StatusRequestEntityTooLarge)
            return
        }
        events, err := v.Verify(r.Context(), []byte(strings.TrimSpace(string(body))))
        switch {
        case errors.Is(err, errRejected):
            w.WriteHeader(http.StatusUnauthorized) // Funlands pauses delivery and alerts
            return
        case err != nil:
            w.WriteHeader(http.StatusServiceUnavailable) // Funlands retries
            return
        }
        tx, err := db.BeginTx(r.Context(), nil)
        if err != nil {
            w.WriteHeader(http.StatusServiceUnavailable)
            return
        }
        defer tx.Rollback()
        for _, e := range events {
            payload, _ := json.Marshal(e)
            var seq any // JSON numbers decode as float64
            if f, ok := e["player_seq"].(float64); ok {
                seq = int64(f)
            }
            if _, err := tx.ExecContext(r.Context(),
                `INSERT INTO funlands_inbox (event_id, type, player_seq, payload)
                 VALUES ($1, $2, $3, $4) ON CONFLICT (event_id) DO NOTHING`,
                e["event_id"], e["type"], seq, payload); err != nil {
                w.WriteHeader(http.StatusServiceUnavailable)
                return
            }
        }
        if err := tx.Commit(); err != nil {
            w.WriteHeader(http.StatusServiceUnavailable)
            return
        }
        w.WriteHeader(http.StatusAccepted)
    }
}
```

Spring Boot

```
// src/main/java/com/partner/funlands/FunlandsSetVerifier.java
package com.partner.funlands;

import com.nimbusds.jose.*;
import com.nimbusds.jose.jwk.source.*;
import com.nimbusds.jose.proc.*;
import com.nimbusds.jwt.*;
import com.nimbusds.jwt.proc.*;
import java.net.URL;
import java.time.Instant;
import java.util.*;
import org.springframework.stereotype.Component;

@Component
public class FunlandsSetVerifier {
    static final String ISSUER = "https://api.sandbox.funlands.example";           // production: https://api.funla
nds.example
    static final String JWKS_URL = "https://api.sandbox.funlands.example/.well-known/jwks.json";
    static final String AUDIENCE = "https://partner.example";

    public static class SetRejected extends Exception {
        public SetRejected(String message) { super(message); }
    }

    private final ConfigurableJWTProcessor<SecurityContext> processor = new DefaultJWTProcessor<>();

    public FunlandsSetVerifier() throws Exception {
        // Default builder: 5 min cache, refetch on unknown kid, rate limited, retrying.
        JWKSource<SecurityContext> keys = JWKSourceBuilder.create(new URL(JWKS_URL)).retrying(true).build();
```

```
        processor.setJWSTypeVerifier(new DefaultJOSEObjectTypeVerifier<>(new JOSEObjectType("secevent+jwt")));
        processor.setJWSKeySelector(new JWSVerificationKeySelector<>(JWSAlgorithm.ES256, keys));
        var claims = new DefaultJWTClaimsVerifier<SecurityContext>(AUDIENCE,
            new JWTClaimsSet.Builder().issuer(ISSUER).build(), Set.of("iat", "jti", "events"));
        claims.setMaxClockSkew(30);
        processor.setJWTClaimsSetVerifier(claims);
    }

    /** Returns one map per event; throws KeySourceException when the JWKS cannot be fetched. */
    public List<Map<String, Object>> verify(String token) throws SetRejected, KeySourceException {
        try {
            JWSHeader h = JWSObject.parse(token).getHeader();
            if (h.getJWKURL() != null || h.getX509CertURL() != null || h.getJWK() != null
                || h.getX509CertChain() != null || h.getCriticalParams() != null || h.getKeyID() == null) {
                throw new SetRejected("unexpected header");
            }
            JWTClaimsSet c = processor.process(token, null);
            Instant iat = c.getIssueTime().toInstant();
            if (iat.isBefore(Instant.now().minusSeconds(330)) || iat.isAfter(Instant.now().plusSeconds(30))) {
                throw new SetRejected("stale or future iat");
            }
            List<Map<String, Object>> events = new ArrayList<>();
            for (var entry : c.getJSONObjectClaim("events").entrySet()) {
                @SuppressWarnings("unchecked")
                var body = new HashMap<>((Map<String, Object>) entry.getValue());
                body.put("type", entry.getKey());
                events.add(body);
            }
            return events;
        } catch (KeySourceException e) {
            throw e;                                    // infrastructure: 503
        } catch (java.text.ParseException | BadJOSEException | JOSEException | ClassCastException e) {
            throw new SetRejected(e.getMessage());
        }
    }
}

// src/main/java/com/partner/funlands/FunlandsWebhookController.java
package com.partner.funlands;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.nimbusds.jose.KeySourceException;
import java.util.*;
import org.springframework.http.*;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.transaction.support.TransactionTemplate;
import org.springframework.web.bind.annotation.*;

@RestController
public class FunlandsWebhookController {
    private final FunlandsSetVerifier verifier;
    private final JdbcTemplate jdbc;
    private final TransactionTemplate tx;
    private final ObjectMapper json;

    public FunlandsWebhookController(FunlandsSetVerifier verifier, JdbcTemplate jdbc,
                                     TransactionTemplate tx, ObjectMapper json) {
        this.verifier = verifier; this.jdbc = jdbc; this.tx = tx; this.json = json;
    }

    @PostMapping(value = "/funlands/events", consumes = "application/secevent+jwt")
    public ResponseEntity<Void> receive(@RequestBody String body) throws Exception {
        if (body.length() > 65_536) return ResponseEntity.status(HttpStatus.PAYLOAD_TOO_LARGE).build();
        List<Map<String, Object>> events;
        try {
            events = verifier.verify(body.trim());
        } catch (FunlandsSetVerifier.SetRejected e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();          // pause + alert
        } catch (KeySourceException e) {
            return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).build();   // retry
        }
```

```
        tx.executeWithoutResult(status -> events.forEach(e -> jdbc.update(
            "INSERT INTO funlands_inbox (event_id, type, player_seq, payload) VALUES (?, ?, ?, ?::jsonb) "
                + "ON CONFLICT (event_id) DO NOTHING",
            e.get("event_id"), e.get("type"), e.get("player_seq"), write(e))));
        return ResponseEntity.accepted().build();
    }

    private String write(Map<String, Object> e) {
        try { return json.writeValueAsString(e); } catch (Exception ex) { throw new IllegalStateException(ex); }
    }
}
```

ASP.NET Core

```
// Funlands/FunlandsSetVerifier.cs  (register as singleton)
using System.Text;
using System.Text.Json;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;

public sealed class SetRejectedException(string message) : Exception(message);

public sealed class FunlandsSetVerifier(HttpClient http)
{
    private const string Issuer = "https://api.sandbox.funlands.example";            // production: https://api.fun
lands.example
    private const string JwksUrl = "https://api.sandbox.funlands.example/.well-known/jwks.json";
    private const string Audience = "https://partner.example";
    private static readonly string[] ForbiddenHeaders = ["jku", "x5u", "jwk", "x5c", "crit"];

    private readonly JsonWebTokenHandler _handler = new();
    private readonly SemaphoreSlim _lock = new(1, 1);
    private IList<SecurityKey> _keys = [];
    private DateTime _fetchedAt = DateTime.MinValue;

    // Cache 5 min; an unknown kid forces one refetch, at most every 30 s.
    private async Task<IList<SecurityKey>> KeysAsync(bool unknownKid)
    {
        var age = DateTime.UtcNow - _fetchedAt;
        if (age < TimeSpan.FromMinutes(5) && !(unknownKid && age > TimeSpan.FromSeconds(30))) return _keys;
        await _lock.WaitAsync();
        try
        {
            var json = await http.GetStringAsync(JwksUrl);   // 3 s timeout, no redirects (see Program.cs)
            _keys = new JsonWebKeySet(json).GetSigningKeys();
            _fetchedAt = DateTime.UtcNow;
            return _keys;
        }
        finally { _lock.Release(); }
    }

    /// <exception cref="SetRejectedException">token rejected (answer 401)</exception>
    /// <exception cref="HttpRequestException">JWKS unreachable (answer 503)</exception>
    public async Task<List<Dictionary<string, JsonElement>>> VerifyAsync(string token)
    {
        JsonWebToken jwt;
        try { jwt = _handler.ReadJsonWebToken(token); }
        catch (Exception) { throw new SetRejectedException("malformed token"); }
        if (jwt.Typ != "secevent+jwt" || jwt.Alg != "ES256" || string.IsNullOrEmpty(jwt.Kid) ||
            ForbiddenHeaders.Any(name => jwt.TryGetHeaderValue<object>(name, out _)))
            throw new SetRejectedException("unexpected header");

        var keys = await KeysAsync(unknownKid: false);
        if (!keys.Any(k => k.KeyId == jwt.Kid)) keys = await KeysAsync(unknownKid: true);

        var result = await _handler.ValidateTokenAsync(token, new TokenValidationParameters
        {
            ValidIssuer = Issuer,
            ValidAudience = Audience,
            IssuerSigningKeys = keys,
            ValidAlgorithms = ["ES256"],
```

```
            ValidTypes = ["secevent+jwt"],
            RequireExpirationTime = false,              // a SET carries no exp
            ClockSkew = TimeSpan.FromSeconds(30),
        });
        if (!result.IsValid) throw new SetRejectedException(result.Exception?.Message ?? "invalid");

        var iat = jwt.IssuedAt;                         // DateTime.MinValue when absent
        if (iat < DateTime.UtcNow.AddSeconds(-330) || iat > DateTime.UtcNow.AddSeconds(30) || string.IsNullOrEmpty
(jwt.Id))
            throw new SetRejectedException("iat/jti");

        using var payload = JsonDocument.Parse(Base64UrlEncoder.Decode(jwt.EncodedPayload));
        if (!payload.RootElement.TryGetProperty("events", out var events) || events.ValueKind != JsonValueKind.Obje
ct)
            throw new SetRejectedException("events");
        var list = new List<Dictionary<string, JsonElement>>();
        foreach (var e in events.EnumerateObject())
        {
            var body = e.Value.Deserialize<Dictionary<string, JsonElement>>()!;
            body["type"] = JsonSerializer.SerializeToElement(e.Name);
            list.Add(body);
        }
        return list;
    }
}

// Program.cs
using System.Text;
using System.Text.Json;
using Npgsql;

// Singleton, so the key cache survives between requests.
builder.Services.AddSingleton(new FunlandsSetVerifier(
    new HttpClient(new HttpClientHandler { AllowAutoRedirect = false }) { Timeout = TimeSpan.FromSeconds(3) }));
builder.Services.AddSingleton(_ => NpgsqlDataSource.Create(builder.Configuration.GetConnectionString("Partner")!));

app.MapPost("/funlands/events", async (HttpRequest request, FunlandsSetVerifier verifier, NpgsqlDataSource db) =>
{
    if (request.ContentType?.StartsWith("application/secevent+jwt") != true) return Results.StatusCode(415);
    if (request.ContentLength > 65_536) return Results.StatusCode(413);
    using var reader = new StreamReader(request.Body, Encoding.ASCII);
    var token = (await reader.ReadToEndAsync()).Trim();

    List<Dictionary<string, JsonElement>> events;
    try { events = await verifier.VerifyAsync(token); }
    catch (SetRejectedException) { return Results.StatusCode(401); }        // pause + alert
    catch (HttpRequestException) { return Results.StatusCode(503); }        // retry
    catch (TaskCanceledException) { return Results.StatusCode(503); }

    await using var conn = await db.OpenConnectionAsync();
    await using var tx = await conn.BeginTransactionAsync();
    foreach (var e in events)
    {
        await using var cmd = new NpgsqlCommand(
            "INSERT INTO funlands_inbox (event_id, type, player_seq, payload) VALUES ($1, $2, $3, $4::jsonb) " +
            "ON CONFLICT (event_id) DO NOTHING", conn, tx);
        cmd.Parameters.AddWithValue(e["event_id"].GetString()!);
        cmd.Parameters.AddWithValue(e["type"].GetString()!);
        cmd.Parameters.AddWithValue(e.TryGetValue("player_seq", out var seq) ? seq.GetInt64() : (object)DBNull.Valu
e);
        cmd.Parameters.AddWithValue(JsonSerializer.Serialize(e));
        await cmd.ExecuteNonQueryAsync();
    }
    await tx.CommitAsync();
    return Results.StatusCode(202);
});
```

cURL

```
# What a delivery looks like (Funlands -> your registered webhook URL):
#   POST /funlands/events HTTP/1.1
```

```
#   Host: partner.example
#   Content-Type: application/secevent+jwt
#
#   eyJhbGciOiJFUzI1NiIsInR5cCI6InNlY2V2ZW50K2p3dCIsImtpZCI6ImZsLXNhbmRib3gtMjAyNi0xMSJ9.eyJ...

# Fetch the Funlands public keys your verifier uses:
curl -sS https://api.sandbox.funlands.example/.well-known/jwks.json | jq '.keys[] | {kid, kty, crv, alg, use}'

# Replay a captured sandbox delivery against your own receiver within 5 minutes of its iat
# (expect 202, then 202 again for the duplicate event_id; after 5 minutes expect 401):
curl -i https://partner.example/funlands/events \
  -H 'Content-Type: application/secevent+jwt' \
  --data-binary @captured-set.jwt
```

پاسخ شما و تکرار ارسال

پاسخ شما	رفتار فان‌لندز
`200` یا `202`	تحویل‌شده حساب میشه. برای `event_id` تکراری هم همین رو برگردونید. کد دیگه، حتی `204`، موفق حساب نمیشه.
`400`	خطای دائمی توی شکل داده. تکرار نمیشه و به هر دو تیم هشدار داده میشه. بعد از رفع، رویداد replay میشه.
`401` یا `403`	مشکل امضا یا مجوز. ارسال به محیط شما متوقف میشه، رویدادها توی صف می‌مونن و هشدار داده میشه. وقتی به فان‌لندز خبر بدید که مشکل رفع شده، ارسال از همون‌جا ادامه پیدا می‌کنه؛ رویدادهای مونده دوباره فرستاده میشن و پنجره ۷۲ ساعته از نو شروع میشه.
`429`	دوباره فرستاده میشه و هدر `Retry-After` رعایت میشه.
`5xx`، timeout، خطای شبکه، ریدایرکت و هر کد دیگه (مثل `404` یا `408`)	گذرا حساب میشه و با فاصله‌های بیشتر و بیشتر تکرار میشه. ریدایرکت دنبال نمیشه.

- مهلت جواب ۵ ثانیه‌ست و جواب دیرتر timeout حساب میشه.
- فاصله تلاش‌ها بعد از تلاش اول: ۱ دقیقه، ۵ دقیقه، ۳۰ دقیقه، ۲ ساعت، ۶ ساعت و ۲۴ ساعت، هر کدوم با حداکثر ۱۰٪ jitter. بعدش هر ۲۴ ساعت یک بار تا ۷۲ ساعت از اولین تلاش؛ آخرین تلاش درست سر ۷۲ ساعته.
- بعد از هر خطای گذرا، بقیه رویدادهای محیط شما هم حدود یک دقیقه صبر می‌کنن تا گیرنده‌ای که مشکل داره زیر فشار صف پر نره.
- رویدادی که تا ۷۲ ساعت تحویل نشه به dead letter میره. بعد از رفع مشکل می‌تونید replayش رو از فان‌لندز بخواید. replay همون `event_id` رو داره، هر تلاشش یک SET تازه‌ست و پنجره ۷۲ ساعته تازه می‌گیره. هر replay حسابرسی (audit) میشه.
- خرابی Webhook شما بازی رو متوقف نمی‌کنه و روی شریک‌های دیگه اثری نداره.
- آخرین `player_seq` هر بازیکن رو نگه دارید. اگه عددی جا افتاد، رویداد رو «ناقص» علامت بزنید و جای خالی رو از API بازیابی پر کنید. فقط مرتب کردن یک دسته بر اساس `player_seq` کافی نیست.

دریافت با pull: `GET /partner/v1/events`

اگه سرور شما مدتی قطع بوده یا توی `player_seq` جای خالی دیدید، رویدادها رو از این feed بخونید. feed فقط رویدادهای خود شما رو به ترتیب انتشار برمی‌گردونه. توکنش رو مثل API گزارش با scope `events:read` بگیرید.

HTTP

```
GET /partner/v1/events?cursor=c_9mQ2xR7tLw&limit=100 HTTP/1.1
Host: api.sandbox.funlands.example
Authorization: Bearer <token with scope events:read>
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "items": [
    {
      "event_id": "evt_0193a0e1b2c37d4e8f50a1b2c3d4e5f6",
      "type": "https://api.funlands.example/events/game.completed",
      "schema_version": 1,
      "subject": { "format": "iss_sub", "iss": "https://partner.example", "sub": "u_7f3a9c2e5b1d4e08" },
      "player_seq": 42,
      "occurred_at": "2026-11-05T09:31:12Z",
      "game_id": "star-runner",
      "game_version": null,
      "score": 1250,
      "duration_ms": null,
      "result": "win"
    }
  ],
  "next_cursor": "c_9mQ2xR7tLx",
  "has_more": false,
  "generated_at": "2026-11-05T09:35:00Z"
}
```

- بار اول بدون `cursor` صدا بزنید تا از قدیمی‌ترین رویدادی که نگه داشته شده شروع بشه. بعد هر بار `next_cursor` پاسخ قبلی رو بفرستید تا `has_more` برابر `false` بشه.
- `next_cursor` حتی وقتی `has_more` برابر `false` هست هم برمی‌گرده (صفحه خالی همون cursor ارسالی رو برمی‌گردونه). نگهش دارید تا دفعه بعد از همون‌جا ادامه بدید.
- ترتیب feed نسبت به commit امنه: هر رویداد وقتی توی feed دیده میشه که همه تراکنش‌هایی که می‌تونستن رویدادی قبل از اون بذارن تموم شده باشن (معمولا کمتر از یک ثانیه). پس رویدادی که دیرتر commit شده هیچ‌وقت پشت cursor شما جا نمی‌افته.
- `cursor` رو opaque در نظر بگیرید: ذخیره‌اش کنید و دستش نزنید (حداکثر ۲۵۶ کاراکتر). `limit` پیش‌فرض ۵۰ و حداکثر ۲۰۰ هست. cursor یا limit نامعتبر `400 PARTNER_REPORT_INVALID` می‌گیره.
- هر آیتم همون فیلدهای رویداد داخل SET رو داره، به‌اضافه `type` (همون URI). dedupe رو مثل Webhook روی `event_id` انجام بدید. هر دو مسیر می‌تونن توی یک جدول بنویسن.
- feed دست‌کم به اندازه پنجره ۷۲ ساعته تکرار به‌اضافه زمان قطعی توافق‌شده نگه داشته میشه. عدد دقیق توی قرارداد داده میاد.

API v۷ گزارش‌ها

APIهای `/partner/v1` سرور به سرور هستن. توکنشون رو هیچ‌وقت به مرورگر یا اپ موبایل ندید و لاگش نکنید. access token بازی برای این APIها کار نمی‌کنه.

گرفتن توکن: `POST /oauth/token`

توکن با grant نوع `client_credentials` و احراز هویت `private_key_jwt` گرفته میشه: یک JWT کوتاه‌عمر (client assertion) با کلید خصوصی خودتون امضا می‌کنید و فان‌لندز اون رو با JWKS ثبت‌شده برای `client_id` شما چک می‌کنه. رمز یا client secret در کار نیست.

فیلد assertion	قاعده
هدر `typ`	`JWT`
`iss` و `sub`	هر دو برابر `client_id`
`aud`	دقیقا نشانی token endpoint همین محیط
`iat` ، `exp`	`exp` حداکثر ۶۰ ثانیه بعد از `iat`
`jti`	یکتا. تکرارش رد میشه.

JSON

```
// header
{ "alg": "ES256", "typ": "JWT", "kid": "partner-api-sandbox-2026-11" }

// claims
{
  "iss": "cli_sandbox_0000example",
  "sub": "cli_sandbox_0000example",
  "aud": "https://api.sandbox.funlands.example/oauth/token",
  "iat": 1793871000,
  "exp": 1793871060,
  "jti": "3f6a2d9c-1b7e-4c50-8a4d-6e2f9b1c7d05"
}
```

HTTP

```
POST /oauth/token HTTP/1.1
Host: api.sandbox.funlands.example
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&scope=reports:read players:read
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCIsImtpZCI6InBhcnRuZXItYXBpLXNhbmRib3gtMjAyNi0xMSJ9.eyJ...

HTTP/1.1 200 OK
Cache-Control: no-store
Content-Type: application/json

{ "access_token": "eyJhbGciOiJFUzI1NiIs...", "token_type": "Bearer", "expires_in": 900, "scope": "reports:read play
ers:read" }
```

scope	دسترسی
`reports:read`	`/metrics/summary` ، `/metrics/daily` ، `/games`
`players:read`	`/players/{external_sub}` و `/players/{external_sub}/activity`
`events:read`	`/events`

- چند scope رو با فاصله از هم جدا کنید. فقط scopeهایی صادر میشن که برای `client_id` شما ثبت شدن.
- توکن ۹۰۰ ثانیه عمر داره و refresh token نداره. تا کمی قبل از تموم شدنش کشش کنید و بعد توکن تازه بگیرید.
- خطاهای این endpoint قالب استاندارد OAuth (RFC 6749) رو دارن، نه پاکت خطای فان‌لندز: `invalid_request` ، `invalid_client` (assertion نامعتبر، کلید ناشناخته یا `jti` تکراری)، `unauthorized_client` ، `unsupported_grant_type` و `invalid_scope` .
- اگه از سقف نرخ درخواست توکن برای `client_id` خودتون رد بشید، جواب `429` با `error` برابر `temporarily_unavailable` و هدر `Retry-After` هست. مشکل گذرا سمت فان‌لندز هم `503` با همین `error` و `Retry-After` میده. توی هر دو حالت تا `Retry-After` صبر کنید.
- توکن صادرشده یک JWT با هدر `typ` برابر `at+jwt` و `aud` برابر `partner-api` هست که فان‌لندز با کلید خودش امضاش می‌کنه و `iss` ش همون issuer فان‌لندز همین محیطه. فقط با `Authorization: Bearer` بفرستیدش و منطق خودتون رو روی محتواش نسازید، چون فقط فان‌لندز چکش می‌کنه.

HTTP

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
WWW-Authenticate: Bearer error="invalid_client"

{ "error": "invalid_client", "error_description": "client assertion rejected" }

HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 30

{ "error": "temporarily_unavailable", "error_description": "rate limit exceeded" }

# Decoded access token: header { "alg": "ES256", "typ": "at+jwt", "kid": "<funlands kid>" },
# claims { "iss": "https://api.sandbox.funlands.example", "aud": "partner-api", "sub": "<environment id>",
#          "client_id", "partner_id", "scope", "iat", "exp" (iat + 900), "jti" }
```

نمونه: گرفتن توکن با private_key_jwt و خوندن خلاصه آمار

NestJS

```
// src/funlands/funlands-reports.client.ts
import { Injectable, OnModuleInit } from '@nestjs/common';
import { importPKCS8, SignJWT } from 'jose';
import { randomUUID } from 'node:crypto';
import { readFile } from 'node:fs/promises';

const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';                // production: https://api.funlands.e
xample
const TOKEN_ENDPOINT = `${FUNLANDS_BASE}/oauth/token`;
const CLIENT_ID = 'cli_sandbox_0000example';                                 // from your registration sheet
const ASSERTION_KID = 'partner-api-sandbox-2026-11';

@Injectable()
export class FunlandsReportsClient implements OnModuleInit {
  private privateKey!: Awaited<ReturnType<typeof importPKCS8>>;
  private token?: { value: string; expiresAt: number };

  async onModuleInit() {
```

```
    this.privateKey = await importPKCS8(await readFile(`keys/${ASSERTION_KID}.pem`, 'utf8'), 'ES256');
  }

  private async clientAssertion() {
    const now = Math.floor(Date.now() / 1000);
    return new SignJWT({})
      .setProtectedHeader({ alg: 'ES256', typ: 'JWT', kid: ASSERTION_KID })
      .setIssuer(CLIENT_ID)
      .setSubject(CLIENT_ID)
      .setAudience(TOKEN_ENDPOINT)
      .setIssuedAt(now)
      .setExpirationTime(now + 60)
      .setJti(randomUUID())
      .sign(this.privateKey);
  }

  // 900 s token, no refresh token: request a new one shortly before expiry.
  private async accessToken(): Promise<string> {
    if (this.token && Date.now() < this.token.expiresAt) return this.token.value;
    const res = await fetch(TOKEN_ENDPOINT, {
      method: 'POST',
      headers: { 'Content-Type': 'application/x-www-form-urlencoded', Accept: 'application/json' },
      body: new URLSearchParams({
        grant_type: 'client_credentials',
        scope: 'reports:read',
        client_assertion_type: 'urn:ietf:params:oauth:client-assertion-type:jwt-bearer',
        client_assertion: await this.clientAssertion(),
      }),
      signal: AbortSignal.timeout(5000),
    });
    const body = await res.json();
    if (!res.ok) throw new Error(`token endpoint ${res.status}: ${body.error}`);
    this.token = { value: body.access_token, expiresAt: Date.now() + (body.expires_in - 60) * 1000 };
    return this.token.value;
  }

  async metricsSummary(from: string, to: string) {
    const url = new URL('/partner/v1/metrics/summary', FUNLANDS_BASE);
    url.search = new URLSearchParams({ from, to }).toString();
    const res = await fetch(url, {
      headers: { Authorization: `Bearer ${await this.accessToken()}`, Accept: 'application/json' },
      signal: AbortSignal.timeout(10_000),
    });
    if (!res.ok) throw new Error(`summary ${res.status}: ${(await res.json()).code}`);
    return res.json();   // { range, metrics, generated_at, data_lag_seconds, metric_definition_version }
  }
}
```

Express

```
// funlands-reports.mjs
import { importPKCS8, SignJWT } from 'jose';
import { randomUUID } from 'node:crypto';
import { readFile } from 'node:fs/promises';

const FUNLANDS_BASE = 'https://api.sandbox.funlands.example';                // production: https://api.funlands.example
const TOKEN_ENDPOINT = `${FUNLANDS_BASE}/oauth/token`;
const CLIENT_ID = 'cli_sandbox_0000example';
const ASSERTION_KID = 'partner-api-sandbox-2026-11';
const privateKey = await importPKCS8(await readFile(`keys/${ASSERTION_KID}.pem`, 'utf8'), 'ES256');

let cached;

async function clientAssertion() {
  const now = Math.floor(Date.now() / 1000);
  return new SignJWT({})
    .setProtectedHeader({ alg: 'ES256', typ: 'JWT', kid: ASSERTION_KID })
    .setIssuer(CLIENT_ID)
    .setSubject(CLIENT_ID)
    .setAudience(TOKEN_ENDPOINT)
```

```
    .setIssuedAt(now)
    .setExpirationTime(now + 60)
    .setJti(randomUUID())
    .sign(privateKey);
}

async function accessToken() {
  if (cached && Date.now() < cached.expiresAt) return cached.value;
  const res = await fetch(TOKEN_ENDPOINT, {
    method: 'POST',
    headers: { 'Content-Type': 'application/x-www-form-urlencoded', Accept: 'application/json' },
    body: new URLSearchParams({
      grant_type: 'client_credentials',
      scope: 'reports:read',
      client_assertion_type: 'urn:ietf:params:oauth:client-assertion-type:jwt-bearer',
      client_assertion: await clientAssertion(),
    }),
    signal: AbortSignal.timeout(5000),
  });
  const body = await res.json();
  if (!res.ok) throw new Error(`token endpoint ${res.status}: ${body.error}`);
  cached = { value: body.access_token, expiresAt: Date.now() + (body.expires_in - 60) * 1000 };
  return cached.value;
}

export async function metricsSummary(from, to) {
  const url = new URL('/partner/v1/metrics/summary', FUNLANDS_BASE);
  url.search = new URLSearchParams({ from, to }).toString();
  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${await accessToken()}`, Accept: 'application/json' },
    signal: AbortSignal.timeout(10_000),
  });
  if (!res.ok) throw new Error(`summary ${res.status}: ${(await res.json()).code}`);
  return res.json();
}

// usage (server side only; never send this token to a browser):
// const summary = await metricsSummary('2026-10-01', '2026-10-31');
```

FastAPI

```
# app/funlands_reports.py
import time
import uuid
from pathlib import Path

import httpx
import jwt
from cryptography.hazmat.primitives import serialization

FUNLANDS_BASE = "https://api.sandbox.funlands.example"            # production: https://api.funlands.example
TOKEN_ENDPOINT = f"{FUNLANDS_BASE}/oauth/token"
CLIENT_ID = "cli_sandbox_0000example"
ASSERTION_KID = "partner-api-sandbox-2026-11"
PRIVATE_KEY = serialization.load_pem_private_key(Path(f"keys/{ASSERTION_KID}.pem").read_bytes(), password=None)


class FunlandsReports:
    def __init__(self) -> None:
        self._http = httpx.AsyncClient(base_url=FUNLANDS_BASE, timeout=10, follow_redirects=False)
        self._token: str | None = None
        self._expires_at = 0.0

    def _client_assertion(self) -> str:
        now = int(time.time())
        claims = {"iss": CLIENT_ID, "sub": CLIENT_ID, "aud": TOKEN_ENDPOINT,
                  "iat": now, "exp": now + 60, "jti": str(uuid.uuid4())}
        return jwt.encode(claims, PRIVATE_KEY, algorithm="ES256", headers={"typ": "JWT", "kid": ASSERTION_KID})

    async def _access_token(self) -> str:
        if self._token and time.time() < self._expires_at:
```

```
            return self._token
        res = await self._http.post("/oauth/token", data={
            "grant_type": "client_credentials",
            "scope": "reports:read",
            "client_assertion_type": "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
            "client_assertion": self._client_assertion(),
        })
        body = res.json()
        if res.status_code != 200:
            raise RuntimeError(f"token endpoint {res.status_code}: {body.get('error')}")
        self._token = body["access_token"]
        self._expires_at = time.time() + body["expires_in"] - 60   # 900 s, no refresh token
        return self._token

    async def metrics_summary(self, date_from: str, date_to: str) -> dict:
        res = await self._http.get(
            "/partner/v1/metrics/summary",
            params={"from": date_from, "to": date_to},
            headers={"Authorization": f"Bearer {await self._access_token()}", "Accept": "application/json"},
        )
        if res.status_code != 200:
            raise RuntimeError(f"summary {res.status_code}: {res.json().get('code')}")
        return res.json()


# usage inside an async route or job:
# reports = FunlandsReports()
# summary = await reports.metrics_summary("2026-10-01", "2026-10-31")
```

Django

```
# partner/funlands_reports.py
import time
import uuid

import jwt
import requests

from .funlands import FUNLANDS_BASE, private_key

TOKEN_ENDPOINT = f"{FUNLANDS_BASE}/oauth/token"
CLIENT_ID = "cli_sandbox_0000example"
ASSERTION_KID = "partner-api-sandbox-2026-11"

_session = requests.Session()
_token: dict = {"value": None, "expires_at": 0.0}


def _client_assertion() -> str:
    now = int(time.time())
    claims = {"iss": CLIENT_ID, "sub": CLIENT_ID, "aud": TOKEN_ENDPOINT,
              "iat": now, "exp": now + 60, "jti": str(uuid.uuid4())}
    return jwt.encode(claims, private_key(ASSERTION_KID), algorithm="ES256",
                      headers={"typ": "JWT", "kid": ASSERTION_KID})


def _access_token() -> str:
    if _token["value"] and time.time() < _token["expires_at"]:
        return _token["value"]
    res = _session.post(TOKEN_ENDPOINT, timeout=5, allow_redirects=False, data={
        "grant_type": "client_credentials",
        "scope": "reports:read",
        "client_assertion_type": "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
        "client_assertion": _client_assertion(),
    })
    body = res.json()
    if res.status_code != 200:
        raise RuntimeError(f"token endpoint {res.status_code}: {body.get('error')}")
    _token.update(value=body["access_token"], expires_at=time.time() + body["expires_in"] - 60)
    return _token["value"]
```

```python
def metrics_summary(date_from: str, date_to: str) -> dict:
    res = _session.get(
        f"{FUNLANDS_BASE}/partner/v1/metrics/summary",
        params={"from": date_from, "to": date_to},
        headers={"Authorization": f"Bearer {_access_token()}", "Accept": "application/json"},
        timeout=10, allow_redirects=False,
    )
    if res.status_code != 200:
        raise RuntimeError(f"summary {res.status_code}: {res.json().get('code')}")
    return res.json()

# usage in a view, management command or Celery task:
# summary = metrics_summary("2026-10-01", "2026-10-31")
```

Laravel

```php
<?php
// app/Funlands/FunlandsReports.php
namespace App\Funlands;

use Firebase\JWT\JWT;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Str;

final class FunlandsReports
{
    private const CLIENT_ID = 'cli_sandbox_0000example';
    private const ASSERTION_KID = 'partner-api-sandbox-2026-11';

    private function tokenEndpoint(): string
    {
        return config('funlands.base') . '/oauth/token';
    }

    private function clientAssertion(): string
    {
        $now = time();
        $claims = [
            'iss' => self::CLIENT_ID,
            'sub' => self::CLIENT_ID,
            'aud' => $this->tokenEndpoint(),
            'iat' => $now,
            'exp' => $now + 60,
            'jti' => (string) Str::uuid(),
        ];
        $pem = file_get_contents(storage_path('keys/' . self::ASSERTION_KID . '.pem'));

        return JWT::encode($claims, $pem, 'ES256', self::ASSERTION_KID, ['typ' => 'JWT']);
    }

    private function accessToken(): string
    {
        // 900 s token, no refresh token: cache it for 840 s.
        return Cache::remember('funlands:report-token', 840, function () {
            return Http::asForm()->acceptJson()->timeout(5)->withoutRedirecting()
                ->post($this->tokenEndpoint(), [
                    'grant_type' => 'client_credentials',
                    'scope' => 'reports:read',
                    'client_assertion_type' => 'urn:ietf:params:oauth:client-assertion-type:jwt-bearer',
                    'client_assertion' => $this->clientAssertion(),
                ])->throw()->json('access_token');
        });
    }

    public function metricsSummary(string $from, string $to): array
    {
        return Http::withToken($this->accessToken())->acceptJson()->timeout(10)->withoutRedirecting()
            ->get(config('funlands.base') . '/partner/v1/metrics/summary', ['from' => $from, 'to' => $to])
            ->throw()->json();
```

```
    }
}

// usage (job, command or controller):
// $summary = app(\App\Funlands\FunlandsReports::class)->metricsSummary('2026-10-01', '2026-10-31');
```

PHP

```php
<?php
// src/FunlandsReports.php
declare(strict_types=1);

use Firebase\JWT\JWT;
use GuzzleHttp\Client;

final class FunlandsReports
{
    private const BASE = 'https://api.sandbox.funlands.example';           // production: https://api.funlands.exam
ple
    private const TOKEN_ENDPOINT = self::BASE . '/oauth/token';
    private const CLIENT_ID = 'cli_sandbox_0000example';
    private const ASSERTION_KID = 'partner-api-sandbox-2026-11';

    private Client $http;
    private ?string $token = null;
    private int $expiresAt = 0;

    public function __construct(private readonly string $privateKeyPem)
    {
        $this->http = new Client(['timeout' => 10, 'allow_redirects' => false]);
    }

    private function clientAssertion(): string
    {
        $now = time();
        return JWT::encode([
            'iss' => self::CLIENT_ID,
            'sub' => self::CLIENT_ID,
            'aud' => self::TOKEN_ENDPOINT,
            'iat' => $now,
            'exp' => $now + 60,
            'jti' => bin2hex(random_bytes(16)),
        ], $this->privateKeyPem, 'ES256', self::ASSERTION_KID, ['typ' => 'JWT']);
    }

    private function accessToken(): string
    {
        if ($this->token !== null && time() < $this->expiresAt) {
            return $this->token;
        }
        $res = $this->http->post(self::TOKEN_ENDPOINT, [
            'headers' => ['Accept' => 'application/json'],
            'form_params' => [
                'grant_type' => 'client_credentials',
                'scope' => 'reports:read',
                'client_assertion_type' => 'urn:ietf:params:oauth:client-assertion-type:jwt-bearer',
                'client_assertion' => $this->clientAssertion(),
            ],
        ]);
        $body = json_decode((string) $res->getBody(), true, 16, JSON_THROW_ON_ERROR);
        $this->token = $body['access_token'];
        $this->expiresAt = time() + $body['expires_in'] - 60;
        return $this->token;
    }

    public function metricsSummary(string $from, string $to): array
    {
        $res = $this->http->get(self::BASE . '/partner/v1/metrics/summary', [
            'headers' => ['Authorization' => 'Bearer ' . $this->accessToken(), 'Accept' => 'application/json'],
            'query' => ['from' => $from, 'to' => $to],
        ]);
```

```php
        return json_decode((string) $res->getBody(), true, 32, JSON_THROW_ON_ERROR);
    }
}

// usage (Guzzle throws on 4xx/5xx by default):
// $reports = new FunlandsReports(file_get_contents(__DIR__ . '/../keys/partner-api-sandbox-2026-11.pem'));
// $summary = $reports->metricsSummary('2026-10-01', '2026-10-31');
```

Go

```go
// reports.go
package partner

import (
    "encoding/json"
    "fmt"
    "net/http"
    "net/url"
    "strings"
    "sync"
    "time"

    "github.com/lestrrat-go/jwx/v2/jwk"
)

const (
    TokenEndpoint = FunlandsBase + "/oauth/token"
    ClientID      = "cli_sandbox_0000example"
    AssertionKID  = "partner-api-sandbox-2026-11"
)

type ReportsClient struct {
    key       jwk.Key // LoadPrivateKey(AssertionKID)
    http      *http.Client
    mu        sync.Mutex
    token     string
    expiresAt time.Time
}

func NewReportsClient(key jwk.Key) *ReportsClient {
    return &ReportsClient{key: key, http: &http.Client{Timeout: 10 * time.Second,
        CheckRedirect: func(*http.Request, []*http.Request) error { return http.ErrUseLastResponse }}}
}

func (c *ReportsClient) accessToken() (string, error) {
    c.mu.Lock()
    defer c.mu.Unlock()
    if c.token != "" && time.Now().Before(c.expiresAt) {
        return c.token, nil
    }
    now := time.Now().Unix()
    assertion, err := SignJWT(c.key, "JWT", map[string]any{
        "iss": ClientID, "sub": ClientID, "aud": TokenEndpoint,
        "iat": now, "exp": now + 60, "jti": randomB64(16),
    })
    if err != nil {
        return "", err
    }
    form := url.Values{
        "grant_type":            {"client_credentials"},
        "scope":                 {"reports:read"},
        "client_assertion_type": {"urn:ietf:params:oauth:client-assertion-type:jwt-bearer"},
        "client_assertion":      {assertion},
    }
    res, err := c.http.Post(TokenEndpoint, "application/x-www-form-urlencoded", strings.NewReader(form.Encode()))
    if err != nil {
        return "", err
    }
    defer res.Body.Close()
    var body struct {
        AccessToken string `json:"access_token"`
```

```
        ExpiresIn   int    `json:"expires_in"`
        Error       string `json:"error"`
    }
    if err := json.NewDecoder(res.Body).Decode(&body); err != nil {
        return "", err
    }
    if res.StatusCode != http.StatusOK {
        return "", fmt.Errorf("token endpoint %d: %s", res.StatusCode, body.Error)
    }
    c.token = body.AccessToken
    c.expiresAt = time.Now().Add(time.Duration(body.ExpiresIn-60) * time.Second)
    return c.token, nil
}

func (c *ReportsClient) MetricsSummary(from, to string) (map[string]any, error) {
    token, err := c.accessToken()
    if err != nil {
        return nil, err
    }
    q := url.Values{"from": {from}, "to": {to}}
    req, _ := http.NewRequest(http.MethodGet, FunlandsBase+"/partner/v1/metrics/summary?"+q.Encode(), nil)
    req.Header.Set("Authorization", "Bearer "+token)
    req.Header.Set("Accept", "application/json")
    res, err := c.http.Do(req)
    if err != nil {
        return nil, err
    }
    defer res.Body.Close()
    var out map[string]any
    if err := json.NewDecoder(res.Body).Decode(&out); err != nil {
        return nil, err
    }
    if res.StatusCode != http.StatusOK {
        return nil, fmt.Errorf("summary %d: %v", res.StatusCode, out["code"])
    }
    return out, nil
}
```

Spring Boot

```
// src/main/java/com/partner/funlands/FunlandsReportsClient.java
package com.partner.funlands;

import static com.partner.funlands.FunlandsConfig.*;

import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.ECDSASigner;
import com.nimbusds.jwt.*;
import java.time.Instant;
import java.util.*;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;
import org.springframework.util.*;
import org.springframework.web.client.RestClient;

@Service
public class FunlandsReportsClient {
    private static final String CLIENT_ID = "cli_sandbox_0000example";
    private static final String ASSERTION_KID = "partner-api-sandbox-2026-11";

    private final RestClient http = RestClient.builder().baseUrl(BASE).build();
    private final JWSSigner signer;
    private String token;
    private Instant expiresAt = Instant.EPOCH;

    public FunlandsReportsClient() throws Exception {
        this.signer = new ECDSASigner(PemKeys.privateKey(ASSERTION_KID));
    }

    private String clientAssertion() throws JOSEException {
        Instant now = Instant.now();
```

```
        var jwt = new SignedJWT(
            new JWSHeader.Builder(JWSAlgorithm.ES256).type(JOSEObjectType.JWT).keyID(ASSERTION_KID).build(),
            new JWTClaimsSet.Builder()
                .issuer(CLIENT_ID).subject(CLIENT_ID).audience(TOKEN_ENDPOINT)
                .issueTime(Date.from(now)).expirationTime(Date.from(now.plusSeconds(60)))
                .jwtID(UUID.randomUUID().toString()).build());
        jwt.sign(signer);
        return jwt.serialize();
    }

    private synchronized String accessToken() throws JOSEException {
        if (token != null && Instant.now().isBefore(expiresAt)) return token;
        MultiValueMap<String, String> form = new LinkedMultiValueMap<>();
        form.add("grant_type", "client_credentials");
        form.add("scope", "reports:read");
        form.add("client_assertion_type", "urn:ietf:params:oauth:client-assertion-type:jwt-bearer");
        form.add("client_assertion", clientAssertion());
        Map<?, ?> body = http.post().uri("/oauth/token")
            .contentType(MediaType.APPLICATION_FORM_URLENCODED).body(form)
            .retrieve().body(Map.class);                       // 4xx/5xx throw RestClientResponseException
        token = (String) body.get("access_token");
        expiresAt = Instant.now().plusSeconds(((Number) body.get("expires_in")).longValue() - 60);
        return token;
    }

    public Map<?, ?> metricsSummary(String from, String to) throws JOSEException {
        return http.get()
            .uri(b -> b.path("/partner/v1/metrics/summary").queryParam("from", from).queryParam("to", to).build())
            .header("Authorization", "Bearer " + accessToken())
            .accept(MediaType.APPLICATION_JSON)
            .retrieve().body(Map.class);
    }
}
```

ASP.NET Core

```
// Funlands/FunlandsReportsClient.cs  (register as singleton)
using System.Net.Http.Headers;
using System.Net.Http.Json;
using System.Text.Json;

public sealed class FunlandsReportsClient
{
    private const string ClientId = "cli_sandbox_0000example";
    private const string AssertionKid = "partner-api-sandbox-2026-11";

    private readonly HttpClient _http = new(new HttpClientHandler { AllowAutoRedirect = false })
    {
        BaseAddress = new Uri(FunlandsConfig.Base),
        Timeout = TimeSpan.FromSeconds(10),
    };
    private readonly PartnerSigner _signer;
    private readonly Microsoft.IdentityModel.Tokens.SigningCredentials _credentials;
    private readonly SemaphoreSlim _lock = new(1, 1);
    private string? _token;
    private DateTime _expiresAt;

    public FunlandsReportsClient(PartnerSigner signer)
    {
        _signer = signer;
        _credentials = signer.Credentials(AssertionKid);
    }

    private async Task<string> AccessTokenAsync()
    {
        await _lock.WaitAsync();
        try
        {
            if (_token is not null && DateTime.UtcNow < _expiresAt) return _token;
            var assertion = _signer.Sign("JWT", _credentials, ClientId, FunlandsConfig.TokenEndpoint, 60, new()
            {
```

```
                ["sub"] = ClientId,
                ["jti"] = Guid.NewGuid().ToString(),
            });
            using var res = await _http.PostAsync("/oauth/token", new FormUrlEncodedContent(new Dictionary<string,
string>
            {
                ["grant_type"] = "client_credentials",
                ["scope"] = "reports:read",
                ["client_assertion_type"] = "urn:ietf:params:oauth:client-assertion-type:jwt-bearer",
                ["client_assertion"] = assertion,
            }));
            var body = await res.Content.ReadFromJsonAsync<JsonElement>();
            if (!res.IsSuccessStatusCode)
                throw new HttpRequestException($"token endpoint {(int)res.StatusCode}: {body.GetProperty("erro
r")}");
            _token = body.GetProperty("access_token").GetString()!;
            _expiresAt = DateTime.UtcNow.AddSeconds(body.GetProperty("expires_in").GetInt32() - 60);
            return _token;
        }
        finally { _lock.Release(); }
    }

    public async Task<JsonElement> MetricsSummaryAsync(string from, string to)
    {
        using var req = new HttpRequestMessage(HttpMethod.Get,
            $"/partner/v1/metrics/summary?from={Uri.EscapeDataString(from)}&to={Uri.EscapeDataString(to)}");
        req.Headers.Authorization = new AuthenticationHeaderValue("Bearer", await AccessTokenAsync());
        req.Headers.Accept.Add(new MediaTypeWithQualityHeaderValue("application/json"));
        using var res = await _http.SendAsync(req);
        var body = await res.Content.ReadFromJsonAsync<JsonElement>();
        if (!res.IsSuccessStatusCode)
            throw new HttpRequestException($"summary {(int)res.StatusCode}: {body.GetProperty("code")}");
        return body;
    }
}
```

cURL

```
# 1) Exchange a signed client assertion for a 900 s report token.
#    CLIENT_ASSERTION: header {"alg":"ES256","typ":"JWT","kid":"partner-api-sandbox-2026-11"}
#    claims: iss = sub = client_id, aud = token endpoint, exp = iat + 60, fresh jti
curl -sS https://api.sandbox.funlands.example/oauth/token \
  -d grant_type=client_credentials \
  -d scope=reports:read \
  -d client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer \
  --data-urlencode "client_assertion=${CLIENT_ASSERTION}"

# {"access_token":"eyJhbGciOiJFUzI1NiIs...","token_type":"Bearer","expires_in":900,"scope":"reports:read"}

# 2) Call the report API with it.
curl -sS 'https://api.sandbox.funlands.example/partner/v1/metrics/summary?from=2026-10-01&to=2026-10-31' \
  -H "Authorization: Bearer ${ACCESS_TOKEN}" \
  -H 'Accept: application/json' | jq
```

endpointهای گزارش

مسیر	scope	پارامترها	محتوا
`GET /partner/v1/metrics/summary`	`reports:read`	`from`، `to`	خلاصه بازه: میانگین DAU، MAU، شروع و تکمیل بازی، زمان بازی
`GET /partner/v1/metrics/daily`	`reports:read`	`from`، `to`	یک ردیف برای هر روز، با تاریخ محلی و بازه UTC همون روز
`GET /partner/v1/games`	`reports:read`	`from`، `to`	آمار به تفکیک `game_id` و `game_version`
`GET /partner/v1/players/{external_sub}`	`players:read`	–	حداقل پروفایل و جمع‌ها
`GET /partner/v1/players/{external_sub}/activity`	`players:read`	`from`، `to`، `cursor`، `limit`	فعالیت بازیکن با صفحه‌بندی cursor

- `from` و `to` تاریخ محلی با قالب `YYYY-MM-DD` هستن، هر دو سر شامل، به وقت `Asia/Tehran`. منطقه زمانی جدا برای هر شریک هنوز ثبت نمیشه. پاسخ بازه UTC معادل رو هم برمی‌گردونه.
- `external_sub` همون `sub` یه که توی launch فرستادید و توی مسیر باید URL-encode بشه. اگه این `sub` مال شما نباشه، وجود نداشته باشه یا فقط توی محیط دیگه شما بازی کرده باشه، جواب `404 PARTNER_PLAYER_NOT_FOUND` هست و این حالت‌ها از هم قابل تشخیص نیستن.
- `from`، `to`، `cursor` یا `limit` نامعتبر (قالب اشتباه، تاریخ ناممکن، بازه برعکس یا بیشتر از ۳۶۶ روز، cursor دستکاری‌شده، یا `limit` بیرون از ۱ تا ۲۰۰) جواب `400 PARTNER_REPORT_INVALID` می‌گیره. `limit` پیش‌فرض ۵۰ هست.
- هر جستجوی بازیکن با `client_id`، زمان و نتیجه حسابرسی (audit) میشه.
- همه پاسخ‌ها سه فیلد مشترک دارن: `generated_at` (زمان ساخت پاسخ به UTC)، `data_lag_seconds` (اینکه داده چقدر از زمان واقعی عقبه) و `metric_definition_version`. بقیه فیلدها به مسیر بستگی دارن.
- `data_lag_seconds` فعلا حدود ۲۱۰۰ ثانیه‌ست: یک نشست بازی باز حداکثر ۳۰ دقیقه بعد بسته میشه و بستنش تا ۵ دقیقه دیگه طول می‌کشه، پس عددهای این بازه اخیر هنوز ممکنه عوض بشن.
- چند فیلد فعلا همیشه `null` هستن، چون داده‌شون هنوز ثبت نمیشه: `game_version` در `/games`، `age_band` در پروفایل بازیکن و `result` در رویدادهای `game.completed` فعالیت. فعالیت از نشست‌های بازی ساخته میشه و نشست بازی برد و باخت رو نگه نمی‌داره. `result` رویداد Webhook و feed جداست و می‌تونه مقدار داشته باشه (انواع رویداد). این فیلدها توی پاسخ می‌مونن و وقتی داده‌شون برسه پر میشن. کد شما باید `null` رو قبول کنه.

خلاصه

JSON

```
// GET /partner/v1/metrics/summary?from=2026-10-01&to=2026-10-31       scope: reports:read
{
  "range": {
    "from": "2026-10-01",
    "to": "2026-10-31",
    "timezone": "Asia/Tehran",
    "utc_start": "2026-09-30T20:30:00Z",
    "utc_end": "2026-10-31T20:30:00Z"
  },
```

```
  "metrics": {
    "dau_average": 1840,
    "mau": 12950,
    "game_starts": 48210,
    "game_completions": 31077,
    "completion_rate": 0.6446,
    "playtime_seconds": 9123400
  },
  "generated_at": "2026-11-05T08:00:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

روزانه

JSON

```
// GET /partner/v1/metrics/daily?from=2026-11-01&to=2026-11-02          scope: reports:read
{
  "timezone": "Asia/Tehran",
  "items": [
    { "date": "2026-11-01", "utc_start": "2026-10-31T20:30:00Z", "utc_end": "2026-11-01T20:30:00Z",
      "dau": 1902, "game_starts": 1655, "game_completions": 1071, "playtime_seconds": 301220 },
    { "date": "2026-11-02", "utc_start": "2026-11-01T20:30:00Z", "utc_end": "2026-11-02T20:30:00Z",
      "dau": 1877, "game_starts": 1610, "game_completions": 1049, "playtime_seconds": 296480 }
  ],
  "generated_at": "2026-11-05T08:00:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

به تفکیک بازی

JSON

```
// GET /partner/v1/games?from=2026-10-01&to=2026-10-31                  scope: reports:read
// game_version is null for now: Game does not record the played build yet.
{
  "range": {
    "from": "2026-10-01",
    "to": "2026-10-31",
    "timezone": "Asia/Tehran",
    "utc_start": "2026-09-30T20:30:00Z",
    "utc_end": "2026-10-31T20:30:00Z"
  },
  "items": [
    { "game_id": "star-runner", "game_version": null, "players": 6120,
      "game_starts": 21944, "game_completions": 15003, "completion_rate": 0.6837, "playtime_seconds": 4012300 },
    { "game_id": "color-lab", "game_version": null, "players": 4388,
      "game_starts": 13870, "game_completions": 8015, "completion_rate": 0.5779, "playtime_seconds": 2650100 }
  ],
  "generated_at": "2026-11-05T08:00:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

بازیکن

JSON

```
// GET /partner/v1/players/u_7f3a9c2e5b1d4e08                           scope: players:read
// age_band is null for now: the launch age_band is not stored on the player yet.
{
  "external_sub": "u_7f3a9c2e5b1d4e08",
  "status": "active",
  "age_band": null,
  "first_seen_at": "2026-09-12T15:20:04Z",
```

```
  "last_seen_at": "2026-11-05T09:31:12Z",
  "totals": { "sessions": 58, "game_starts": 214, "game_completions": 151, "playtime_seconds": 40310 },
  "generated_at": "2026-11-05T09:35:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

فعالیت بازیکن

JSON

```
// GET /partner/v1/players/u_7f3a9c2e5b1d4e08/activity?from=2026-11-01&to=2026-11-05&limit=2   scope: players:read
// result is null for now; score and duration_ms appear on game.completed only.
{
  "items": [
    { "event_id": "gc_0193a0e1b2c37d4e8f50a1b2c3d4e5f6", "type": "game.completed", "game_id": "star-runner",
      "occurred_at": "2026-11-05T09:31:12Z", "score": 1250, "duration_ms": 130000, "result": null },
    { "event_id": "gs_0193a0e1b2c37d4e8f50a1b2c3d4e5f6", "type": "game.started", "game_id": "star-runner",
      "occurred_at": "2026-11-05T09:29:02Z" }
  ],
  "next_cursor": "c_Lw7tR2xQ9m",
  "has_more": true,
  "generated_at": "2026-11-05T09:35:00Z",
  "data_lag_seconds": 2100,
  "metric_definition_version": 1
}
```

تعریف شاخص‌ها

این تعریف‌ها توی API و پنل شریک یکی هستن و با `metric_definition_version` نسخه‌بندی میشن. همه شمارش‌ها فقط بازیکن‌ها و فعالیت خود شما رو در بر می‌گیرن. گزارش‌ها از نشست‌های بازی فان‌لندز ساخته میشن، نه از رویدادهای Webhook: شروع هر نشست بازی یک `game.started` حساب میشه و نشستی که کامل تموم بشه یک `game.completed`. برای همین DAU و نرخ تکمیل همین الان هم عدد دارن، با اینکه رویداد Webhook `game.started` هنوز فرستاده نمیشه. روزها به وقت `Asia/Tehran` شمرده میشن.

شاخص	تعریف
DAU (`dau`)	تعداد `sub` یکتا که توی یک روز تقویمی به وقت `Asia/Tehran` دست‌کم یک نشست بازی شروع کردن (یک `game.started` به معنای بالا).
`dau_average`	میانگین DAU روی همه روزهای بازه. روز بدون بازی صفر حساب میشه.
MAU (`mau`)	تعداد `sub` یکتا با دست‌کم یک `game.started` توی ماه تقویمی‌ای که `to` توشه، محدود به بازه درخواست: از `from` یا روز اول اون ماه، هر کدوم دیرتر بود، تا آخر روز `to`.
`game_starts`، `game_completions`	تعداد نشست‌های بازی که توی بازه شروع شدن، و تعداد اون‌هایی که کامل تموم شدن.
`completion_rate`	`game_completions` تقسیم بر `game_starts` توی همون گروه شروع (cohort). اگه شروعی نباشه `null` هست.
`playtime_seconds`	از بازه‌های نشست بازی حساب میشه، با حذف هم‌پوشانی و با سقف برای هر بازه، و به روز شروع نشست نسبت داده میشه. جمع ساده زمان‌هایی که کلاینت گزارش کرده نیست.
ماندگاری روز N	سهم بازیکن‌های گروه «اولین بازی» یک روز که N روز بعد هم `game.started` داشتن. توی پنل شریک نشون داده میشه و توی `v1` endpoint جدا نداره.

- رویدادهای دیررس ممکنه عددهای روزهای اخیر رو تا آخر دوره `data_lag_seconds` عوض کنن. برای گزارش نهایی یک روز، بعد از تموم شدن اون روز و گذشتن این تاخیر بخونید.

- هیچ شمارنده‌ای مبنای صورتحساب نیست، مگر اینکه فرمول و روال تطبیقش جدا تعریف و امضا شده باشه.

۸ خطاها

همه endpointهای فان‌لندز، جز `/oauth/token` که قالب OAuth داره، خطا رو با این پاکت برمی‌گردونن:

JSON

```
// 401 Unauthorized            response header  X-Correlation-Id: req_01JD3X8Q2M5N7P9R
{
  "state": false,
  "code": "TOKEN_EXPIRED",
  "message": "زمان اعتبار توکن گذشته است",
  "correlation_id": "req_01JD3X8Q2M5N7P9R"
}

// 404 Not Found   (unknown sub, another partner's sub, or a sub of your other environment)
{
  "state": false,
  "code": "PARTNER_PLAYER_NOT_FOUND",
  "message": "بازیکن یافت نشد",
  "correlation_id": "req_01JD3X9A4B6C8D0E"
}

// 429 Too Many Requests   (also sent as the Retry-After: 30 header)
{
  "state": false,
  "code": "RATE_LIMITED",
  "message": "تعداد درخواست‌ها بیش از حد مجاز است؛ کمی بعد دوباره تلاش کنید",
  "correlation_id": "req_01JD3XB2C4D6E8F0",
  "retry_after_seconds": 30
}
```

- کد شما فقط بر اساس `code` تصمیم بگیره. `message` متن فارسی برای نشون دادن به کاربره و ممکنه عوض بشه.
- شناسه درخواست توی هدر پاسخ `X-Correlation-Id` میاد و توی خطاهای endpointهای شریک (`/v1/partner/*`، `/v1/session/bootstrap` و `/partner/v1/*`) همون مقدار توی بدنه با نام `correlation_id` هم هست. اون رو توی لاگ خودتون نگه دارید و موقع تماس با پشتیبانی بفرستید تا درخواست رو توی لاگ‌های فان‌لندز پیدا کنیم.
- `retry_after_seconds` فقط توی خطاهای گذرا میاد و همون مقدار توی هدر `Retry-After` هم هست.
- پیام خطا هیچ‌وقت جزئیات امضا، وجود یا نبود کاربر یا داده شریک دیگه رو نشون نمیده.

HTTP	code	معنی	چه کار کنید
403	ORIGIN_NOT_ALLOWED	هدر Origin فرم launch ثبت نشده، خالیه یا null هست.	Origin دقیق صفحه رو (با scheme و پورت) ثبت کنید و فرم رو از همون صفحه بفرستید.
401	PARTNER_UNKNOWN	iss با هیچ شریک فعالی توی این محیط نمی‌خونه.	issuer رو دقیقا طبق برگ شریک همین محیط بذارید.
403	PARTNER_SUSPENDED	شریک یا محیط شما غیرفعاله.	با فان‌لندز تماس بگیرید و تکرار خودکار نکنید.
401	TOKEN_TYPE_INVALID	typ اشتباهه یا نیست، یا یکی از هدرهای ممنوع (jku ، x5u ، jwk ، x5c ، crit) توی توکن هست.	typ درست هر توکن رو بذارید و هدرهای ممنوع رو بردارید.
401	ALG_NOT_ALLOWED	الگوریتم با الگوریتم ثبت‌شده محیط نمی‌خونه.	با الگوریتم ثبت‌شده امضا کنید و عوض کردن الگوریتم رو جدا هماهنگ کنید.
401	KEY_NOT_FOUND	kid حتی بعد از خوندن دوباره JWKS پیدا نشد، یا کلید ابطال شده.	کلید رو دست‌کم ۲۴ ساعت قبل از استفاده منتشر کنید. اگه ابطال شده، با کلید تازه امضا کنید.
401	SIGNATURE_INVALID	امضا با کلید عمومی منتشرشده تایید نمیشه.	جفت بودن کلید خصوصی و عمومی و kid رو چک کنید.
401	AUDIENCE_MISMATCH	aud با audience این محیط و این نوع توکن نمی‌خونه، مثلا توکن sandbox روی production.	audience درست رو از برگ شریک همین محیط بذارید.
401	TOKEN_EXPIRED	exp گذشته، عمر توکن بیشتر از حد مجازه یا iat بیشتر از ۳۰ ثانیه در آینده‌ست. access token بازی هم وقتی نشستش تموم شده یا شریکش غیرفعال شده همین کد رو می‌گیره.	ساعت سرور رو با NTP هماهنگ کنید و توکن رو درست قبل از ارسال بسازید.
401	TOKEN_REPLAYED	این jti قبلا استفاده شده.	برای هر ورود یا خروج یک توکن تازه با jti تازه بسازید.
400	CLAIMS_INVALID	claim اجباری نیست یا شکلش درست نیست، game فرم و توکن فرق دارن، یا verifier با challenge نمی‌خونه.	جدول claimها و محاسبه challenge رو چک کنید.
409	LAUNCH_PAYLOAD_CHANGED	همون jti با محتوای متفاوت فرستاده شده.	jti رو هیچ‌وقت بین دو توکن مشترک نکنید.
403	PLAYER_SUSPENDED	بازیکن در فان‌لندز تعلیق شده.	پیام پشتیبانی نشون بدید و تکرار خودکار نکنید.
400	BOOTSTRAP_INVALID	کد bootstrap اشتباه، منقضی یا مصرف‌شده‌ست، یا verifier نمی‌خونه.	launch تازه بسازید.
401	TOKEN_REUSE	refresh tokenی که قبلا چرخیده دوباره استفاده شده و همه نشست‌های این بازیکن باطل شدن. refresh token ناشناخته یا منقضی TOKEN_EXPIRED می‌گیره.	launch تازه بسازید.

401	SESSION_REVOKED	refresh نشستی رسیده که با خروج یا غیرفعال شدن شریک یا محیط بسته شده ولی هنوز حذف نشده، از جمله اولین refresh نشستی که بدون launch باز شده و شریکش غیرفعال شده. bootstrap هم وقتی این کد رو می‌گیره که خروج همون `sid` زودتر رسیده باشه. بعد از حذف نشست، refresh همون `TOKEN_EXPIRED` می‌گیره (تمدید نشست).	مثل `TOKEN_EXPIRED` روی refresh رفتار کنید: نشست تموم شده و اگه کاربر هنوز توی اپ شما وارده، launch تازه بسازید.
403	PARTNER_TOKEN_MISMATCH	درخواست یا handshakeای که توکن داره کلید شریک دیگه‌ای همراهش داره، یا توکن پلتفرم همراه کلید شریک اومده.	روی درخواست‌هایی که توکن دارن کلید شریک نفرستید، یا فقط کلید همون شریک رو بفرستید. تکرار خودکار نکنید.
401	PARTNER_KEY_INVALID	`x-partner-api-key` یا `X-Partner-Public-Key` ناشناخته‌ست، چرخیده یا قالبش درست نیست. سکرت اشتباه `server_to_server` جواب `PARTNER_SECRET_INVALID` می‌گیره.	کلید فعلی برگ شریک رو بذارید، یا روی درخواست‌های توکن‌دار اصلا نفرستید. این خطا با refresh درست نمیشه.
400	PARTNER_REPORT_INVALID	پارامتر گزارش (`from`، `to`، `cursor` یا `limit`) نامعتبره.	قالب `YYYY-MM-DD`، بازه حداکثر ۳۶۶ روزه، `limit` بین ۱ و ۲۰۰ و فقط cursorی که از پاسخ قبلی گرفتید رو بفرستید.
404	PARTNER_PLAYER_NOT_FOUND	این `external_sub` توی این محیط بازیکن شما نیست (ناشناخته‌ست، مال شریک دیگه‌ست یا فقط توی محیط دیگه شماست).	`sub` و محیط توکن رو چک کنید و تکرار خودکار نکنید.
403	SCOPE_NOT_ALLOWED	توکن API گزارش scope لازم برای این endpoint رو نداره.	توکن رو با scope درست بگیرید. اگه scope برای شما ثبت نشده، درخواستش رو بدید.
429	RATE_LIMITED	از سقف نرخ درخواست رد شدید.	بعد از `Retry-After` دوباره تلاش کنید و درخواست‌ها رو یکنواخت‌تر بفرستید.
503	UPSTREAM_UNAVAILABLE	مشکل گذرا سمت فان‌لندز.	بعد از `Retry-After` دوباره تلاش کنید و به کاربر پیام قابل بازیابی نشون بدید.

سقف نرخ هر شریک و هر endpoint توی برگ شریک میاد. روی `429` همیشه `Retry-After` رو رعایت کنید.

۹ قبل از Production

چک‌لیست

- [ ] جفت‌کلیدهای production جدا از sandbox ساخته شدن و کلید خصوصی فقط توی KMS یا HSM هست.
- [ ] JWKS production روی HTTPS و بدون ریدایرکت در دسترسه، فقط فیلدهای عمومی داره و `kid` ها یکتا هستن.
- [ ] برنامه چرخش کلید و مالکش مشخصه.
- [ ] تماس اضطراری امنیتی ثبت شده و روال ابطال اضطراری یک بار تمرین شده.
- [ ] ساعت همه سرورهای امضاکننده با NTP هماهنگه.
- [ ] توکن launch فقط سمت سرور و فقط بعد از احراز هویت ساخته میشه و هیچ توکن، verifier یا کدی توی URL یا لاگ نیست.
- [ ] `sub` مستعار و پایداره و `sid` شناسه‌ای جدا از کوکی نشسته.
- [ ] Originهای production ثبت شدن، فرم launch به Origin بازی production میره و CSP صفحه شما `frame-src` و `form-action` رو به همون Origin محدود کرده.
- [ ] صفحه بازی بعد از bootstrap فقط `Authorization: Bearer` (و توی WebSocket فقط `auth.token`) می‌فرسته، هیچ کلید یا سکرت شریکی توی صفحه بازی یا مرورگر نیست، و درخواستی که توکن داره با کلید شریک دیگه `403 PARTNER_TOKEN_MISMATCH` می‌گیره.
- [ ] ورود دوباره با postMessage و پیام خطای قابل فهم پیاده شده.
- [ ] خروج سرور به سرور با صف تکرار پیاده شده.
- [ ] Webhook رویداد رو ماندگار و بدون تکرار ذخیره می‌کنه، تا ۵ ثانیه جواب میده و `401` و `503` رو درست از هم جدا می‌کنه.
- [ ] بازیابی از `/partner/v1/events` برای پر کردن جای خالی `player_seq` پیاده شده.
- [ ] توکن API گزارش فقط روی سروره و هیچ‌وقت به مرورگر یا لاگ نمی‌رسه.
- [ ] قرارداد داده امضا شده: رده‌های سنی، فهرست رویدادها، تعریف شاخص‌ها، منطقه زمانی، مدت نگهداری و روال حذف.
- [ ] همه سناریوهای sandbox پایین با موفقیت اجرا شدن.

سناریوهای تست در sandbox

سناریو	نتیجه مورد انتظار
launch معتبر	بازی باز میشه و `player.created` و `session.started` می‌رسن.
launch دوم با همون `sub`	همون بازیکن، فقط با یک `session.started` تازه.
همون `sub` روی production بعد از بازی در sandbox	یک بازیکن جدا در production. گزارش‌ها و `/players` هر محیط فقط بازیکن همون محیط رو نشون میدن.
دو launch هم‌زمان برای یک `sub` تازه	دقیقا یک بازیکن ساخته میشه.
فرستادن توکن بعد از ۱۲۰ ثانیه	`401 TOKEN_EXPIRED`
توکن sandbox روی production، یا audience خروج توی launch	`401 AUDIENCE_MISMATCH`
بدون `typ`، یا با `alg=none` یا `HS256`	`TOKEN_TYPE_INVALID` یا `ALG_NOT_ALLOWED`
هدر `jku` توی توکن	`401 TOKEN_TYPE_INVALID`
`kid` منتشرنشده	`401 KEY_NOT_FOUND`
عوض کردن یک بایت از امضا	`401 SIGNATURE_INVALID`
فرم از Origin ثبت‌نشده یا بدون Origin	`403 ORIGIN_NOT_ALLOWED`
verifier اشتباه	`400 CLAIMS_INVALID`
فرستادن دوباره همون فرم با verifier درست / بدون verifier	`200` با کد تازه / `401 TOKEN_REPLAYED`
همون `jti` با `game` متفاوت	`409 LAUNCH_PAYLOAD_CHANGED`
reload صفحه بازی	`funlands:relaunch_requested` می‌رسه و بازی با launch تازه ادامه پیدا می‌کنه.
خروج کاربر از اپ شما	خروج `200` می‌گیره، بازی باز همون لحظه قطع میشه، access token و refresh با `401` `TOKEN_EXPIRED` رد میشن و صفحه بازی `funlands:relaunch_requested` می‌فرسته.
درخواست API بازی و اتصال WebSocket فقط با access token و بدون کلید شریک	قبول میشن و فضای شریک از توکن خونده میشه.
فرم launch به نشانی پایه API به‌جای Origin بازی (وقتی این دو فرق دارن)	`funlands:launch_failed` با `ORIGIN_NOT_ALLOWED`
تکرار همون logout token	`200` (قبلا انجام شده)
Webhook شما `503` برگردونه	رویداد با همون `event_id` و `jti` تازه دوباره می‌رسه.
Webhook شما `401` برگردونه	ارسال متوقف میشه و هشدار داده میشه. بعد از رفع مشکل و خبر دادن به فان‌لندز ادامه پیدا می‌کنه.
رسیدن یک رویداد برای دو بار	فقط یک ردیف توی صندوق ورودی، و هر دو بار `202`.
قطع Webhook برای یک ساعت	همه رویدادها با تکرار یا از feed `/events` بازیابی میشن و جای خالی‌ای توی `player_seq` نمی‌مونه.
client assertion با `jti` تکراری یا `aud` اشتباه	`401 invalid_client`
صدا زدن `/players` با توکنی که فقط `reports:read` داره	`403 SCOPE_NOT_ALLOWED`

صدا زدن API گزارش با access token بازی	`401`
چرخش کلید طبق روال	توکن‌های K1 و K2 توی دوره هم‌پوشانی قبول میشن.
تمرین ابطال اضطراری	بعد از خبر دادن، توکن با `kid` ابطال‌شده با `KEY_NOT_FOUND` رد میشه.

۱۰ سوال‌های رایج

چرا امضا با کلید مشترک (HMAC) نیست؟

هر کسی که کلید مشترک رو داشته باشه می‌تونه به‌جای هر دو طرف توکن بسازه، و چرخش و ابطالش هم هماهنگی دستی می‌خواد. با امضای نامتقارن، کلید خصوصی هیچ‌وقت از سیستم شما بیرون نمیره و ما فقط کلید عمومی رو از نشانی ثبت‌شده می‌خونیم.

کاربرهای ما با OIDC وارد میشن. می‌تونیم همون id_token رو بفرستیم؟

نه. `id_token` برای اپ شما صادر شده و audience و `typ` دیگه‌ای داره. توکن launch رو جدا با `typ` برابر `funlands-launch+jwt` بسازید. اگه واقعا لازم دارید `id_token` قبول بشه، این یک پروفایل جدا با audience و `nonce` خودشه که باید جدا توافق بشه.

اپ Android یا iOS ما WebView داره. همین روش کار می‌کنه؟

نسخه فعلی فقط برای وبه. WebView معمولا `Origin` قابل اعتماد نمی‌فرسته و درخواست بدون Origin رد میشه. ورود از اپ موبایل یا TV یک تحویل سرور به سرور جدا لازم داره که برای هر پلتفرم جدا تایید میشه و بعدا منتشر میشه.

ساعت سرورهای ما کمی جلو یا عقبه. مشکلی پیش میاد؟

تا ۳۰ ثانیه اختلاف قبول میشه. بیشتر از اون، توکن‌ها با `TOKEN_EXPIRED` رد میشن. ساعت سرورهای امضاکننده رو با NTP هماهنگ کنید.

چرا هر بار که یک رویداد دوباره فرستاده میشه jti عوض میشه؟

چون هر تلاش ارسال یک SET تازه با `iat` تازه‌ست. اگه همون JWT اول بعد از ۷۲ ساعت دوباره می‌اومد، گیرنده‌ای که SET قدیمی‌تر از ۵ دقیقه رو رد می‌کنه نمی‌تونست قبولش کنه. برای همین dedupe رو همیشه روی `event_id` انجام بدید.

Webhook ما چند ساعت از کار افتاد. رویدادها از دست رفتن؟

نه. فان‌لندز تا ۷۲ ساعت دوباره تلاش می‌کنه. اگه بیشتر طول کشید یا توی `player_seq` جای خالی دیدید، از `GET /partner/v1/events` با cursor بخونید. اگه رویدادی به dead letter رفته، replayش رو درخواست کنید.

کاربر صفحه رو reload کرد و بازی دوباره ورود خواست. طبیعیه؟

بله. توکن‌های بازی فقط توی حافظه صفحه نگه داشته میشن. صفحه بازی داخل iframe پیام `funlands:relaunch_requested` می‌فرسته و صفحه شما باید یک launch تازه از سرور خودتون بگیره. پیشرفت بازیکن روی سرور فان‌لندز ذخیره‌ست و از بین نمیره.

می‌تونیم شناسه کاربر خودمون رو مستقیم به‌عنوان sub بفرستیم؟

اگه اون شناسه جای دیگه‌ای هم دیده میشه (مثلا توی URLها یا سرویس‌های دیگه)، نه. برای هر کاربر یک شناسه تصادفی پایدار مخصوص فان‌لندز بسازید و نگهش دارید، یا HMAC شناسه داخلی رو با یک کلید مخفی حساب کنید. این شناسه نباید بعدا عوض بشه، چون `sub` تازه یعنی بازیکن تازه.

صفحه بازی برای درخواست‌هاش کلید شریک لازم داره؟

نه. بعد از bootstrap فقط access token بازیکن فرستاده میشه و شریک از خود توکن خونده میشه. این برای شریک‌های `server_to_server` هم درسته، پس سکرتشون هیچ‌وقت به مرورگر نمی‌رسه (فضای شریک در صفحه بازی).